

DeepSeek Harness 핸드북

dsh-handbook · 한국어 완역판

DeepSeek가 공식 오픈소스로 공개한 Agent 런타임을 0부터 1까지 — 초보자를 위한 백과사전: 설치, 사용, 플러그인 개발, 튜닝, 보안, 예산까지.

16개 챕터 + 부록(용어집 / 패키지 / Benchmark)

docs/에서 동기화 · dsh 0.1.0-rc.6 · 2026-08-17

<https://github.com/Electricitysheep/dsh-handbook>

목차

- 01 제 1 장: DeepSeek Harness 이해하기
- 02 제 2 장: 5분 빠른 시작
- 03 제 3 장: profile과 플러그인 시스템
- 04 제 4 장: 플러그인 개발 실전
- 05 제 5 장: dsh가 당신을 도울 수 있는 것 — 활용 시나리오 전체 조망
- 06 제 6 장: 심화와 성능 튜닝
- 07 제 7 장: 생태계와 리소스
- 08 제 8 장: 도구와 컨텍스트 시스템
- 09 제 9 장: MCP, 서브에이전트, 워크플로우
- 10 제 10 장: 복잡한 실전 사례(dsh가 실제로 해낸 것)
- 11 제 11 장: 미래 전망 — dsh와 Agent 생태계의 진화 예측
- 12 제 12 장: 알려진 한계와 경계 — 솔직한 버전
- 13 제 13 장: 보안과 샌드박스 모델
- 14 제 14 장: 캐시와 비용 엔지니어링
- 15 제 15 장: DSH 생태계 전체 리포트 — 4000+ 플러그인과 2400개 디스커션 스레드가 말해주는 것
- 16 제 16 장: Claude Code에서 이전하기
- 부록 A: 용어집과 명령어 빠른 참조
- 부록 B: 공식 패키지 빠른 참조 총정리(@deepseek-ai/*)
- 부록 C: 동일 모델 × 다른 Agent 실측 비교(Benchmark)

제 1 장: DeepSeek Harness 이해하기

본 장의 목표: 완전히 사전 지식이 없는 상태에서 DeepSeek Harness(dsh)에 대한 완전한 인지를 구축합니다 — 그것이 무엇인지, 왜 배울 가치가 있는지, 주류 Agent와 무엇이 다른지, 언제 써야 하는지. 이 장을 다 읽으면 어떤 사전 지식 없이도 dsh가 전체 AI 개발 도구 지형에서 어디에 위치하는지 이해할 수 있습니다.

TL;DR(본 장 핵심, 30초 버전)

1. **dsh = Agent의 레고 베이스플레이트**: 공식 오픈소스(MIT) 런타임으로, 모든 능력이 플러그인이며 자유롭게 조립할 수 있습니다
2. **harness = 모델 바깥의 엔지니어링 레이어**: 세션, 도구, 컨텍스트, 루프 제어 — 모델이 저장소 안에서 실제로 일할 수 있게 만듭니다
3. **Claude Code와의 차이**: Claude Code는 "완성차"이고, dsh는 "베이스플레이트 + 블록"입니다 — 커스터마이징 가능한 범위가 완전히 다릅니다
4. **생태계 창구**: 2026-08-13 오픈소스 공개, 그 이전 한국어(중국어) 튜토리얼은 전무 — 지금 진입하는 것은 초기 단계입니다
5. **누가 써야 하나**: 깊은 커스터마이징/생태계 참여/CI 실행이 필요한 개발자; 바로 쓸 수 있는 것을 원한다면 Claude Code를 선택하세요

<details><summary>본 장 목차</summary>

- 1.1 먼저 세 가지 직관 세우기
- 1.2 공식 정의와 핵심 사실
- 1.3 아키텍처는 어떻게 "모든 것이 플러그인"이 되는가
- 1.4 DSH와 주류 Agent의 전면 비교
- 1.5 언제 dsh를 써야 하는가(도구 선택 결정)
- 1.6 자주 묻는 질문(FAQ)

1.1 먼저 세 가지 직관 세우기

기술을 설명하기 전에, 먼저 세 가지 비유로 직관을 세워봅시다:

① dsh는 무엇인가? — "Agent의 레고 베이스플레이트"입니다.

레고를 떠올려보세요: 공식이 베이스판과 표준 블록(런타임 + 핵심 플러그인)을 제공하면, 여러분은 자유롭게 조립합니다(플러그인 추가, 인터페이스 교체, 동작 변경). 이에 비해 Claude Code는 "완성차"에 더 가깝습니다 — 운전하기는 편하지만 엔진을 개조하려면 제조사를 찾아가야 합니다.

② 왜 "harness"라는 단어가 필요한가? — "모델 바깥을 감싸는 엔지니어링 레이어"이기 때문입니다.

모델(DeepSeek V4) 자체는 "텍스트로 답변하는" 것밖에 하지 못합니다. 모델이 여러분의 코드 저장소 안에서 실제로 일하게(파일 읽기, 명령 실행, 코드 수정, 다회차 루프) 하려면, 바깥에 한 층의 엔지니어링이 필요합니다: 세션 관리, 도구 호출, 컨텍스트 제어, 오류 복구. 이 엔지니어링 레이어를 **harness**라고 부릅니다. dsh는 DeepSeek 공식이 이 엔지니어링 레이어를 오픈소스로 공개한 제품입니다.

③ 왜 2026년에야 오픈소스로 공개했는가? — Agent가 "프로그래머블 시대"에 진입했기 때문입니다.

2025년은 "모델 성능 경쟁"(누가 더 좋은 코드를 생성하는가)의 해였고, 2026년은 "Agent 엔지니어링 경쟁"(누가 모델을 더 잘 조직해 일을 시키는가)의 해로 진입합니다. DeepSeek가 harness를 오픈소스로 공개한 전략적 의도는: "어떻게 Agent를 조직할 것인가"라는 문제를 개방된 생태계로 만드는 것입니다 — 마치 예전 Android가 오픈소스로 스마트폰 생태계를 바꾼 것처럼요.

1.2 공식 정의와 핵심 사실

한 줄 정의: DeepSeek Harness(dsh)는 DeepSeek 공식이 오픈소스로 공개한 Agent 런타임으로, "모든 것이 플러그인"(everything is a plugin) 아키텍처를 채택했으며 Cordis 플러그인 컨테이너를 기반으로 구축되었습니다.

사실	내용
오픈소스 공개일	2026-08-13
라이선스	MIT(상업적 이용 가능, 수정 가능)
언어	TypeScript(Node.js ≥ 22)
버전 라인	0.1.0-rc.x(현재 rc.6, 빠른 반복 주기, 공식이 "파괴적 변경이 있을 것"이라고 명시)
하부 구조	Cordis (https://github.com/cordiverse/cordis)(조합 가능한 플러그인 컨테이너)
내장 형태	web(Web UI) + headless(일회성 CLI)
공식 포지셔닝	공식 README 원문: "everything is a plugin"

1.3 아키텍처는 어떻게 "모든 것이 플러그인"이 되는가

```
flowchart TB
    subgraph Profile["당신의 profile(실행 가능한 형태)"]
        P1["dsh web(Web UI)"]
        P2["dsh headless(CLI)"]
        P3["커스텀 profile(TUI/데스크톱/봇...)"]
    end
    end
    subgraph Plugins["능력 레이어(능력 하나 = 플러그인 하나)"]
        L["llm: 모델 연동 + 추론 강도"]
        T["tools: 파일 쓰기/터미널/검색/스킬"]
        S["session: 세션 영속화"]
        C["client: 인터페이스(web/터미널)"]
        ST["settings: 사용자 설정"]
    end
    end
    subgraph Cordis["Cordis 플러그인 컨테이너"]
        D["의존성 주입 · 이벤트 · 생명주기"]
    end
    end
    Profile --> Cordis
    Cordis --> Plugins
    Plugins --> L
    Plugins --> T
    Plugins --> S
    Plugins --> C
    Plugins --> ST
```

레이어 뷰

당신의 profile(실행 가능한 형태)	
= bundle 스택 + 당신의 patch 레이어	
· dsh web(Web UI 형태)	
· dsh headless(CLI 형태)	
· 당신의 커스텀 profile(TUI/데스크톱/봇...)	
능력 레이어(능력 하나 = 플러그인 하나)	
· llm: 모델 연동(DeepSeek V4 계열 + 추론 강도)	
· tools: 도구(파일 쓰기/터미널/검색/스킬...)	
· session: 세션 영속화	
· client: 인터페이스(web 브라우저 반쪽 / 터미널 반쪽)	
· settings: 사용자 설정	
...(공식 패키지 60+개)	
Cordis 플러그인 컨테이너: 로딩, 의존성 주입, 이벤트, 생명주기	

반드시 알아야 할 세 가지 개념

① profile(실행 가능한 형태)

하나의 profile = `$DSH_HOME/profiles/<name>/` 디렉터리이며, 다음을 포함합니다:

- `package.json`: 플러그인 의존성 + 매니페스트(`dsh.profile.bundles`가 bundle 순서를 지정)
- `cordis.patch.yml`: 당신의 patch 레이어(플러그인 장착/오버라이드)

시작 시 순서대로 합성됩니다: 내장 bundle → profile patch → 전역 patch → `--patch` 오버라이드.

② host 반쪽 / client 반쪽(플러그인 하나, 두 개의 얼굴)

반쪽	어디서 실행되는가	무엇을 하는가
host 반쪽	Node 프로세스	도구, 서비스, 이벤트, 파일 시스템 — <code>apply(ctx)</code> 로 등록
client 반쪽	브라우저(web profile)	UI, 상호작용 — <code>package.json</code> 의 <code>dsh.client</code> 로 선언

npm 패키지 하나가 두 반쪽을 동시에 가질 수 있습니다(`exports["."] + exports["./client"]`).

③ 확장 포인트(extension point)

공식 원칙: **"Plugins, not loop changes"** — 동작을 바꾸려면 공식 훅을 먼저 쓰고, 코어를 fork하지 마세요. 자

주 쓰는 확장 포인트(이후 장에서 하나씩 실습):

- `agent/request waterfall`: 매 모델 요청 전에 설정 변경(도구 호출 속도 개선 플러그인의 훅 지점)
- `conversationEvents.register`: 대화 이벤트 구독/주입
- `ctx.slots.inject`: 인터페이스 슬롯에 UI 주입
- `settings` 서비스: 사용자가 설정 가능한 항목 등록

1.4 DSH와 주류 Agent의 전면 비교

능력 매트릭스(핵심 6개사)

항목	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
오픈소스	✅ MIT	❌ 클로즈드소	❌ 클로즈드소	✅ MIT	❌ 클로즈드소	❌ 클로즈드소

		스	스		스	스
모델 종속	모델 독립적(공식은 DeepSeek 지원)	Claude 계열	GPT 계열	임의	Gemini 계열	Kimi 계열
공식 런타임	✅(web + headless + 플러그인 생태계)	제품이 곧 런타임	제품이 곧 런타임	클라이언트(공식 백엔드 없음)	제품이 곧 런타임	제품이 곧 런타임
플러그인 체계	공식 레벨: 모든 것이 플러그인, 공식 패키지 60+개	설정/훅 위주	설정 위주	설정 위주	없음	없음
커스텀 UI	✅(client 반쪽 = 자유로운 UI)	❌	❌	일부(TUI 고정)	❌	❌
자동화/CI	✅ headless profile	✅	✅	✅	✅	✅
TUI	플러그인으로 가능(공식 미내장)	✅ 내장	✅ 내장	✅ 내장	✅	✅
생태계 단계	초일부터 시작 (2026-08-13)	성숙	성숙	성숙	성숙	초기
적합한 대상	깊은 커스터마이징 + 생태계 참여를 원하는 개발자	바로 사용	바로 사용	OpenCode에 익숙한 사용자	Google 생태계	Kimi 생태계

그 외 주류 Agent 훑어보기(한 줄 포지셔닝)

Agent	한 줄 포지셔닝	dsh와의 핵심 차이
Cursor	IDE 내장형 AI 코딩 어시스턴트 (Composer/Agent 모드)	IDE에 깊게 종속; dsh는 독립 런타임으로 어떤 에디터/터미널에도 배치 가능
Amp	터미널 네이티브, 에이전트 우선 AI 코딩 agent	경량 터미널 형태; dsh는 공식 백엔드 + 플러그인 생태계가 추가됨
Devin	클라우드 "AI 소프트웨어 엔지니어"(독립 작업/브라우저/워크스페이스)	클라우드에 호스팅; dsh는 로컬 실행, 데이터가 로컬을 벗어나지 않음
Windsurf	IDE 내장형 코딩 agent(Flow/Agent 모드)	Cursor와 동일, IDE에 종속
Aider	오픈소스, Git 우선의 터미널 페어 프로그래밍 agent	"코드 수정 + git"에 집중; dsh는 범용 런타임
Qwen Code	알리바바 통이(通义) coding agent CLI(DeepSeek provider 내장)	Qwen 모델 위주; dsh는 공식이 DeepSeek을 지원하며 플러그인화되어 있음
GLM CLI	지푸(智谱) coding agent CLI	GLM 모델 위주; dsh는 모델 독립적 +

		플러그인 생태계
Grok CLI	xAI 터미널 coding agent	Grok 모델 위주

결론: 주류 agent는 대체로 세 가지로 나뉩니다 — IDE 내장형(Cursor/Windsurf), 터미널 코딩 어시스턴

트(Codex/OpenCode/Aider/Qwen Code/GLM CLI/Grok CLI), 런타임/플랫폼(dsh/Devin). dsh는 현재 유일하게 "공식 오픈소스 + 모델 독립적 + 플러그인 생태계"를 갖춘 런타임형 선수입니다.

쉬운 말로: 각 회사는 무엇이고, 누구에게 적합하며, dsh와 무엇이 다른가

Claude Code(Anthropic): 현재 가장 성숙한 터미널 코딩 어시스턴트입니다. 바로 사용 가능하고, TUI 경험이 좋고, 생태계가 성숙합니다 — **당장 일을 시작하고 싶은 사람에게 적합합니다.** 단점: 클로즈드소스, Claude 모델에 종속, 커스터마이징 여지가 제한적(설정/흑만 가능). dsh와 비교하면: dsh가 바꿀 수 있는 것들(인터페이스/도구 체인/백엔드)을 Claude Code는 바꿀 수 없지만, dsh의 "바로 사용 가능함"은 아직 Claude Code에 못 미칩니다.

OpenAI Codex: OpenAI의 터미널 agent입니다. 엔지니어링 능력이 강하고 GPT 계열 모델의 지원을 받습니다. 마찬가지로 클로즈드소스에 종속됩니다. dsh와 비교하면: 능력 라인은 비슷하지만 dsh는 오픈소스로 자유롭게 수정 가능합니다.

OpenCode: 오픈소스, 터미널, 임의의 모델을 설정할 수 있음 — dsh와 가장 닮은 "이웃"입니다. 핵심 차이:

OpenCode는 공식 백엔드 런타임이 없습니다(클라이언트 + 설정에 불과), dsh는 공식 bundle + 60+개 패키지 + 플러그인 생태계를 갖추고 있어 커스터마이징 가능 범위가 더 깊습니다. **OpenCode 사용자라면 dsh로 옮겨가는 비용이 매우 낮습니다**(개념이 비슷함).

Gemini CLI: Google의 터미널 agent입니다. 긴 컨텍스트/멀티모달이 강점입니다(Gemini 모델의 장점). Google 생태계에 종속됩니다.

Kimi CLI: 문샷 AI(月之暗面)의 터미널 agent입니다. 중국어 시나리오에 강하고 Kimi 모델의 지원을 받습니다. 생태계는 초기 단계입니다.

Cursor / Windsurf: IDE 내장형 — 이들의 장점은 "에디터 안에서 바로 사용"이고, 단점은 **IDE에 갇힌다**는 것입니다. dsh는 독립 런타임으로, 어떤 환경(터미널/CI/서버/향후 TUI)에서도 동일한 agent 능력을 사용할 수 있습니다.

Devin: 클라우드 엔지니어 — 작업을 주면 클라우드에서 일을 처리합니다. 장점은 호스팅되며 브라우저가 있다는 것; 단점은 **데이터가 로컬을 벗어나고**, 비용이 높다는 것입니다. dsh는 로컬에서 실행되어 프라이버시를 제어할 수 있습니다.

Aider: 오래된 오픈소스, Git 우선, 경량입니다. "AI가 코드 수정만 도와주면 되는" 미니멀리스트에게 적합합니다. dsh는 더 무거운 런타임입니다 — Aider가 하는 일만 필요하다면 Aider로 충분하고, Agent 시스템을 만들고 싶다면 dsh가 베이스가 됩니다.

Qwen Code / GLM CLI / Grok CLI: 각 회사 모델의 터미널 agent — 모델 종속은 이들의 자연스러운 속성입니다. dsh는 모델 독립적(공식은 DeepSeek을 지원하며 OpenAI 호환 모델도 연결 가능)이라 "모델을 바꿀 수 있음"을 원하는 사람에게 더 적합합니다.

그림 한 장으로 이해하는 dsh의 차별화 포지션



- **우상단:** dsh — 커스터마이징 가능 범위가 가장 크지만, "바로 사용 가능함"은 생태계로 보완해야 함
- **좌상단:** 자체 구축 프레임워크 — 완전히 자유롭지만 전부 직접 구축해야 함
- **우하단:** 제품형 agent — 가장 쓰기 편하지만 가장 폐쇄적임

사례 비교: 같은 작업, 다른 접근 방식

작업: Agent가 저장소 안에서 "특정 함수를 호출하는 모든 곳을 찾아 파라미터 하나를 통일해서 바꾸도록" 시키기.

Agent	어떻게 하는가	경험
dsh(web)	dsh web → 지시 입력 → 모델이 Grep/Read/Edit 도구로 완료	Web UI + 우측 플러그인 사이드바(Git 패널 추가 가능)
dsh(headless)	dsh --profile headless "작"	CI에 넣을 수 있음: 0이 아닌 종료 코드

	업" → 결과 출력 후 종료	가 곧 실패
Claude Code / Codex / OpenCode / Kimi	TUI 열기 → 지시 입력 → 모델이 완료	터미널 TUI, 바로 사용 가능
Cursor / Windsurf	IDE에서 코드 선택 → 지시 입력	IDE 내장형 경험
Devin	웹에서 작업 생성 → 클라우드에서 완료	호스팅됨, 브라우저 있음, 데이터가 로컬을 벗어남

차이점은 어디에 있는가: 같은 한 마디 말이지만, **dsh는 선택 차원을 하나 더 줍니다** — 인터페이스와 도구 체인 모두 바꿀 수 있다는 것입니다. 다른 Agent의 인터페이스/도구 체인은 공식이 정한 것이고, dsh는 여러분이 직접 조립할 수 있는 것입니다.

사례 비교: 실제 워크플로우(우리의 실측)

다음은 dsh 생태계를 실제로 개발하면서 얻은 비교 관찰입니다(2026-08-13):

시나리오	dsh 실측	비고
단순 파일 생성	콜드 스타트 ~110초(첫 라운드에 컨텍스트 주입 포함) → 핫 캐시 ~1초	추론 강도가 주요 변수
50단계 도구 체인 작업	LLM 소요 10분+, 도구 호출 9분+	각 단계마다 사고 시간이 누적됨 — 속도 개선 플러그인의 가치가 여기에 있음
플러그인 개발	처음부터 실행 가능한 플러그인까지: 1일(테스트+실기 검증 포함)	확장 포인트가 명확함(agent/request waterfall)
Claude Code와 동일 작업 비교	dsh + V4-Flash 조합의 비용은 Claude 대비 약 1/10~1/30	가격 차원에서 dsh 생태계가 확연히 우위

동일 모델 × 다른 Agent의 엄격한 비교는 Benchmark 부록 참고(omp 36초 / dsh 85초 / opencode 114초).

1.5 언제 dsh를 써야 하는가(도구 선택 결정)

✅ 지금 진입을 추천:

- 모델 독립적, 인터페이스 선택 가능, 동작 변경 가능한 Agent 베이스를 만들고 싶은 경우
- dsh 생태계 초기 기여자가 되고 싶은 경우(선점 우위, 공식이 명시적으로 장려)
- 서버/CI에서 Agent를 실행해야 하는 경우(headless profile)
- 비용에 민감한 경우(DeepSeek 모델 + 오픈소스 생태계)

▮ 잠시 관망:

- "바로 사용 가능한 코딩 어시스턴트"만 필요하다면 — Claude Code 등이 더 성숙함
- rc 단계의 파괴적 변경을 받아들일 수 없다면 — 0.1.0 정식 버전을 기다리세요
- 특정 모델 고유의 능력(예: Claude artifacts)에 의존도가 높다면 — 모델 종속 시나리오

1.6 자주 묻는 질문(FAQ)

Q1: dsh는 모델인가요?

아닙니다. dsh는 런타임/프레임워크이며, 모델은 llm 플러그인을 통해 연결됩니다(공식은 DeepSeek V4 계열을 지원하며, 이론적으로 다른 OpenAI 호환 모델도 연결 가능).

Q2: dsh와 OpenCode는 무슨 관계인가요?

둘 다 오픈소스 Agent 클라이언트이지만 포지셔닝이 다릅니다: OpenCode는 "클라이언트 + 설정"이고, dsh는 "런타임 + 공식 플러그인 생태계"(공식 백엔드 bundle과 60+개 공식 패키지 보유)입니다. dsh가 더 하위 레벨이고 커스터마이징 가능 범위가 더 큼니다.

Q3: TypeScript를 써본 적이 없어도 할 수 있나요?

가능합니다. 사용(제2장)에는 프로그래밍이 필요 없습니다; 플러그인 작성(제4장)에는 기초적인 TS가 필요하지만, 튜토리얼에 완전한 코드가 제공됩니다.

Q4: dsh는 안정적인가요?

현재 rc 단계(0.1.0-rc.6)로, 반복 속도가 빠르고 파괴적 변경이 있습니다. 프로덕션 핵심 의존성은 정식 버전을 기다리는 것을 권장하며, 생태계를 탐색하기에는 지금이 딱 좋은 시기입니다.

Q5: 지금 dsh를 배우는 것이 왜 가치 있나요?

생태계 초일 + 공식이 명시적으로 커뮤니티를 장려 + 한국어(중국어) 튜토리얼 공백 — 모든 초기 생태계에는 "가장 먼저 게를 먹어본 사람"의 보너스가 있습니다, 지금이 진입할 창구입니다.

실습 문제(진짜로 이해했는지 검증)

1. **한 줄 테스트:** 기술을 모르는 사람에게 "dsh가 무엇인지" 설명해보세요("Agent/harness/플러그인" 같은 단어 사용 금지)
2. **비교 테스트:** dsh와 Claude Code의 본질적 차이 3가지를 말해보세요(기능 목록이 아니라)
3. **선택 테스트:** 아래 시나리오에 도구를 선택하고 이유를 설명하세요:
 - 시나리오 A: 바로 사용 가능한 터미널 코딩 어시스턴트를 원함
 - 시나리오 B: "모델 독립적, 인터페이스 커스텀"인 회사 내부 Agent 플랫폼을 만들고 싶음
 - 시나리오 C: CI에서 매일 자동으로 데이터 분석 작업을 실행하고 싶음
4. **아키텍처 테스트:** "profile → 플러그인 → 확장 포인트"의 관계도를 그려보세요(원문을 보지 않고)
5. **FAQ 테스트:** "dsh는 모델인가요?", "TS를 써본 적 없어도 할 수 있나요?"에 답해보세요

실습을 마쳤다면 제 2 장으로 가서 직접 설치해봅시다.

제 2 장: 5분 빠른 시작

본 장의 목표: *따라 하면서 실행해보기*. 모든 명령어마다 예상 출력과 흔한 오류 해결법을 제공합니다. 터미널을 열어놓고 함께 따라 해보시길 권장합니다.

TL;DR(본 장 핵심, 30초 버전)

1. 설치: `npx -y @deepseek-ai/dsh web` → `http://127.0.0.1:3080`
2. 자주 쓰는 두 가지 모드: `web`(대화 UI) / `headless`(`dsh --profile headless` "작업", CI 친화적)
— 공식은 총 4가지 실행 모드를 지원(2.4.1절 개요 참고)
3. 추론 강도: `off` / `low`(사고 끄기 또는 약한 사고/최고 속도, 공식 provider는 `off`, 게이트웨이는 `low` 사용) / `high`(기본값) / `max`(최고 성능) — 도구 체인 작업 시간의 90%가 사고에 쓰이므로, 강도를 낮추는 것이 가장 빠른 속도 개선책입니다
4. 모델: `deepseek-v4-flash`(기본값, 가성비) 또는 `deepseek-v4-pro`(플러그인)
5. 설정: `~/.dsh/settings.yaml`(모델 + 추론 강도)

<details><summary>본 장 목차</summary>

- 2.1 준비 작업(30초 점검)
- Node 버전 레드라인(≥ 22.19)
- 2.2 설치(세 가지 방법)
 - 설치 트러블슈팅(커뮤니티 실제 경험)
- 2.3 모드 1: Web UI(`dsh web`)
- 2.4 모드 2: Headless(일회성 작업, 스크립트/CI에 적합)
 - 2.4.1 공식 4가지 실행 모드 개요
- 2.5 첫 번째 플러그인: `web`에 Git 패널 추가하기
- 2.6 설정과 디렉터리 빠른 참조
- 2.7 명령어 빠른 참조

- 2.8 트러블슈팅 빠른 참조

2.1 준비 작업(30초 점검)

필요 사항	확인 명령어	통과 기준
Node.js ≥ 22.19	<code>node --version</code>	v22.19.0 또는 그 이상(레드라인, 아래 참고)
npm(Node에 포함)	<code>npm --version</code>	버전 번호만 있으면 OK
네트워크	npm registry 접근 가능	패키지 설치 가능
(선택) DeepSeek API Key	https://platform.deepseek.com	실제 대화용

API Key가 없어도 dsh는 시작할 수 있습니다(화면은 열림), 하지만 대화하려면 Key가 필요합니다. 본 백서의 예시는 이미 설정되어 있다고 가정합니다.

Node 버전 레드라인(≥ 22.19)

커뮤니티 실측 결과 Node < 22.19에서는 치명적인 두 가지 API 누락이 발생합니다:

누락된 API	오류 예시	관련 스레드
node:zlib에 <code>createZstdDecompress</code> 없음	<code>TypeError: zlib.createZstdDecompress is not a function</code>	#100 (https://github.com/deepseek-ai/deepseek-harness/discussions/100)
<code>AbortSignal.timeout</code> 미구현	<code>AbortSignal.timeout is not a function</code>	#311 (https://github.com/deepseek-ai/deepseek-harness/discussions/311)

해결법: nvm/volta/fnm으로 22.19.0+로 전환하세요. Node 24 초기 버전(예: v24.15)도 install -

lefthook failed를 유발할 수 있습니다 — 이 오류를 만나면 22.19.0으로 고정하는 것이 가장 안정적입니다(#748 (<https://github.com/deepseek-ai/deepseek-harness/discussions/748>)).

2.2 설치(세 가지 방법)

방법 1: 직접 실행(초보자 추천)

```
npx -y @deepseek-ai/dsh --version
```

처음 실행하면 dsh를 다운로드합니다(패키지 용량이 커서 40+개 플러그인 모듈 포함, 약 1-3분 소요). 버전 번호가 보이면 성공입니다:

```
0.1.0-rc.6
```

방법 2: 전역 설치(자주 사용할 경우 추천)

```
npm install -g @deepseek-ai/dsh
dsh --version
```

방법 3: Node 설치가 필요 없는 설치 패키지(초보자 선택 가능)

Node를 설치하고 싶지 않으신가요? 커뮤니티 개발자 codeAnqiang-ma(#380)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/380>) 플러그인 트러블슈팅 스레드 작성자, 수록 허가 받음)

가 **Node 설치가 필요 없는 설치 패키지**를 제공합니다 — mac DMG / Windows exe, 하부는 공식 dsh를 그대로 패키징하고 Node 런타임을 내장해 사전 의존성이 전혀 없습니다:

dsh-installers (<https://github.com/codeAnqiang-ma/dsh-installers>)(비공식, 공식 rc 버전에 맞춰 해당 설치 패키지 배포, SHA256 검증 포함)

"Node 환경을 건드리고 싶지 않고 더블클릭으로 바로 쓰고 싶은" 체험 사용자에게 적합합니다; 플러그인 개발/찾은 업그레이드가 필요하다면 방법 1이나 2를 권장합니다.

설치 트러블슈팅(커뮤니티 실제 경험)

문제	증상	해결	스레드
최초 npx 극도로 느림 (Windows)	npx dsh web이 Windows에서 처음 실행될 때 8분+ 동안 아무 반응 없음, npm이 500+ 패키지를 다운로드해야 함	인내심을 갖고 기다리거나, npm i -g @deepseek-ai/dsh로 전환하면 이후 즉시 시작됨	#176 (https://github.com/deepseek-ai/deepseek-harness/discussions/176)
pnpm dlx 404	pnpm dlx @deepseek-ai/dsh web이 @deepseek-ai/dsh-pty@0.0.1-rc.2가 게시되지 않았다는 오류	pnpm dlx 대신 npx나 npm i -g 사용	#369 (https://github.com/deepseek-ai/deepseek-harness/discussions/369)
pnpm 전역 설치 후 플러그인을 찾지 못함	pnpm add -g @deepseek-ai/dsh 후 시작 시 Cannot find package 'cordis-plugin-timer' 오류	pnpm 전역 설치의 의존성 해석 전략이 npm과 다름; npm i -g나 npx 권장	#55 (https://github.com/deepseek-ai/deepseek-harness/discussions/55)
WSL2에서 npx 설치 실패	WSL2(Ubuntu 등 배포판)에서 npx -y @deepseek-ai/dsh 설치 실패	먼저 npm 프록시 설정 확인; 그래도 안 되면 소스 설치로 전환 (커뮤니티 실측 성공)	#118 (https://github.com/deepseek-ai/deepseek-harness/discussions/118)

WSL2 설치 참고사항: 커뮤니티는 #118 (<https://github.com/deepseek-ai/deepseek-harness/discussions/118>) 에서 WSL2에서 npx 설치가 실패할 수 있다고 보고했습니다(Windows 네이티브 쪽은 정상, 해당 스레드의 "윈도우에서는 이미 다 설치하고 놓고 있음" 대조 참고). 점검 순서: ① npm 프록시/registry 설정 확인; ② npx가 여전히 안 되면 먼저 소스 설치를 시도(공식 저장소를 clone한 후 `pnpm install`, 커뮤니티 실측 가능). 자세한 내용은 해당 스레드 댓글 참고.

전역 설치 방식 비교: `npm i -g`가 가장 안정적(커뮤니티 검증이 가장 많음); `npx`는 체험용으로 적합; `pnpm dlx`는 아직 권장하지 않음(rc 단계에서 `pty` 패키지가 `pnpm`에서 보이는 registry에 게시되지 않음).

2.3 모드 1: Web UI(dsh web)

시작

```
dsh web
```

예상 출력:

```
dsh web: http://127.0.0.1:3080
```

브라우저에서 <http://127.0.0.1:3080> 을 엽니다.

화면 구성 이해하기(스크린샷 대조)



영역	내용
왼쪽 열	세션 목록 / 워크스페이스 전환 / 새 세션
가운데 열	대화 영역: 입력창, 모델 선택(DeepSeek V4 Flash), 추

	론 등급(High)
오른쪽/하단	플러그인 사이드바(기본적으로 비어 있음; 커뮤니티 플러그인 설치 후 표시됨)
우측 상단	Session log(세션 로그) / 트레이스(도구 호출 트레이스)

첫 번째 대화

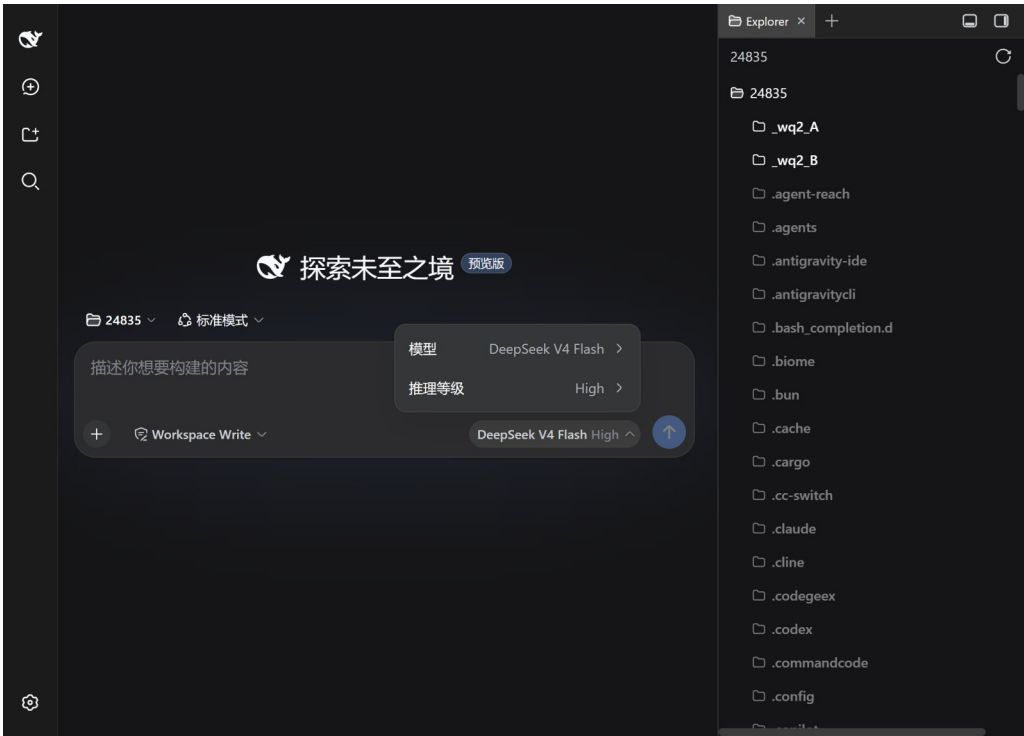
1. "새 세션" 클릭
2. 입력창에 입력: 안녕, 한 문장으로 자기소개 해줘
3. Enter로 전송

예상 답변은 다음과 비슷합니다:

안녕하세요! 저는 DeepSeek 기반 AI 코딩 어시스턴트입니다. 코드 작성, 디버깅, 파일 처리, 자료 검색은 물론 다양한 개발 및 업무 작업을 도와드릴 수 있습니다.

모델과 추론 강도

입력창 옆의 "모델 선택"을 클릭하면 모델과 추론 강도 선택기가 열립니다:



모델	포지셔닝
deepseek-v4-flash(기본값)	가성비: 빠르고 저렴, 일상 사용에 충분
deepseek-v4-pro	플래그십: 더 강력하지만 더 비싸고 느림

추론 등급(사고 모드 3단계, 2026-08-13부터 지원):

강도	속도	품질	권장 시나리오
low	가장 빠름	충분함	단순/결정론적 작업, 배치, 도구 체인의 저렴한 라운드
high(기본값)	중간	좋음	일상적인 Agent 작업
max	가장 느림	최고	복잡한 추론, 긴 체인 계획

위 표의 강도 등급은 본 백서가 실측한 환경(pi-ai / opencode-go 게이트웨이)이 지원하는 것입니다. DeepSeek 공식 어댑터(기본 provider=deepseek-official, llm-deepseek 플러그인)는 off(사고 끄기, 최고 속도) / high / max 세 등급만 받으며, low를 넣으면 UNSUPPORTED_REASONING Effort를 던집니다. settings.yaml에서 기본 provider를 쓸 때는 off / high / max를 사용하세요.

💡 **성능의 핵심 인자:** 모델은 매 도구 호출 전마다 다시 사고합니다. 실측 결과 "파일 생성" 작업 하나에서 사고가 벽시계 시간의 ~90%를 차지했고, 50단계 도구 체인 작업에서는 사고 누적 시간이 수 분에서 십수 분에 이를 수 있습니다. 추론 강도를 낮추는 것이 가성비가 가장 높은 속도 개선 수단입니다(제6장 + 속도 개선 플러그인 예제 참고).

2.4 모드 2: Headless(일회성 작업, 스크립트/CI에 적합)

```
dsh --profile headless "안녕, 한 문장으로 자기소개 해줘"
```

예상 출력(결과를 출력한 후 프로세스 종료):

```
안녕하세요! 저는 DeepSeek 기반 AI 코딩 어시스턴트입니다. 코드 작성, 디버깅, 파일 처리, 자료 검색은 물론 다양한 개발 및 업무 작업을 도와드릴 수 있습니다.
```

Headless의 핵심 가치:

- **자동화:** CI, 서버, cron에 넣을 수 있음
- **스크립트 친화적:** 0이 아닌 종료 코드 = 실패; 출력을 파이프로 처리 가능
- **세션 격리:** 매 호출마다 새로운 세션(--resume으로 복원 가능, dsh --profile headless --help 참고)

실전: 매일 한 번 "일일 리포트 생성"을 실행하는 스크립트 작성:

```
dsh --profile headless "위크스페이스의 오늘 git log를 읽고 한국어 일일 리포트 요약을 생성해줘" > daily-report.md
echo "exit=$?"
```

2.4.1 공식 4가지 실행 모드 개요

출처: 공식 사이트 <https://deepseek.com/harness/> 및 공식 저장소 docs/ (2026-08 스냅샷). 앞의 두 절(web / headless)은 본 백서가 직접 실측하며 상세히 다룬 내용입니다, 나머지 두 모드는 공식이 발표했지만 본 백서에서는 아직 항목별로 실측하지 않았습니다.

모드	한 줄 설명	용도	백서 커버리지
Standard mode (표준)	완전한 도구 세트 + 브라우저 인터페이스	일상 대화/개발(dsh web)	✅ 2.3절 실측
Minimal mode (미니멀)	bash와 str_replace_editor 두 도구만 사용	벤치마크 테스트, 최소 공격 표면	⚠️ 미실측, 공식 docs 참고
Code mode (코드)	모델이 TypeScript를 작성해, 단일 프로그램 안에서 여러 라운드의 도구 호출을 오케스트레이션	긴 체인 자동화, 결정론적 실행	⚠️ 미실측, 공식 docs 참고
Creator mode (생성)	런타임 자체 점검 + 메모리 내 Cordis 플러그인 테스트 + 새 모드 조합	플러그인 개발, 빠른 프로토타이핑	⚠️ 미실측, 공식 docs 참고

초보자에게: 일상에는 **Standard(web)**, 스크립트에는 **Headless**면 충분합니다, Code/Creator 모드는 고급 기능이니 공식 문서가 안정화되면 더 깊이 파보세요. 공식 문서 사이트(VitePress): <https://deepseek-harness.github.io/deepseek-harness/>

2.5 첫 번째 플러그인: web에 Git 패널 추가하기

dsh의 사이드바는 기본적으로 비어 있습니다 — 커뮤니티 플러그인 dsh-better-sidebar를 설치해 "모든 것이 플러그인"을 체험해봅시다(자세한 원리는 제3장 참고, 여기서는 먼저 실행부터 해봅시다):

```
# 1. 당신의 web profile을 찾으세요
# Windows: %USERPROFILE%\dsh\profiles\web
# macOS/Linux: ~/.dsh/profiles/web

# 2. package.json의 dependencies에 한 줄 추가(link:가 플러그인 소스 코드를 가리킴)
# "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"

# 3. cordis.patch.yml에 장착 줄 추가
# - insert:
#   - id: better-sidebar
#     name: dsh-better-sidebar
```

```
# 4. 설치 후 재시작
cd ~/.dsh/profiles/web && pnpm install
# (Windows cmd에서는 `~`가 전개되지 않으니 다음을 사용하세요: cd %USERPROFILE%
\dsh\profiles\web)
dsh web
```

재시작 후 오른쪽에 파일 관리 / 터미널 / **Git 패널** / 브라우저 등의 탭이 나타납니다:

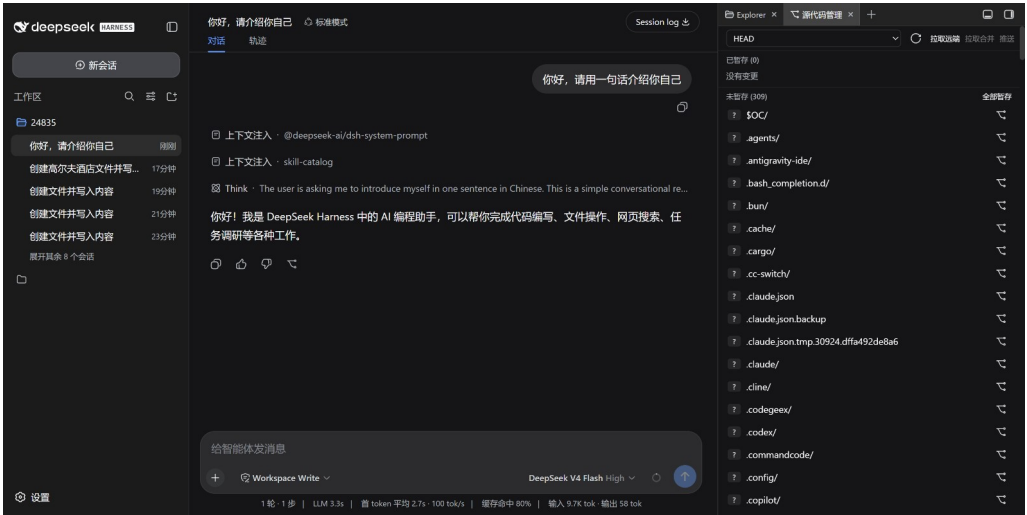


그림 속 "원격 가져오기 / 가져와서 병합 / 푸시" 버튼은 커뮤니티 PR로 구현된 것입니다(제5장 사례 참고) — 이것이 바로 "플러그인 생태계"가 작동하는 방식입니다.

2.6 설정과 디렉터리 빠른 참조

첫 실행 후 생성되는 디렉터리:

```
~/ .dsh/
├─ settings.yaml           # 전역 설정 (모델, 추론 강도)
├─ profiles/              # profile 디렉터리
│   └─ web/
│       ├── package.json   # 플러그인 의존성 + 매니페스트
│       └─ cordis.patch.yml # 패치 레이어 (플러그인 장착)
├─ sessions/              # 세션 데이터
└─ storages/               # 영속 저장소
```

settings.yaml 예시:

```
agent-default-model:
  model: deepseek-v4-flash
  reasoningEffort: high
```

2.7 명령어 빠른 참조

명령어	용도
dsh web	Web UI 시작(=dsh --profile web)
dsh --profile headless "작업"	일회성 작업, 결과 출력 후 종료
dsh plugin --profile <name> add <pkg>	profile에 플러그인 설치
dsh --dump-config	합성된 설정 트리 출력
dsh --profile tui	TUI 모드(tui 플러그인을 먼저 설치해야 함, 공식 미내장)
dsh --version	버전

2.8 트러블슈팅 빠른 참조

증상	원인과 해법
dsh: profile "tui" does not exist	tui profile은 플러그인으로 만들어야 함(dsh plugin --profile tui add <pkg>)
npm가 극도로 느림	첫 다운로드 용량이 큼; npm i -g 사용 시 더 빠름
브라우저에서 3080 포트가 안 열림	포트 점유 상태: netstat -ano findstr 3080 → PID kill
모델이 응답하지 않음	~/ .dsh/settings.yaml의 모델 설정 + API Key 확인
플러그인 설치 실패(404)	rc.1 의존성 단절 : 의존성이 ^0.1.0-rc.6 라인을 사용하는지 확인(제3장 흔한 트러블슈팅 #1)
업그레이드 후 동작이 바뀜	rc 단계의 파괴적 변경은 정상이니 공식 changelog 확인

소스 코드 방식으로 시작이 느린 근본 원인(#1424 커뮤니티 실측 확인): pnpm dsh web은 매번 시작할 때 tsx/esbuild로 전체 TS 소스 그래프를 그 자리에서 트랜스파일합니다(전체 빌드가 아님), 여기에 기계식 디스크나 파일 수가 많으면 콜드 스타트가 수 분에 이를 수 있습니다. **실측 시간**: tsx 핫 캐시~40초 / 콜드 캐시~5분 / 컴파일 산출물 lib/bin.js 버전~12초(페이지 응답 5ms). **해결**: ① 시작 명령을 node apps\cli\lib\bin.js web로 바꿔 컴파일 산출물을 사용 ② 또는 먼저 pnpm build로 전체 빌드를 한 번 실행 ③ 또는 그냥 npx @deepseek-ai/dsh web(배포 버전) 사용.

Windows 공통 트러블슈팅 1 — 미니멀 모드가 실행되지 않음(#1889 실측): terminal inspection is unsupported on platform win32 오류가 나면 **node-pty와는 무관합니다** — 호출 순서상 플랫폼 inspector를 먼저 파싱한 후 spawn하는데, inspector가 앞에 있기 때문입니다. node-pty 재설치/Node 교체/VS Build Tools 설치 모두 소용없습니다; dsh-win32 계열 패치가 필요합니다.

Windows 공통 트러블슈팅 2 — 당일 배포된 플러그인 버전이 설치되지 않음: profile의 pnpm-workspace.yaml에서 minimumReleaseAgeExclude는 연결 시점의 버전만 기록하므로, 플러그인이 새 버전을 배포한 후 약 하루 동안은

`dsh plugin add <pkg>@latest`가 Already up to date를 반환합니다. 즉시 설치하려면 `--config.minimumReleaseAge=0`.

실습 문제(10분 이내 완료)

1. **설치:** `npx -y @deepseek-ai/dsh --version`으로 버전 확인
2. **Web 대화:** `dsh web` 시작, 새 세션에서 "안녕" 전송, 답변과 화면 레이아웃 관찰
3. **Headless:** `dsh --profile headless "1+1은 몇이야"`, 결과 출력 후 종료되는지 확인
4. **추론 강도 실험:** `settings.yaml`의 `reasoningEffort`를 `off`(공식 provider)나 `low`(서드파티 게이트웨이)로 바꾸고 단순 작업을 다시 실행해 속도 차이를 체감
5. **트러블슈팅 연습:** "포트 점유" 상황을 재현하고(먼저 3080을 점유하는 서비스를 하나 실행), `netstat`으로 원인을 찾아보기

전부 통과했다면 제 3 장으로 가서 "왜 이렇게 바꿀 수 있는지" 이해해봅시다.

제 3 장: profile과 플러그인 시스템

본 장의 목표: dsh의 커스터마이징 가능한 골격을 이해합니다 — profile은 어떻게 구성되는지, 플러그인은 어떻게 장착하는지, host/client 두 반쪽이란 무엇인지. 이것이 "사용자"에서 "개발자"로 넘어가는 분기점입니다.

TL;DR(본 장 핵심, 30초 버전)

1. **profile** = 하나의 실행 가능한 형태: `~/.dsh/profiles/<name>/` 디렉터리이며, `package.json`(플러그인 매니페스트) + `cordis.patch.yml`(패치 레이어)로 구성됩니다
2. 플러그인 장착은 두 단계뿐: `package.json`에 의존성 추가 + `cordis.patch.yml`에 장착 줄 추가, 그 다음 `pnpm install` 후 재시작
3. **host** 반쪽은 Node에서, **client** 반쪽은 브라우저에서 실행: npm 패키지 하나가 `exports["."]`와 `exports["./client"]`로 두 얼굴을 동시에 가질 수 있습니다
4. 동작을 바꾸려면 확장 포인트를 찾고, 코어를 fork하지 마세요: `agent/request waterfall`, `conversationEvents`, `ctx.slots.inject`, `settings` 서비스가 4대 자주 쓰는 흑입니다
5. **rc 단계의 3대 트러블슈팅**: 의존성 버전은 `^0.1.0-rc.6` 라인 사용, `next()`는 반드시 `await`, client 테스트에는 dsh 런타임이 필요

<details><summary>본 장 목차</summary>

- 3.1 profile: 실행 가능한 설정 스택
 - 내장 Agent 프리셋: 표준 / PTC / 미니멀 / 생성
- 3.2 플러그인 하나 장착하기: 두 곳만 수정
- 3.3 host 반쪽과 client 반쪽: 하나의 패키지, 두 얼굴
- 3.4 확장 포인트: 동작을 바꾸려면 흑을 먼저 찾고, 코어를 fork하지 마세요
- 3.5 흔한 트러블슈팅(실제 경험)
 - 의존성 해석 트러블슈팅
 - 플러그인 설치 형식 트러블슈팅
 - 그 외 흔한 트러블슈팅

3.1 profile: 실행 가능한 설정 스택

dsh는 **profile**로 "실행 가능한 형태 하나"를 표현합니다. 공식이 두 개를 내장하고 있고, 나머지는 플러그인으로 만듭니다:

profile	용도	명령어
web	Web UI(대화 + 사이드바 + 도구)	dsh web
headless	일회성 CLI 작업	dsh --profile headless "작업"
tui(플러그인 필요)	터미널 UI	dsh --profile tui(미내장, 플러그인 설치 필요)

profile 디렉터리는 이렇게 생겼습니다(~/.dsh/profiles/<name>/):

```
profiles/web/
├─ package.json      # 플러그인 의존성 + dsh.profile 매니페스트(bundles 순서)
├─ cordis.patch.yml  # 당신의 패치 레이어: 플러그인 장착/오버라이드 선언
├─ cordis.yml        # (생성된) 합성 설정
├─ pnpm-workspace.yaml
└─ node_modules/
```

로딩 순서(공식 문서 원문): 내장 bundle(dsh-base → dsh-web-app) → profile의 cordis.patch.yml → 사용자 레벨 ~/.dsh/cordis.patch.yml → --patch 오버라이드 레이어.

내장 Agent 프리셋: 표준 / PTC / 미니멀 / 생성

profile은 "어떤 형태로 시작할 것인가"를 해결하고, Agent 프리셋은 "어떤 방식으로 Agent를 실행할 것인가"를 해결합니다. dsh는 네 가지 Agent 프리셋을 내장하고 있으며(소스는 공식 저장소 apps/cli/config/agent-presets/의 agent.cordis.yml), 각 프리셋마다 기본으로 로드하는 플러그인 조합이 다릅니다:

프리셋	포지셔닝	로드되는 내용	전형적인 시나리오
표준	기본 프리셋	완전한 도구 조합	일상적인 Agent 작업
PTC(Code mode)	코드 주도 도구 체인	프로그래매틱 도구 호출 — 모델이 여러 라운드의 도구 호출을 조합하는 코드 조각을 생성	복잡한 다단계 도구 워크플로우
미니멀	벤치마크 테스트	bash + 파일 편집 도구 하나만	최소 환경에서의 모델 벤치마크 테스트
생성	플러그인 개발용	현재 런타임 검사, 메모리 내	새로운 플러그인 조합의 구

		에서 Cordis 플러그인 실험, 새 프리셋 조합	측과 테스트
--	--	--------------------------------	--------

PTC 명칭 설명(#1052 (<https://github.com/deepseek-ai/deepseek-harness/discussions/1052>) 댓글의 커뮤니티 상세 설명):

PTC = **Programmatic Tool Calling**(**프로그래매틱 도구 호출**). 공식 중국어 사이트는 "PTC 모드"라는 마케팅 이름만 사용했고(공식 사이트 원문: "PTC 모드는 모델이 생성한 코드 조각을 통해 여러 라운드의 도구 호출을 조합합니다"), 공식 영문 페이지에서는 이에 대응하는 이름이 **Code mode**입니다; "Programmatic Tool Calling"이라는 완전한 풀네임은 커뮤니티의 상세 설명(cnblogs 가이드 (<https://www.cnblogs.com/sing1ee/p/22455466>))에서 나온 것이며 소스 코드 구현과 완전히 일치합니다. 메커니즘과 이점은 제8장 8.3.1절 참고; 4가지 실행 모드 전체 개요는 제2장 2.4.1절 참고.

3.2 플러그인 하나 장착하기: 두 곳만 수정

커뮤니티 플러그인 dsh-better-sidebar를 장착하는 것을 예로 들어봅니다(실제 조작):

① **package.json**에 의존성 추가(link:는 로컬 소스 코드를 가리키거나, npm 패키지 이름을 사용):

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② **cordis.patch.yml**에 장착 줄 추가:

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 설치하고 재시작:

```
cd ~/.dsh/profiles/web && pnpm install
dsh web # 재시작 후 적용됨
```

💡 플러그인은 기본적으로 **세션별로 격리됩니다**(better-sidebar의 레이아웃/탭은 세션 단위로 영속화됩니다) —
장착은 profile 레벨이지만, 상태는 세션 레벨입니다.

3.3 host 반쪽과 client 반쪽: 하나의 패키지, 두 얼굴

"반쪽"(half) = 플러그인 구현의 절반. 플러그인 하나는 두 부분으로 나뉘어 서로 다른 환경에서 실행될 수 있습니다: **host 반쪽**은 Node 프로세스에서 실행되고(파일 시스템/명령/서비스 권한 보유), **client 반쪽**은 브라우저에서 실행됩니다(web UI, 인터페이스와 상호작용만 가능). 둘은 cordis 서비스로 연결됩니다 (ctx.provide/ctx.get).

플러그인은 **host 반쪽만 있을 수도 있습니다** — UI가 필요 없는 플러그인(도구류, 서비스류)은 host 반쪽만 있으면 됩니다: dsh.client를 선언하지 않고 exports["./client"]를 작성하지 않으면 됩니다(제4장의 템플릿이 바로 순수 host 플러그인입니다). 반대로 client 반쪽만 있을 수도 있지만(3.5절 Q4 참고), client 반쪽은 파일 시스템에 직접 접근할 수 없어서 host 반쪽의 연결이 필요합니다.

플러그인은 두 실행 반쪽을 동시에 가질 수 있습니다:

반쪽	실행 위치	역할	예시
host 반쪽	Node 프로세스	도구, 서비스, 이벤트, 파일 시스템, 프로세스	apply(ctx)로 도구/서비스 등록
client 반쪽	브라우저(web profile)	UI, 상호작용, DOM	package.json의 dsh.client 선언 + src/client/

package.json에서 client 반쪽을 선언하는 방법:

```
{
  "dsh": {
    "client": {
      "platform": "web",
      "inject": [],          // 주입이 필요한 client 서비스(보통 비어 있음; 공식 예시도 [])
      "immediately": true   // web 시작과 동시에 즉시 로드할지 여부
    },
    "exports": {
      ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
      "./client": { "types": "./lib/types/client/index.d.ts", "default":
        "./lib/client.js" }
    }
  }
}
```

- host 반쪽: exports["."] → apply(ctx)(cordis 플러그인 본체)
- client 반쪽: exports["./client"] → 브라우저 쪽의 apply(ctx)

- inject: 필요한 서비스 선언(cordis 의존성 주입)

3.4 확장 포인트: 동작을 바꾸려면 혹은 먼저 찾고, 코어를 fork하지 마세요

공식 원칙(CONTRIBUTING/AGENTS.md에 명시): **"Plugins, not loop changes: new behavior goes on**

documented extension points". 초보자가 가장 흔히 저지르는 실수는 코어 loop를 수정하는 것입니다 — 올바른 방법은 확장 포인트를 사용하는 것입니다.

확인된 자주 쓰는 확장 포인트(이후 장에서 하나씩 실습):

확장 포인트	위치	용도
agent/request waterfall	agent-loop	매 모델 요청 전 설정 변경(provider/model/reasoningEffort/tools) — 속도 개선 플러그인 예제가 이것을 사용
agent/request-error	agent-loop	요청 실패 시 개입(공식 compaction 플러그인이 컨텍스트 오버플로우 복구에 이것을 사용)
conversationEvents.register	client runtime	대화 이벤트 구독/주입(tool/call, turn/start 등)
ctx.slots.inject	client ui-slots	인터페이스 슬롯에 UI 주입(예: turnTail에 산출물 파일 행 표시)
settings 서비스	dsh-settings	사용자가 설정 가능한 네임스페이스 등록(설정 페이지 자동 렌더링)
ctx.provide / ctx.get	cordis	플러그인 간 서비스 제공

사용법(agent/request를 예로) — 플러그인의 apply(ctx)에서 등록:

```
import type {} from '@deepseek-ai/dsh-agent' // 타입 확장

export function apply(ctx: any) {
  ctx.on('agent/request', async (request, next) => {
    // 매 모델 요청 전에 트리거됨: provider / model / reasoningEffort / tools를 바꿀 수 있음
    request.reasoningEffort = request.tools?.length > 3 ? 'low' :
    request.reasoningEffort
    // 반드시 await next()를 하고 그 결과를 return해야 함(Promise이므로, 수정된 요청을 다음 단계로 넘겨줌)
    return await next(request)
  })
}
```

핵심: `agent/request`는 **waterfall**(체인)입니다 — 모든 플러그인이 `await next()`를 하고 결과를 `return`해야 합니다; `await`를 빠뜨리거나 `return`을 하지 않으면 다운스트림이 당신의 수정 사항을 받지 못합니다(3.5절 흔한 트러블슈팅 참고).

3.5 흔한 트러블슈팅(실제 경험)

의존성 해석 트러블슈팅

트러블	증상	해결	스레드
rc.1 의존성 단절	<code>pnpm install</code> 이 <code>@deepseek-ai/dsh-type-meta@0.0.1-rc.1</code> 404 오류	공식 rc.1 시절 여러 패키지가 게시된 적이 없음; <code>^0.1.0-rc.6</code> 라인으로 업그레이드	—
전역 <code>core.hooksPath</code> 충돌	전역으로 <code>core.hooksPath</code> 를 설정한 경우(예: Codex/Claude Code) → <code>pnpm install</code> 시 <code>lefthook postinstall</code> 실패	전역 <code>hooksPath</code> 를 임시로 해제: <code>git config --global --unset core.hooksPath</code> , 설치 후 복원	#139 (https://github.com/deepseek-ai/deepseek-harness/discussions/139)
macOS 전역 설치가 플러그인을 해석하지 못함	<code>pnpm add -g @deepseek-ai/dsh</code> 후 시작 시 약 88개 플러그인 패키지에 <code>ERR_MODULE_NOT_FOUND</code> 오류	<code>pnpm</code> 전역 설치의 모듈 해석 전략이 <code>npm</code> 과 다름; macOS에서는 <code>npm i -g</code> 나 <code>npx</code> 권장	#204 (https://github.com/deepseek-ai/deepseek-harness/discussions/204)
koffi 사전 컴파일 손상 (Windows)	koffi 3.1.3/3.1.4의 win32-x64 사전 컴파일 바이너리가 손상됨 → 디렉터리 선택기 크래시/서비스가 조용히 멈춤	koffi@3.1.2로 고정: <code>pnpm add koffi@3.1.2</code> 또는 <code>package.json</code> 의 <code>resolution</code> 수정	#293 (https://github.com/deepseek-ai/deepseek-harness/discussions/293)
koffi 네이티브 크래시 (Windows)	선택기 worker 크래시 (<code>0xC0000005</code> 액세스 위반) 또는 시작 후 바로 멈춤	위와 동일하게 3.1.2로 고정; 그래도 크래시하면 STA 스레드 <code>CoUninitialize</code> 세그폴트 인지 확인(커뮤니티 패치 존재)	#197 (https://github.com/deepseek-ai/deepseek-harness/discussions/197)

플러그인 설치 형식 트러블슈팅

형식	작성법	주의사항	스레드	
npm 패키지명	<code>dsh plugin add dsh-better-sidebar</code>	npm에 이미 게시되어 있어야 함; rc 단계에서는 많은 커뮤니티 플러그인이 아직 게시되지 않음	—	
GitHub 저장소	<code>dsh plugin add</code>	의존성만 추가할 뿐,	#656	

	github:작성자/저장소	cordis.patch.yml에 자동으로 append되지 않음 , 장착 줄을 수동으로 추가해야 함	(https://github.com/deepseek-ai/deepseek-harness/discussions/656)	
로컬 경로	link:/path/to/plugin(macOS/Linux) link:C:\path\to\plugin(Windows)	개발 디버깅용; 경로 슬래시는 플랫폼마다 다름	—	
공백 포함 경로 (Windows)	dsh plugin <...>에 공백 포함 경로 전달	Windows에서 공백 포함 경로가 내부 전달 시 끊어짐: apps/cli/src/plugin.ts의 runPlugin이 spawnSync("pnpm", args, { shell: true })를 사용 — Node가 공백으로 인자를 이어붙이고 이스케이프하지 않음 (Node 22+부터 DEP0190 경고), 호출자의 따옴표가 전달 레이어까지 도달하지 못함	수정 방향: shell: false(인자 배열을 직접 전달)이나 전달 전 직접 이스케이프	#1420 (https://github.com/deepseek-ai/deepseek-harness/discussions/1420)

커뮤니티 트러블슈팅 요약: github: 형식으로 플러그인을 설치한 후에는 cordis.patch.yml에 - insert: 장착 줄을 수동으로 추가하는 것을 잊지 마세요. 그렇지 않으면 플러그인 의존성은 설치되었지만 dsh가 로드하지 않습니다.

그 외 흔한 트러블슈팅

트러블	증상	해결
플러그인에 main이 없음	dsh: No "exports" main defined	host 플러그인의 package.json은 . 엔트리를 노출해야 함("main": "src/index.ts"는 tsx로 직접 로드 가능)
이벤트 핸들러가 await next()를 빠뜨림	요청에서 provider/model이 유실되어 오류 발생	agent/request의 next()는 Promise를 반환하므로, 반드시 await 후 spread해야 함
'agent/request' is not assignable to keyof Events 타입 오류	npm 타입이 공식 타입 확장을 re-export하지 않음	느슨한 시그니처 사용(ctx.on as unknown as ...)으로 경계에서 변환
client 패키지 산출물이	jsdom으로 client 테스트를 직접 실행	컴포넌트 레벨 테스트에는 dsh web 런

window.__ModuleLoader__에 의존	불가	타임이 필요(공식 ci에서 실행)
-----------------------------	----	--------------------

프리셋 장착 싱글턴 충돌(#1415/#1827 확인): Cordis toolset을 가진 프리셋을 장착할 때마다, tool-cordis는 프로세스 싱글턴인 Host inspect 레지스트리에 4개의 provider ID(Service/Event/Builtin/Tool)를 중복 등록합니다 — 두 번째 장착 시 바로 already registered를 던지며, 세션 복원/여러 프리셋을 겹칠 때 이 문제가 발생합니다. 점검: 동일 프로세스 안에서 tool-cordis가 단 한 번만 장착되는지 확인하세요.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 profiles/web/ 디렉터리 아래의 파일 구조를 그리고, 각 파일의 역할을 말해보세요

정답 확인: 본 장 3.1절 "profile 디렉터리는 이렇게 생겼습니다" 참고

2. **이해 문제:** "host 반쪽"과 "client 반쪽"이 각각 어디서 실행되고 무엇을 담당하는지 설명하세요. 플러그인이 host 반쪽만 가질 수 있나요?

정답 확인: 본 장 3.3절 표 참고

3. **실습 문제:** 여러분의 ~/.dsh/profiles/web/package.json을 열어 dsh.profile.bundles 필드를 찾고, bundle의 로딩 순서를 말해보세요

정답 확인: 본 장 3.1절 "로딩 순서" 단락 참고

4. **실습 문제:** dsh-my-widget이라는 이름의 플러그인을 장착한다고 가정하고, package.json과 cordis.patch.yml에 각각 무엇을 추가해야 하는지 작성해보세요

정답 확인: 본 장 3.2절의 두 곳 수정 예시 참고

5. **실습 문제:** cordis.patch.yml에 잘못된 장착 줄을 하나 추가하고(예: 플러그인 이름 오타), dsh web을 재시작해 오류 메시지를 관찰하고 기록하세요

정답 확인: 본 장 3.5절 "흔한 트러블슈팅" 표와 비교

6. **사고 문제:** 공식 원칙은 "Plugins, not loop changes"라고 말합니다. dsh의 "모델 응답 후 자동 요약" 동작을 바꾸고 싶다면 어떤 확장 포인트를 써야 할까요? 왜 agent-loop 소스 코드를 직접 수정하면 안 될까요?

정답 확인: 본 장 3.4절 확장 포인트 표 + "동작을 바꾸려면 혹은 먼저 찾을 것" 원칙 참고

자주 묻는 질문 FAQ

Q1: profile과 bundle은 무엇이 다른가요?

profile은 "실행 가능한 형태 하나"(예: web, headless)이고, bundle은 "미리 설정된 플러그인 집합 하나"입니다. profile은 `dsh.profile.bundles`로 어떤 bundle을 로드할지 지정한 다음, 자신의 patch를 덧붙입니다. 간단히 말하면: bundle은 블록 키트이고, profile은 조립된 완성품입니다.

Q2: cordis.patch.yml의 - insert:는 무슨 의미인가요? 다른 작업도 있나요?

insert는 "플러그인 체인에 새 플러그인을 삽입한다"는 뜻입니다. patch 파일은 cordis의 설정 병합 메커니즘을 기반으로 하며, insert(삽입), override(기존 플러그인 설정 덮어쓰기) 등의 작업을 지원합니다. 초보자는 insert만 익혀도 대부분의 플러그인을 장착할 수 있습니다.

Q3: 플러그인 상태는 profile 단위로 격리되나요, 세션 단위로 격리되나요?

장착은 profile 레벨입니다(한 번 설치하면 해당 profile의 모든 세션에서 사용 가능). 하지만 상태는 세션 레벨입니다. 예를 들어 better-sidebar의 탭 레이아웃은 세션별로 영속화되어, 세션끼리 서로 간섭하지 않습니다.

Q4: client 반쪽만 있는(순수 UI) 플러그인을 작성하고 싶은데, 가능한가요?

가능합니다. `package.json`에 `dsh.client + exports["./client"]`를 선언하고, `exports["."]`는 작성하지 않으면 됩니다. 단, client 반쪽은 파일 시스템에 직접 접근하거나 명령을 실행할 수 없으므로, host 반쪽의 서비스(`ctx.provide/ctx.get`)를 통해 연결해야 합니다.

Q5: agent/request waterfall과 conversationEvents.register는 무엇이 다른가요? 어떤 것을 써야 하나요?

`agent/request`는 "모델 요청 설정을 바꾸는" 훅입니다(provider/model/tools/reasoningEffort), 매 요청 전에 트리거되어 새 설정을 반환합니다. `conversationEvents.register`는 "대화 이벤트를 구독/주입하는" 훅입니다(tool/call, turn/start 등), 이벤트 스트림을 감시하거나 주입하는 데 사용합니다. 요청 파라미터를 바꾸고 싶다면 전자를, 대화 흐름을 감시하고 싶다면 후자를 사용하세요.

Q6: 왜 제 플러그인이 설치됐는데 반응이 없나요? 어떻게 디버깅하나요?

세 단계로 점검하세요: ① `pnpm install`에 오류가 없고 `node_modules`에 여러분의 플러그인이 있는지 확인; ② `cordis.patch.yml`의 `id/name` 철자가 정확한지 확인; ③ `dsh web`을 재시작한 후 프로세스 로그에 플러그인 로딩 정보가 있는지 확인. `host` 반쪽에 `console.log`가 있다면 로그에서 출력을 볼 수 있어야 합니다.

제 4 장: 플러그인 개발 실전

본 장의 목표: 처음부터 실제로 사용 가능한 host 플러그인을 작성합니다 — agent/request 확장 포인트를 통해 추론 강도를 자동으로 조절합니다. 이것은 속도 개선 플러그인의 완전한 분석이며, 모든 코드는 실행 가능하고 테스트 가능합니다.

TL;DR(본 장 핵심, 30초 버전)

1. **핵심 문제:** dsh는 매 도구 호출 전마다 모델이 다시 사고하며, 50단계 작업에서는 90%+ 시간이 사고에 쓰입니다 — 강도를 낮추는 것이 가장 빠른 속도 개선책입니다
2. **3층 검증:** 순수 함수 단위 테스트(결정이 올바른가) → waterfall 계약 테스트(연결이 올바른가) → 실기 검증(실제 루프에서 작동하는가)
3. **agent/request waterfall:** 매 모델 요청 전에 트리거되고, 리스너의 반환값이 다음 리스너에게 전달되어 "원본 설정 유지 + 특정 필드 오버라이드"를 구현합니다
4. **next()**는 Promise이므로 반드시 await: await 없이 바로 spread하면 빈 객체를 얻게 되어 provider/model이 유실되고 오류가 발생합니다
5. **개발 원칙:** 확장 포인트를 먼저 찾을 것(90%의 동작에 공식 혹은 있음), 로직을 순수 함수로 분리할 것, 실기 검증은 생략 불가

<details><summary>본 장 목차</summary>

- 4.1 우리가 만들 것
- 4.2 프로젝트 골격
- 4.3 순수 함수: 결정 로직(의존성 없음, 단위 테스트 가능)
- 4.4 플러그인 본체: agent/request waterfall에 연결
- 4.5 테스트
- 4.6 초보자를 위한 3가지 개발 원칙

4.1 우리가 만들 것

문제: dsh는 매 도구 호출 전마다 모델이 다시 사고합니다(reasoning_effort). 50단계 도구 체인 작업에서는 "사고"가 벽시계 시간의 90%+ 를 차지합니다.

해결책: agent/request waterfall을 감시하는 host 플러그인을 만들어, 현재 단계의 최근 도구 호출을 기준으로 단순한 라운드의 reasoning_effort를 high에서 low로 낮춥니다.

4.2 프로젝트 골격

```
dsh-speed-plugin/
├─ package.json          # host 플러그인 선언
├─ tsconfig.json
├─ src/
│   └─ effort-decision.ts # 순수 함수: 결정 로직(의존성 없음, 단위 테스트 가능)
│       └─ index.ts       # apply(ctx): 확장 포인트에 연결
└─ tests/
    └─ effort-decision.spec.ts
```

package.json의 핵심 필드:

```
{
  "name": "dsh-speed-plugin",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

⚠ 의존성 버전은 반드시 ^0.1.0-rc.6 라인을 사용하세요 — rc.1 라인의 npm 의존성 체인은 끊어져 있습니다(제3장 혼란 트러블슈팅 참고).

4.3 순수 함수: 결정 로직(의존성 없음, 단위 테스트 가능)

src/effort-decision.ts:

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
```

```

name: string      // 도구 이름, 예: 'write', 'read', 'bash'
argsSize: number  // 파라미터 크기(문자 수)
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId      // 사용자 기준 강도
  allowDowngrade: boolean
  allowUpgrade: boolean
}

const SIMPLE_TOOL_RE = /^(fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|
touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected // 완전히 새 프롬프트: 기준값 유지

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)

  if (ratio >= 0.75 && allowDowngrade) return 'low'
  if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
  if (ratio < 0.75) return allowUpgrade ? 'high' : selected
  return selected
}

```

왜 순수 함수로 분리했는가: 결정 로직을 dsh 런타임에서 분리했습니다 — 단위 테스트는 의존성 없이 밀리초 단위로 실행되며 모든 분기를 커버할 수 있고, 실기에서는 "주입이 실제로 일어났는지"만 검증하면 됩니다.

4.4 플러그인 본체: agent/request waterfall에 연결

src/index.ts:

```

import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface SpeedPluginConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: SpeedPluginConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {

```

```

const events = (agent as { session?: { events?: readonly
unknown[] } }).session?.events ?? []
const out: ToolCallSample[] = []
for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
  const e = events[i] as { type?: string; data?: { name?: string; arguments?:
unknown } } | undefined
  if (e?.type !== 'tool/call') continue
  out.push({
    name: e.data?.name ?? 'tool',
    argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
  })
}
return out.reverse()
}

export function apply(ctx: Context, config: SpeedPluginConfig = DEFAULT_CONFIG): void
{
  if (!config.enabled) return

  // 경계 어댑팅: npm 패키지가 공식 이벤트 타입 확장을 re-export하지 않아서, 여기서 시그니처를 느슨하게 처리(제3장
흔한 트러블슈팅)
  const on = ctx.on as unknown as (
    event: string,
    handler: (payload: Record<string, unknown>, next: () => unknown) => unknown |
Promise<unknown>,
  ) => void

  on('agent/request', async (payload, next) => {
    const seed = await next() as { reasoningEffort?: unknown } // ▲ 반드시 await!
    const calls = recentToolCalls(payload.agent)
    const effort = decideEffort({
      recentCalls: calls,
      selected: config.baseline,
      allowDowngrade: config.allowDowngrade,
      allowUpgrade: config.allowUpgrade,
    })
    console.log(`[speed-plugin] calls=${JSON.stringify(calls)} => reasoningEffort=${$
{effort}}`)
    return { ...seed, reasoningEffort: effort }
  })
}

```

세 가지 핵심 포인트(전부 실제로 겪은 트러블슈팅):

1. **next()**는 **Promise**입니다: `await next()`로 현재 설정을 가져옵니다; `await` 없이 바로 `spread`하면 빈 객체를 얻게 되어 → `provider/model`이 유실 → 오류가 발생합니다.
2. **waterfall** 시맨틱: 리스너의 **반환값**이 다음 리스너/최종 요청에 전달됩니다. `{...seed, reasoningEffort}`를 반환하는 것이 바로 "원본 설정 유지 + 추론 강도 오버라이드"입니다.
3. **agent/request**는 매 단계마다 트리거됩니다: `agent-loop`의 `buildRequest`는 매 단계마다 이 `waterfall`을 거칩니다 — 그래서 동적 결정이 자연스럽게 단계별로 적용됩니다.

4.5 테스트

플러그인 테스트는 "모듈이 로드되는가"에서 멈추면 안 됩니다. 3층 검증을 권장합니다: 순수 함수 단위 테스트는 비즈니스 규칙을, waterfall 계약 테스트는 런타임 경계를, 실기 검증은 완전한 agent 루프를 담당합니다.

1층: 순수 함수 단위 테스트

순수 함수 테스트는 런타임 의존성이 없고 실행이 빠르며, 모든 결정 분기를 커버하기에 적합합니다:

```
import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {
  expect(decideEffort({
    recentCalls: [{ name: 'write', argsSize: 40 }],
    selected: 'high', allowDowngrade: true, allowUpgrade: true,
  })).toBe('low')
})
// ... 더 많은 분기: 완전히 새 프롬프트는 기준값 유지 / 다운그레이드 비활성화 / 초대형 페이로드는 max로 업그레이드 /
혼합 도구는 high로 업그레이드
```

2층: waterfall 계약 테스트

순수 함수가 테스트를 통과했다고 해서 플러그인의 연결이 올바르다는 뜻은 아닙니다. 가장 흔한 숨은 오류는 `await next()`를 빠뜨리는 것으로, 이는 상위의 `provider`, `model`, `tools` 등 필드를 조용히 유실시킵니다. 계약 테스트는 최소한의 Context 대역으로 리스너를 캡처하며, API Key도 필요 없고 dsh를 시작할 필요도 없습니다:

```
it('awaits next, preserves the seed, and only overrides reasoning effort', async () => {
  const next = async () => ({
    provider: 'deepseek-official',
    model: 'deepseek-reasoner',
    reasoningEffort: 'max',
    tools: ['read', 'write'],
  })

  const result = await runRegisteredHandler({ agent: { session: { events: [] } } }, next)

  expect(result).toEqual({
    provider: 'deepseek-official',
    model: 'deepseek-reasoner',
    reasoningEffort: 'high',
    tools: ['read', 'write'],
  })
})
```

함께 제공되는 템플릿의 tests/plugin.spec.ts는 다음도 커버합니다:

- agent/request 리스너를 단 하나만 등록하는지;
- 최근 8개의 tool/call만 읽고 다른 이벤트는 무시하는지;
- 다운스트림 waterfall이 오류를 던질 때 그대로 위로 전파되는지, 조용히 삼키지 않는지.

전체 자동 테스트 실행:

```
cd examples/plugin-template
npm install
npm test
npm run typecheck
```

3층: 실제 agent 루프 검증

계약 테스트는 플러그인의 경계가 예상대로 작동함을 증명하지만, 실제 런타임을 대체할 수는 없습니다. 플러그인을 장착하고(제3장 방법) → dsh web 재시작 → 파일 생성 작업 하나를 보내고 → dsh 프로세스 로그를 관찰합니다:

```
[speed-plugin] agent/request: calls=[] => reasoningEffort=high
[speed-plugin] agent/request: calls=[{"name":"write",...}] => reasoningEffort=low
```

첫 번째 라운드는 도구 호출이 없어 → 기준값 high를 유지; write 도구가 감지되면 → 다음 라운드에서 low로 낮춰집니다. 주입 경로가 완전히 작동합니다.

이 계층을 CI에 넣고 싶다면 공식 디스커션 #462: API Key 없이 waterfall 동작 검증하기

(<https://github.com/deepseek-ai/deepseek-harness/discussions/462>)를 참고하세요: mock LLM으로 고정된 도구 호출을 생성하고, headless profile을 통해 실제 agent loop를 실행한 다음, 감사 스냅샷으로 예상 waterfall과 tools/result를 확인합니다. 디스커션의 명령어는 특정 rc/master 소스 코드를 기반으로 하므로, 재사용할 때는 목표 버전의 스크립트와 이벤트 목록을 기준으로 삼아야 하며, 고정된 이벤트 개수를 버전 간 단언으로 삼지 마세요.

완전히 실행 가능한 코드: examples/plugin-template/ 참고.

⚠️ **강도 지원은 어댑터/모델마다 다릅니다(실제 트러블슈팅):** `decideEffort`가 `low`를 반환한 후, 현재 provider의 어댑터 능력 표가 `low`를 지원하지 않으면(예: `deepseek-official` 어댑터는 `off/high/max`만 지원), 요청이 `does not support reasoning effort "low"` 오류를 던집니다 — 이것은 어댑터 공백이지, 플러그인 버그가 아닙니다(공식 API는 실제로 `low`를 지원합니다, api-docs.deepseek.com/guides/thinking_mode/ 참고; FAQ Q4에 강도 설명 있음).

모델 능력 표에 맞춰 어댑팅하는 작성법: 다운그레이드 목표를 먼저 어댑터 지원 여부로 확인하고, 사용 가능한 강도로 매핑합니다.

```
``ts
```

```
const SUPPORTED = ['off', 'high', 'max'] // 현재 어댑터 능력 표 기준
```

```
const effort = decideEffort({...}) // 플러그인이 원하는 강도
```

```
const final = SUPPORTED.includes(effort) ? effort : (effort === 'low' ? 'high' : effort) // low가 지원되지 않으면 high로 폴백
```

```
,
```

4.6 초보자를 위한 3가지 개발 원칙

1. **확장 포인트를 먼저 찾을 것:** 바꾸고 싶은 동작의 90%는 공식 훅이 있습니다(`agent/request`, `settings`, `conversationEvents`, `slots`) — 코어를 fork하지 마세요.
2. **로직을 순수 함수로 분리할 것:** 결정/계산 로직을 `dsh`에서 분리 → 단위 테스트가 밀리초 단위, 모든 분기 커버.
3. **경계와 실기 모두 검증할 것:** 계약 테스트는 `next()` 전달과 필드 보존을 증명하고, 실기 로그는 실제 `agent loop`가 작동함을 증명합니다.

📖 **공식 cookbook 추가 참고자료**(2026-08 공식 신규 추가, 공식 저장소 `docs/cookbook/`): `adding-a-package.md`(패키지 추가법), `adding-a-tool.md`(도구 추가법), `adding-a-conversation-node.md`(대화 노드 추가), `adding-an-llm-adapter.md`(LLM 어댑터 작성), `adding-a-vendored-package.md`(vendor 패키지), `extension-cookbook.md`(확장 포인트 모음). 본 장은 "최소 속도 개선 플러그인" 경로를 다뤘고, 공식 cookbook은 더 많은 확장 포인트 유형을 커버하니 심화 학습 시 대조하며 읽으세요.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 이 속도 개선 플러그인의 3층 아키텍처(순수 함수 레이어 / 플러그인 본체 레이어 / 테스트 레이어)가 각각 무엇을 담당하는지 말해보세요

정답 확인: 본 장 4.2-4.5 절의 파일 구조 참고

2. 이해 문제: decideEffort를 왜 apply(ctx) 안에 직접 로직을 쓰지 않고 순수 함수로 설계했는지 설명하세요. 결정 로직이 ctx.session에 의존했다면 여전히 단위 테스트가 가능할까요?

정답 확인: 본 장 4.3 절 "왜 순수 함수로 분리했는가" 단락 참고

3. 실습 문제: decideEffort에 새 테스트 케이스를 작성해보세요: recentCalls에 단순 도구 3개 + 초대형 파라미터(argsSize = 5000) 1개가 있을 때 어떤 강도를 반환해야 할까요? 테스트 코드를 작성하고 실행해보세요

정답 확인: 본 장 4.5 절 단위 테스트 예시 참고. 주의: 단순 3개 + 초대형 1개 = 총 4개 호출, ratio = 0.75로 정확히 첫 번째 분기에 해당(ratio >= 0.75 && allowDowngrade) → low 반환(allowDowngrade에 따라 달라지며, allowUpgrade 분기는 실행되지 않음)

4. 실습 문제: apply(ctx) 안에서 await next()를 const seed = next()(await 없이)로 바꾸면 무슨 일이 일어날까요? 추론을 작성한 다음 실기에서 검증해보세요

정답 확인: 본 장 4.4 절 "세 가지 핵심 포인트"의 1번 참고

5. 실습 문제: 비슷한 플러그인을 작성한다고 가정하되, "현재 세션의 도구 호출 총 횟수"에 따라 강도를 결정하도록 바꿔보세요(20회를 초과하면 자동으로 low로 낮춤). 순수 함수 시그니처와 핵심 로직을 작성해보세요

정답 확인: 본 장 4.3 절 순수 함수 설계 패턴 참고, 핵심은 입출력 타입 정의

6. 사고 문제: agent/request waterfall에는 여러 리스너가 있을 수 있습니다. 속도 개선 플러그인과 다른 플러그인이 둘 다 agent/request를 감시한다면, 누가 먼저 실행될까요? 반환값은 어떻게 전달될까요?

정답 확인: 본 장 4.4 절 "waterfall 시맨틱" 단락 + 제3장 3.4 절 확장 포인트 설명 참고

자주 묻는 질문 FAQ

Q1: next()가 반환하는 seed에는 정확히 어떤 필드가 들어있나요?

seed는 현재 waterfall 체인의 상류에서 누적된 요청 설정으로, 보통 provider, model, reasoningEffort, tools 등의 필드를 포함합니다. 여러분의 리스너가 seed를 받으면, spread로 바꾸고 싶은 필드를 오버라이드하

고 나머지는 그대로 전달합니다. 구체적인 필드는 공식 agent-loop 패키지의 타입 정의를 기준으로 하세요(rc 단계에서 변경될 수 있음).

Q2: 왜 npm 패키지의 타입 시그니처를 "느슨하게" 처리해야 하나요? 공식 타입을 바로 쓸 수 없나요?

npm에 게시된 @deepseek-ai/dsh-agent 등의 패키지가 내부 이벤트 타입 확장을 re-export하지 않기 때문입니다(Events 인터페이스가 module augmentation으로 확장되지 않음).

ctx.on('agent/request', ...)을 직접 쓰면 타입 오류가 발생합니다. 해결책은 경계에서 as unknown as로 변환하는 것이며, 이는 rc 단계의 임시 방편이고 정식 버전에서는 수정될 수 있습니다.

Q3: 제 플러그인이 매 도구 호출 후마다 결정을 트리거해야 하는데, agent/request로 충분한가요?

충분합니다. agent/request는 매 모델 요청 전에 트리거됩니다(도구 호출 후 다음 라운드 요청도 포함). 리스너 안에서 최근 도구 호출 이력을 읽기만 하면 됩니다(payload.agent.session.events에서 역순으로 tool/call 이벤트를 찾음), 이렇게 하면 단계별 결정이 가능합니다.

Q4: 순수 함수 테스트도 통과했고 실기 검증도 통과했는데, 사용자가 "가끔 효과가 없다"고 피드백한다면 어떤 이유일 수 있나요?

몇 가지 가능성이 있습니다: ① 사용자의 settings.yaml에서 reasoningEffort가 이미 low라서 다운그레이드 효과가 없음(이미 최저치); ② 사용자의 작업이 전부 단순한 도구라서 원래도 빨라서 차이를 체감하지 못함; ③ waterfall 안의 다른 플러그인이 여러분의 반환값을 덮어씀. 각 단계의 입력/출력 강도를 로그로 남길 것을 권장합니다.

Q5: 플러그인에 사용자가 설정할 수 있는 스위치(설정 페이지에서 켜고 끌 수 있는)를 추가하고 싶은데, 어떻게 하나요?

settings 서비스로 네임스페이스(예: speed-plugin)를 등록하고, 설정 항목(enabled, baseline 등)을 선언하세요. dsh의 설정 페이지가 자동으로 폼을 렌더링하고, 사용자가 수정하면 ctx.get으로 읽을 수 있습니다. 구체적인 API는 공식 dsh-settings 패키지 문서를 참고하세요.

Q6: recentToolCalls 함수에서 왜 reverse()를 하나요?

events 배열의 끝에서부터 앞으로 순회하기 때문입니다(최근 N개를 가져옴), 결과는 역순입니다(최신이 앞에 옴).
reverse로 시간 정순을 복원하면(가장 오래된 것이 앞에 옴) 실제 호출 순서와 일치해서, 결정 로직이 "최근 윈도우"를 이해하기 쉬워집니다.

제 5 장: dsh가 당신을 도울 수 있는 것 — 활용 시나리오 전체 조망

본 장의 목표: "dsh를 어떻게 개발하는가"의 관점을 잠시 내려놓고, **사용자의 관점에서** dsh가 어떤 일상 시나리오에서 실제로 도움이 되는지 살펴봅니다. 각 시나리오마다 그대로 따라 할 수 있는 작업 예시와 실제 소요 시간 참고자료를 제공합니다.

TL;DR(본 장 핵심, 30초 버전)

1. **dsh는 범용 Agent 런타임입니다:** 코딩 어시스턴트에 그치지 않습니다 — 데이터 분석, 문서 생성, 자동화, 코드 이해까지 모두 가능합니다
2. **다섯 가지 핵심 시나리오:** 일상 코딩 / 데이터 처리 / 문서 지식 작업 / 자동화 운영 / 코드 이해와 리팩터링
3. **일관된 사용법:** 한 문장으로 목표를 설명하고 검수 기준을 명확히 쓰면, dsh가 자동으로 도구 체인을 오케스트레이션해 완료합니다
4. **실제 속도:** 단순 작업은 초 단위, 복잡한 다단계 작업은 1-5분(V4-Flash + high 강도 실측)
5. **프롬프트 황금률:** dsh에게 "완료의 기준이 무엇인지"를 알려주는 것(예: "실행 검증 통과")이 과정을 설명하는 것보다 효과적입니다

<details><summary>본 장 목차</summary>

- 5.1 시나리오 개요
 - 5.1.1 고개시 적중률: dsh의 비용 비밀 무기
- 5.2 시나리오 1: 일상 코딩(입문에 가장 적합한 시나리오)
- 5.3 시나리오 2: 데이터 처리와 분석
- 5.4 시나리오 3: 문서와 지식 작업
- 5.5 시나리오 4: 자동화와 운영(headless가 필살기)
- 5.6 시나리오 5: 코드 이해와 리팩터링
- 5.7 시나리오를 현실화하는 세 가지 프롬프트 법칙
- 5.8 나의 첫 실제 작업(15분 시작하기)
- 5.9 산업 관점: dsh가 서로 다른 영역에서 활용되는 방식

5.1 시나리오 개요

시나리오	전형적인 작업	dsh의 도구	실측 참고
일상 코딩	함수 작성, 버그 수정, 테스트 보완	write / read / bash / grep	5개 버그 수정+49개 테스트: 94초
데이터 처리	정제, 분석, 시각화, 리포트	read / write / bash / python	데이터 정제+시각화: 186초
문서 지식	API 문서, 튜토리얼, 요약, 번역	read / write	API 문서 생성: 1분 04초
자동화 운영	배치 처리, CI 작업, 로그 분석	bash / headless	headless 스크립트화
코드 이해	리팩터링, 리뷰, 기술 조사	grep / glob / read / bash	코드 리팩터링: 4분 37초

모든 실측치는 백서 제10장의 실제 사례(합성 데이터)에서 가져왔습니다. 자세한 내용은 benchmark 참고.

5.1.1 고캐시 적중률: dsh의 비용 비밀 무기

이것은 dsh 생태계에서 가장 저평가된 장점입니다. DeepSeek API의 **context cache**(프롬프트 캐시): 반복/유사한 입력 토큰은 캐시 가격으로 과금됩니다. 실측(실제 dsh 세션 통계) 결과 **캐시 적중률이 97%에 달합니다**.

왜 dsh의 캐시 적중률이 특히 높은가:

이유	설명
세션 연속성	같은 세션에서 여러 라운드 대화 시, 시스템 프롬프트/스킬 카탈로그/이력이 반복됨 → 적중
안정적인 도구 스키마	매 단계 요청마다 동일한 도구 정의를 전달함 → 적중
Agent 워크로드 특성	다단계 도구 체인이 반복적으로 동일한 컨텍스트를 전달함 → 적중률이 자연히 높음

비용에 미치는 영향(98% 캐시 할인):

과금 항목	미적중 가격	캐시 적중 가격	할인율
입력(Flash)	\$0.14/M	\$0.0028/M	98%
입력(Pro)	\$0.435/M	\$0.003625/M	99%+

실질적 의미: 긴 세션의 Agent 작업에서는 대부분의 입력 토큰이 캐시 가격으로 처리됩니다 — **실측에서 "50단계 도구 체인 작업, 240만 입력 토큰"의 비용이 미적중 가격 기준 예상치보다 훨씬 낮았습니다**. 고빈도/배치/긴 세션 시나리오에서는 이것이 자릿수 단위의 비용 우위입니다.

캐시 적중률을 더 높이는 방법: **세션 연속성 유지**(자주 새 세션을 열지 마세요), **prompt 접두사 안정화**(시스템 프롬프트/스킬 카탈로그를 자주 바꾸지 마세요), 배치 작업은 같은 세션/같은 접두사에 두세요.

5.2 시나리오 1: 일상 코딩(입문에 가장 적합한 시나리오)

전형적인 작업:

- 요구사항에 따라 새 함수/모듈 작성
- 버그 수정(위치 찾기 → 수정 → 검증)
- 단위 테스트 보완

작업 예시(headless 원클릭 실행):

```
dsh --profile headless "buggy_calculator.py를 검토해: 모든 버그를 찾아 수정하고, 완전한 테스트를 작성하고, 실행해서 전부 통과하는지 검증해"  
# 실측: 5개 버그 전부 수정 + 49개 테스트 통과 (94초)
```

dsh가 적합한 이유: 코딩은 도구 체인이 가장 완결된 시나리오입니다 — 파일 읽기, 코드 작성, 명령 실행, 결과 확인이 매끄럽게 페루프를 형성합니다. 산출물(수정된 파일)은 대화 끝에서 바로 열어볼 수 있습니다.

프롬프트 팁: 반드시 "실행 검증"을 써야 합니다 — dsh는 그대로 페루프를 완성합니다; "버그 수정 도와줘"만 쓰면 수정만 하고 멈춥니다.

5.3 시나리오 2: 데이터 처리와 분석

전형적인 작업:

- 데이터 품질 점검과 정제
- 통계 분석
- 시각화(차트)
- 분석 리포트 생성

작업 예시:

```
dsh --profile headless "sales_data.json의 데이터 품질을 분석해(결측/타입 오류/중복/이상치), 정제 스크립트+시각화 스크립트를 작성하고, 실행 검증한 후 처리 방안을 요약해"  
# 실측: 52→35줄로 정리 + 차트 + 트레이드오프 설명(186초)
```

하이라이트(실측): dsh는 실행만 하는 게 아니라 **트레이드오프를 설명**합니다 — "중위수로 채우면 스파이크가 생길 수 있음", "이상치를 삭제할지는 비즈니스에 달려 있음" — 이것이 agent 엔지니어링 레이어가 "생각한다"는 직접적인 증거입니다.

5.4 시나리오 3: 문서와 지식 작업

전형적인 작업:

- 코드에서 API 문서 생성
- 노트를 구조화된 문서로 정리
- 긴 글 요약, 번역

작업 예시:

```
dsh --profile headless "user_module.py(함수 8개)에 대한 완전한 Markdown API 문서를 생성해: 파라미터 표, 반환값, 예외, 예시, 경계 상황 포함"  
# 실측: 423줄 ReadTheDocs 스타일 문서(1분 04초), "문서 계약 vs 코드 구현" 차이를 능동적으로 식별
```

하이라이트(실측): dsh는 **방어적 글쓰기**를 합니다 — 호출 함정을 표시하고(음수 limit의 동작, 빈 파라미터의 동작), 문서 선언과 구현이 일치하지 않는 부분을 발견하면 있는 그대로 설명합니다.

5.5 시나리오 4: 자동화와 운영(headless가 필살기)

전형적인 작업:

- 매일 정해진 시간에 데이터 작업 실행(일일 리포트, 모니터링 요약)
- 배치 파일 처리(이름 변경, 형식 변환)
- CI에서의 자동화 검사

작업 예시(스크립트화):

```
# cron으로 매일 8시에 일일 리포트 생성
dsh --profile headless "워크스페이스의 git log를 읽고 한국어 일일 리포트를 생성해" > daily-report.md

# 배치 처리: 디렉터리 안의 모든 .txt를 .md로 변환
dsh --profile headless "docs/ 아래의 모든 .txt를 .md로 변환해(구조 유지)"
```

headless가 필살기인 이유: 프로세스 레벨 도구(bash/파일/검색)를 전부 사용할 수 있음 + 0이 아닌 종료 코드가 곧 실패 + 출력을 파이프로 처리 가능 — 이것은 "CI에 넣을 수 있는 Agent"입니다.

5.6 시나리오 5: 코드 이해와 리팩터링

전형적인 작업:

- 절차형 코드를 OOP로 리팩터링
- 전체 저장소 코드 리뷰
- 기술 조사("이 모듈은 어떻게 동작하는가")

작업 예시:

```
dsh --profile headless "legacy_orders.py를 객체지향+책임 분리 구조로 리팩터링해, 동작은 그대로 유지하고,
리팩터링 전후 출력이 같은지 테스트로 검증해"
# 실측: 17/17 테스트 통과, 스크립트 레벨 산출물 비교 포함(4분 37초)
```

하이라이트(실측): dsh의 엔지니어링 인식 — 중복 코드를 식별해 단일 진입점으로 병합, 하위 호환성 유지, 샌드박스에서 tempfile 권한 문제를 발견하고 능동적으로 방안을 변경.

5.7 시나리오를 현실화하는 세 가지 프롬프트 법칙

1. **검수 기준을 쓰고, 과정을 쓰지 마세요:** "실행 검증 통과" > "스크립트 하나 작성해줘" — dsh가 스스로 단계를 오케스트레이션합니다
2. **컨텍스트를 주세요:** 배경을 한 문장으로 명확히 설명하세요(파일이 어디 있는지, 데이터가 어떻게 생겼는지, 목표 독자가 누구인지)
3. **한 번에 하나의 작업:** 복잡한 목표는 여러 라운드로 쪼개는 것이(라운드마다 하나의 작은 페루프) 한 번에 거대한 작업을 주는 것보다 더 신뢰할 수 있습니다

5.8 나의 첫 실제 작업(15분 시작하기)

1. 테스트 디렉터리를 하나 만들고, 손에 있는 작은 파일(코드/데이터/노트)을 하나 넣으세요
2. 실행: `dsh --profile headless` "이 파일을 요약하고, 개선할 수 있는 3가지를 짚어줘"
3. 실행: `dsh --profile headless` "이 파일에 대한 테스트를 작성하거나/문서를 생성해줘"
4. 출력을 비교하며 dsh의 "완성도"를 체감해보세요 — 대략적인 것만 준 건 아닌가요? 검수 기준을 명확히 썼나요?

실습 문제

1. **시나리오 식별:** 여러분이 가진 3개의 일상 작업을 주고, 각각 어느 시나리오(코딩/데이터/문서/자동화/이해)에 속하는지 판단해보세요
2. **프롬프트 비교:** 같은 작업에 대해 두 가지 버전의 프롬프트를 작성해보세요 — 하나는 "과정을 설명", 하나는 "검수 기준을 작성" — 한 번씩 실행해 결과를 비교하세요
3. **headless 스크립트화:** "매일 git 일일 리포트"를 한 줄의 cron/bash 명령으로 작성하고 출력을 검증하세요
4. **다단계 작업:** 시나리오를 넘나드는 작업을 설계해보세요(예: "코드 읽기 → 문서 생성 → 리포트 출력"), dsh가 어떻게 오케스트레이션하는지 관찰하세요
5. **사고 문제:** 어떤 시나리오에는 dsh가 적합하지 않을까요?(힌트: 실시간 상호작용, 사람의 승인이 필요한 민감한 작업)

자주 묻는 질문(FAQ)

- **Q: dsh는 코딩 어시스턴트일 뿐인가요?** 아닙니다 — 도구 체인(읽기/쓰기/실행/검색)이 범용성을 만들어 줍니다. 데이터 처리/문서/자동화 모두 검증되었습니다
- **Q: 복잡한 작업은 얼마나 걸리나요?** 실측 1-5분(V4-Flash); 더 빠르게 하려면 추론 강도를 낮추세요(제6장)
- **Q: 작업이 실패하면 어떻게 하나요?** 출력의 도구 호출 트레이스를 보고 원인을 찾으세요; 대부분은 "검수 기준이 불명확"한 경우이니, 프롬프트를 다시 써서 재실행하세요
- **Q: 자동으로 실행할 수 있나요?** 가능합니다 — `headless + 00`이 아닌 종료 코드 + 파이프 출력으로 cron/CI에 넣을 수 있습니다

- **Q: 안전한가요?** 도구 실행에는 샌드박스와 승인 레이어가 있습니다; 민감한 작업은 사람의 확인을 권장합니다(제8장 보안 모델)
- **Q: Claude Code를 직접 쓰는 것과 무엇이 다른가요?** 같은 모델이라면 능력은 비슷하지만, dsh는 도구 체인/인터페이스/플러그인을 커스터마이징할 수 있습니다(제1장 매트릭스 참고)

5.9 산업 관점: dsh가 서로 다른 영역에서 활용되는 방식

같은 도구 체인이지만 산업마다 사용법이 다릅니다. 아래는 범용 시나리오입니다(실제 비즈니스 데이터 미포함). 규제가 민감한 영역(의료/법률)은 반드시 사람이 직접 검토하세요.

금융과 투자 리서치

- **작업:** 재무제표 요약, 리서치 보고서 핵심 추출, 팩터 설명 문서, 투자 메모
- **예시:** `dsh --profile headless "이 재무 데이터를 읽고, 전년 대비/전분기 대비 변화가 포함된 요약 리포트를 생성해"`
- **dsh의 장점:** 데이터 처리 파이프라인이 손에 익음 + 고캐시 적중률(긴 리포트 세션이 저렴함)

교육과 학습

- **작업:** 교안 생성, 문제 풀이, 오답 정리, 학습 계획
- **예시:** `dsh --profile headless "이 챕터의 지식 포인트를 자가 테스트 문제 10개로 정리해줘 (해설 포함)"`
- **dsh의 장점:** 문서 생성 + 구조화가 강점, 배치로 문제 출제/정리 가능

영업과 고객 서비스

- **작업:** FAQ 정리, 스크립트 생성, 고객 피드백 분류
- **예시:** `dsh --profile headless "고객 피드백 100건을 문제 유형별로 분류하고 개선 제안을 제시해 줘"`
- **dsh의 장점:** 배치 텍스트 처리 + 분류 정리

운영과 콘텐츠

- **작업:** 주간/월간 리포트, 데이터 대시보드 설명, 콘텐츠 주제 선정, SEO 요약

- 예시: `dsh --profile headless` "이번 주 데이터 파일을 읽고 운영 주간 리포트를 생성해줘 (차트 코드 포함)"

- **dsh의 장점:** 데이터 + 문서 + 자동화의 조합

연구와 리뷰(학술/기술)

- **작업:** 논문 핵심 추출, 기술 조사, 문헌 비교

- 예시: `dsh --profile headless` "이 논문 3편의 방법론과 결론을 비교해서 비교표를 출력해줘"

- **dsh의 장점:** 긴 문서 처리 + 구조화된 출력(1M 컨텍스트)

⚠️ 규제 준수 안내

의료 진단, 법률 의견, 재무 자문 등 **고위험 시나리오**에서는: dsh의 출력을 반드시 사람이 검토해야 하며, 의사결정에 직접 사용할 수 없습니다 — 도구 실행에는 샌드박스가 있지만, **콘텐츠에 대한 책임은 사용자에게 있습니다.**

제 6 장: 심화와 성능 튜닝

본 장의 목표: "돌아간다"에서 "잘 돌아간다"로 — 추론 강도 전략, 도구 호출 소요 시간 분석, 실제 트러블슈팅 목록.

TL;DR(본 장 핵심, 30초 버전)

1. 시간이 어디로 가는가: 모델 사고가 ~90%, 도구 실행 <1%, 네트워크/렌더링 ~10% — 사고 시간을 최적화하는 것이 가성비가 가장 높은 속도 개선책입니다
2. 3단계 전략: low(단순/배치 라운드), high(일상 기본값), max(복잡한 추론/디버그) — 수동으로 UI에서 선택하거나 플러그인이 자동으로 조절
3. 소요 시간 시각화: Web UI 하단 통계 행(LLM Xs / 도구 호출 Ys)이 병목을 찾는 가장 빠른 방법입니다
4. 8개의 실제 트러블슈팅: rc.1 의존성 단절, 플러그인 main 누락, next() await 누락, 이벤트 타입 인식 실패, client 테스트 실행 불가, 캐시 적중 오판, 포트 점유, 추론 직렬화 시 reasoning 유실(#739)
5. 평가 시 물어야 할 세 가지: 누가 측정했나, 어떤 harness인가, 검증기는 얼마나 엄격한가 — 성적표는 조건과 함께 봐야 합니다

<details><summary>본 장 목차</summary>

- 6.1 성능 모델: dsh의 시간은 어디로 가는가
- 6.2 reasoning_effort 전략(공식 3단계)
- 6.3 도구 호출 소요 시간 시각화
- 6.4 흔한 트러블슈팅 목록(실제 경험, 해법 포함)
- 6.5 평가 관점: 공식 성적표 vs 독립 실측
- 6.6 캐시 전략: 매 호출을 더 저렴하게 만들기

6.1 성능 모델: dsh의 시간은 어디로 가는가

"파일 생성" 작업 하나의 소요 시간 분포를 실측했습니다:

단계	비율	설명
모델 사고(Think)	~90%(단일 저장소/명확한 위치 시나리오)	매 도구 호출 전마다 다시 사고하는 것이 절대적인 대부분을 차지

도구 호출/검색(Tool)	다중 저장소 시나리오에서는 87%까지 역전 가능(#2019 (https://github.com/deepseek-ai/deepseek-harness/discussions/2019) 실측: LLM 4분 8초 / 도구 29분 17초)	workspace/에 여러 git 저장소가 있으면 harness가 저장소 사이를 헤매며 위치를 찾음 — 검색/위치 확인 비용이 지 사고 비용이 아님; 범위를 명시하면 1분 만에 해결
도구 실행	<1%	파일 쓰기 등은 밀리초 단위
네트워크/렌더링	~10%	API 왕복 + UI 업데이트

추론:

- 단순 작업 → 사고 시간을 최적화(추론 강도 낮추기)
- 긴 도구 체인 작업 → 매 단계마다 사고 시간을 절약하면 누적 이득이 가장 큼
- 도구 자체가 느린 경우(검색/대용량 파일) → 강도 조절이 아니라 도구 구현을 최적화

6.2 reasoning_effort 전략(공식 3단계)

강도	시나리오 추천
low	단순/결정론적 라운드: 파일 조작, 배치, 도구 체인 중 저렴한 단계
high	일상적인 Agent 작업(기본값)
max	복잡한 추론, 긴 체인 계획, 디버그

수동: UI의 "추론 등급" 선택기, 또는 ~/.dsh/settings.yaml의 reasoningEffort.

자동: 플러그인이 도구 라운드에 따라 동적으로 강도를 낮춤(속도 개선 플러그인 예제, 제4장).

"공식 3단계"는 기본 DeepSeek 어댑터(deepseek-official)에서는 off / high / max입니다; low는 pi-ai(opencode-go) 게이트웨이의 강도입니다(본 백서의 실측 환경, 즉 benchmark가 사용하는 게이트웨이). 도구 체인 강도 다운그레이드 서사는 게이트웨이 환경에서 성립하며, settings.yaml에서 기본 provider를 쓸 때는 off를 사용하세요.

6.3 도구 호출 소요 시간 시각화

dsh의 세션 통계 행(Web UI 하단)에는 다음이 표시됩니다: N 라운드 · M 단계 | LLM Xs · 도구 호출 Ys |

첫 토큰 평균 ... — 이것이 병목을 찾는 가장 빠른 방법입니다.

심화: host 플러그인이 도구 이벤트를 감시해 도구별 시간을 측정합니다(속도 개선 플러그인 예제에 이미 host 로 그 버전이 내장되어 있음), "어떤 도구가 가장 느린지"를 드러냅니다.

6.4 혼한 트러블슈팅 목록(실제 경험, 해법 포함)

#	트러블	증상	해법
1	rc.1 의존성 단절	pnpm install 404(dsh-type-meta 등이 게시된 적 없음)	의존성은 ^0.1.0-rc.6 라인 사용
2	플러그인에 main 없음	No "exports" main defined	. 엔트리 노출; "main": "src/index.ts"는 tsx로 로드 가능
3	next()에서 await 누락	provider/model 유실로 오류 발생	agent/request의 next()는 Promise를 반환하므로 반드시 await
4	이벤트 타입 인식 실패	'agent/request' is not assignable to keyof Events	npm이 타입 확장을 re-export하지 않아 경계에서 시그니처를 느슨하게
5	client 테스트 실행 불가	jsdom에서 window.__ModuleLoader__ undefined 오류	client 산출물이 dsh 부트스트랩 메커니즘에 의존, 컴포넌트 테스트는 공식 CI에서 실행
6	단순 작업이 "갑자기 빨라짐"에 대한 오판	1초 vs 110초 차이를 잘못 귀속	DeepSeek context cache 적중도 속도를 높임 — A/B 테스트는 완전히 새로운 prompt를 사용해야 함
7	포트 점유	dsh web이 시작되지 않음	netstat -ano findstr 3080으로 PID를 찾아 kill
8	추론 직렬화 시 빈 reasoning 생략(#739 (https://github.com/deepseek-ai/deepseek-harness/discussions/739))	검증 포인트: off(thinking:disabled) 강도는 정상(high/max에서만 발생); high/max에서도 구 호출 라운드에 사고 내용이 없을 때 , serializeAssistant의 toolCalls.length > 0 && reasoning.length > 0 조건이 빈 reasoning 필드를 생략함 → 다음 라운드 요청이 400 오류(invalid pi-ai replay state)	버그로 판단(reasoning 자리 표시자를 유지해야 함): 수정 방향은 직렬화 시 해당 필드를 항상 유지 하는 것

6.5 평가 관점: 공식 성적표 vs 독립 실측

(0813 정식 버전 출시와 결합하여) Agent 모델 성적표를 볼 때는 세 가지를 물어야 합니다:

1. 누가 측정했나? 공식 자체 측정(자체 Harness) vs 독립 서드파티(AA 등)
2. 어떤 harness인가? 프레임워크가 다르면 점수 차이가 큼(공식 Terminal-Bench 87.9 vs AA 독립 79)
3. 검증기는 얼마나 엄격한가? 느슨한 검증기(SWE-bench Verified 8.5% 위양성) vs 엄격한 검증기(DeepSWE 0.3%)

dsh의 agent/request waterfall은 모델 성능을 재현 가능하게 만듭니다 — 이것이 클로즈드소스 제품 대비 dsh의 엔지니어링 우위입니다.

6.6 캐시 전략: 매 호출을 더 저렴하게 만들기

제5장에서 고캐시 적중률의 가치를 다뤘습니다(실측 세션 캐시 적중률 97%; 캐시 할인 Flash 등급 98% / Pro 등급 99%+), 여기서는 **실행 가능한 전략**을 제시합니다:

목표	방법
적중률 높이기	긴 작업은 세션 연속성 유지 (자주 새 세션을 만들지 않기)
적중률 높이기	시스템 프롬프트/스킬 카탈로그 등 접두사를 안정적으로 유지 (설정을 자주 바꾸지 않기)
적중률 높이기	배치 작업은 같은 세션/같은 접두사에 두기(예: 같은 배치의 파일 분석)
비용 낮추기	단순한 라운드는 low 등급 사용(사고 토큰 감소 → 총 토큰 하락)
모니터링	세션 통계 행에서 "캐시 적중 %" 확인(Web UI 하단) — 예상보다 낮으면 접두사 안정성을 점검

비용 모델 요약: 총 비용 $\approx$ 출력 토큰 $\times$ 출력 가격 + 입력 미적중 $\times$ 미적중 가격 + 입력 적중 $\times$ 적중 가격 — Agent 워크로드의 입력 비중이 높아서, **캐시 적중률이 비용의 첫 번째 변수**입니다.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 dsh 작업 소요 시간의 세 가지 구성 요소와 각각의 비율을 말해보세요. 왜 "사고 시간 최적화"가 가성비가 가장 높은 속도 개선 수단일까요?

정답 확인: 본 장 6.1절 소요 시간 분포 표 참고

2. **이해 문제:** low / high / max 세 가지 추론 강도가 각각 어떤 시나리오에 적합한지 설명하세요. "파일 10개를 읽고 간단히 종합하는" 작업이라면 어떤 강도를 써야 할까요?

정답 확인: 본 장 6.2절 3단계 전략 표 참고

3. **실습 문제:** dsh web을 열고 "파일 3개 생성" 작업을 실행해, Web UI 하단의 통계 행(LLM Xs / 도구 호출 Ys)을 관찰하고, 데이터를 기록해 병목이 어디인지 분석해보세요

정답 확인: 본 장 6.3절 "소요 시간 시각화" 단락 참고

4. **실습 문제:** ~/.dsh/settings.yaml의 reasoningEffort를 high에서 low로 바꾸고, 같은 단순 작업을 실행해 소요 시간 차이를 비교하세요. 다시 high로 되돌리고, 복잡한 추론 작업을 실행해 차이를 비교하세요

정답 확인: 본 장 6.2절 "수동" 단락 참고

5. **실습 문제:** "포트 점유" 트러블을 재현하고(먼저 3080 포트를 점유하는 서비스를 하나 실행), netstat -ano | findstr 3080으로 PID를 찾아 kill한 다음, dsh web을 정상적으로 시작해보세요

정답 확인: 본 장 6.4절 트러블 #7 참고

6. **사고 문제:** 본 장 6.5절은 "평가 시 세 가지를 물어야 한다: 누가 측정했나, 어떤 harness인가, 검증기는 얼마나 엄격한가"라고 말합니다. 구체적인 예시로 설명해보세요: 같은 모델인데 harness가 다르면 왜 점수 차이가 크게 나는가요?

정답 확인: 본 장 6.5절 "공식 Terminal-Bench 87.9 vs AA 독립 79" 예시 참고

자주 묻는 질문 FAQ

Q1: 왜 제 dsh가 "가끔은 유독 빠르고, 가끔은 유독 느린가요"?

두 가지 주요 원인이 있습니다: ① DeepSeek context cache가 적중할 때는 첫 라운드 컨텍스트 주입이 훨씬 빨라 집니다(1초 vs 110초 차이가 여기서 나올 수 있음); ② 추론 강도가 다르면 사고 시간이 몇 배 차이 납니다. A/B 비교 테스트를 할 때는 완전히 새로운 prompt를 사용하고(캐시 적중 방지) + 강도를 고정해야 합니다.

Q2: 예제 속도 개선 플러그인이 실제로 얼마나 빨라지나요? 실측 데이터가 있나요?

속도 개선 효과는 작업 유형에 따라 다릅니다. 단순 작업(1-3단계)은 dsh가 원래 빨라서 플러그인 효과가 뚜렷하지 않습니다. 긴 도구 체인 작업(20-50단계)에서 이득이 가장 큽니다, 매 단계 사고 강도 다운그레이드의 누적 효과가 뚜렷하기 때문입니다. 구체적인 데이터는 제4장 예제 코드의 실측 로그를 참고하세요.

Q3: 추론 강도를 low로 설정하면 모델 품질이 많이 떨어지나요?

작업에 따라 다릅니다. 단순 파일 조작, 배치 처리, 결정론적 작업은 low를 써도 품질 차이가 거의 없습니다. 복잡한 추론, 긴 체인 계획, 디버그는 low를 쓰면 핵심 단계를 놓칠 수 있습니다. 권장: 일상에는 high를 쓰고, 배치/단순 작업은 수동으로 low로 전환하거나 속도 개선 플러그인으로 자동 다운그레이드하세요.

Q4: 트러블 #6에서 말한 "context cache 적중"이 뭔가요? 적중 여부는 어떻게 판단하나요?

DeepSeek API는 반복/유사한 컨텍스트를 캐시하며, 적중 시 첫 토큰 시간이 크게 단축됩니다. 판단 방법: dsh 세션 로그의 "첫 토큰 평균" 지표를 보고, 어느 순간 갑자기 10초+에서 1초 미만으로 떨어졌다면 캐시가 적중했을 가능성이 큼니다. 성능 비교 테스트를 할 때는 완전히 새로운 prompt를 사용해 캐시 간섭을 피하세요.

Q5: rc 단계의 파괴적 변경이 성능에 영향을 미치나요? 업그레이드 후 무엇을 주의해야 하나요?

영향을 줄 수 있습니다. rc 단계의 agent-loop, waterfall 시그니처, 플러그인 API는 모두 바뀔 수 있습니다. 업그레이드 후: ① dsh --version을 실행해 버전 확인; ② 공식 changelog에 breaking changes가 있는지 확인; ③ 익숙한 작업 하나를 실행해 소요 시간과 동작을 비교; ④ 커스텀 플러그인의 타입 호환성을 확인하세요.

Q6: 성능 벤치마크 테스트를 하고 싶은데, 조언이 있나요?

세 가지 원칙: ① 환경 고정(같은 모델, 같은 네트워크, 같은 강도); ② 캐시를 피하기 위해 완전히 새로운 prompt 사용; ③ 여러 번 실행해 중앙값을 취하기(단일 실행은 변동이 큼). 기록할 항목: 벽시계 시간, LLM 소요 시간, 도구 호출 소요 시간, 첫 토큰 시간. 본 장 6.1절의 소요 시간 분포 모델을 참고해 여러분의 benchmark를 설계하세요.

제 7 장: 생태계와 리소스

본 장의 목표: "dsh 생태계에 합류하는" 지도를 드립니다 — 공식 채널, 커뮤니티 현황, 플러그인 실습, 그리고 참여 방법.

TL;DR(본 장 핵심, 30초 버전)

1. **공식 채널:** GitHub 저장소(소스 + issue), 공식 문서 사이트(VitePress), API 문서, Discord, Discussions
— 현재 기여 채널은 Discussions가 중심입니다
2. **외부 PR은 아직 받지 않음**(2026-08-13 시점), 하지만 공식은 dsh-plugin 생태계 프로젝트 만들기, 튜토리얼/블로그 작성을 장려합니다 — 커뮤니티가 이미 자발적으로 "보고-재현-근본원인-수정 브랜치" 기여 템플릿을 만들었습니다(7.2절 참고)
3. **플러그인 생태계 스냅샷:** DSH-better-sidebar(가장 완성도 높은 커뮤니티 플러그인), 예제 속도 개선 플러그인(제4장에서 완전 분석), 본 백서 — 플러그인을 찾으려면 `topic:dsh-plugin` 검색
4. **초보자 참여 경로:** 사용해보기 → 작은 개선(커뮤니티 플러그인에 PR 제출) → 플러그인 공개 → 콘텐츠 작성
5. **생태계 초일 = 선점 우위:** 모든 초기 생태계에는 "가장 먼저 게를 먹어본 사람"의 보너스가 있으며, 지금이 딱 진입할 때입니다

<details><summary>본 장 목차</summary>

- 7.1 공식 채널
- 7.2 커뮤니티 기여 방식
- 7.3 플러그인 생태계(2026-08-13 스냅샷)
- 7.4 생태계에 참여하는 방법(초보자 경로)
- 7.5 추천 읽기 순서
- 7.6 실제 플러그인 설치 실습
- 7.7 플러그인을 찾는 방법
- 7.8 생태계 참여의 전체 흐름
- 7.9 생태계 마일스톤 타임라인

- 7.10 공식 디스커션과의 연계

7.1 공식 채널

리소스	주소	용도
공식 저장소	https://github.com/deepseek-ai/deepseek-harness	소스 코드, 아키텍처 문서, issue
공식 문서 사이트	https://deepseek-harness.github.io/deepseek-harness/	공식 VitePress 문서 사이트(guide/providers/development/reference, 2026-08 오픈)
제품 사이트	https://deepseek.com/harness/	공식 제품 소개(4가지 실행 모드, 트레이스 뷰 등)
API 문서	https://api-docs.deepseek.com	모델, 가격, API 가이드
Discord	공식 README 내 링크	커뮤니티 토론
Discussions	공식 저장소 Discussions	제안/도움 요청(공식이 현재 권장하는 기여 채널)

참고: 공식 CONTRIBUTING은 "현재 외부 PR을 받지 않습니다"라고 명시합니다(2026-08-13 시점) — 하지만 다음은 장려합니다:

- Discussions에 제안 작성(공식이 검토함)
- dsh-plugin 생태계 프로젝트 만들기(공식이 명시적으로 인정하는 방식)
- 튜토리얼/블로그 작성

 **공식 자체 문서의 보충:** 위의 채널 외에도, 공식 저장소는 체계적인 내부 문서를 유지하고 있습니다 —

`packages/README.md`(패키지 목록 공식 표), `docs/subsystems/`(서브시스템 설계), `docs/cookbook/`(9개 개발 가이드), `docs/tool-catalog.md` / `docs/config-catalog.md` / `docs/module-graph.md`(자동 생성 목록). 백서는 "초보자 관점의 보충"이고, 공식 문서는 "아키텍처의 권위"이니 둘을 함께 읽으세요.

7.2 커뮤니티 기여 방식

공식은 아직 외부 PR을 받지 않지만, 커뮤니티는 이미 자발적으로 성숙한 "보고-재현-근본원인-수정 브랜치" 기여 템플릿을 만들었습니다 — **#341** (<https://github.com/deepseek-ai/deepseek-harness/discussions/341>), **#371** (<https://github.com/deepseek-ai/deepseek-harness/discussions/371>), **#775** (<https://github.com/deepseek-ai/deepseek-harness/discussions/775>)에서 반복적으로 공식에 Issues/PR을 열어달라고 호소했지만 공식은 아직 응답하지 않았고, 그래서 기여자들이 수정 사항을 게시물에 직접 담기 시작했습니다.

커뮤니티 기여 템플릿 4단계:

1. **보고:** Discussion으로 게시, 제목에 **【Bug】** 표시, OS / Node / dsh 버전 첨부
2. **재현:** 최소 재현 단계 + 오류 로그 앞 20줄
3. **근본원인:** 구체적인 파일과 함수까지 특정(예: readUtf16이 하위 바이트 0x00만 판단)
4. **수정 브랜치:** fork 후 수정 사항 제출, 게시물에 `git cherry-pick <commit>` 명령 첨부

cherry-pick 가능한 수정을 포함한 게시물 예시:

스레드	문제	수정 형태
#244 (https://github.com/deepseek-ai/deepseek-harness/discussions/244)	Windows 디렉터리 선택기가 "需求" 등 한자가 포함된 경로를 자름	수정 패치 첨부
#295 (https://github.com/deepseek-ai/deepseek-harness/discussions/295)	워크스페이스 생성 시 중국어 디렉터리 경로가 잘림	재현 + 수정 코드 조각
#580 (https://github.com/deepseek-ai/deepseek-harness/discussions/580)	Win32 네이티브 디렉터리 선택기가 U+XX00에서 UTF-16 경로를 자름	cherry-pick 가능한 수정 첨부

이런 게시물들의 공통점: "공식이 고쳐주세요"가 아니라 "제가 고쳤으니 쓰세요"입니다. 공식 Issues가 닫힌 단계에서, 이것이 dsh 커뮤니티의 가장 효율적인 협업 방식입니다 — 공식의 부담을 나누는 동시에 자신의 커뮤니티 신뢰도도 쌓습니다.

7.3 플러그인 생태계(2026-08-13 스냅샷)

프로젝트	포지셔닝	상태
DSH- better-sidebar	파일 관리/터미널/Git/브라우저 사이드바	가장 완성도 높은 커뮤니티 플러그인
예제 속도 개선 플러그인	도구 호출 속도 개선(reasoning_effort 자동 조절)	교육용 예제(제4장)
dsh- handbook(본 백서)	초보자 튜토리얼	생태계 문서
DeepSeek Desktop	Windows 데스크톱(x64 커뮤니티 설치 패키지, v0.2.0 오프라인 설치 프로그램)	#872 (https://github.com/deepseek-ai/deepseek-harness/discussions/872), 관련 #529 (https://github.com/deepseek-ai/deepseek-harness/discussions/529) #446 (https://github.com/deepseek-ai/deepseek-harness/discussions/446)
turtle-ui	터미널 TUI 인터페이스 플러그인 (turtle1999/turtle-ui) (https://github.com/turtle1999/turtle-ui)	커뮤니티 TUI 솔루션, 설치 시도는 #871 (https://github.com/deepseek-ai/deepseek-harness/discussions/871) 참고
memory 플러그인 계열	세션 간 장기 기억(설계 제안 / 장기 기	#192 (https://github.com/deepseek-ai/deepseek-harness/discussions/192)

	억 / MEMORY.md·USER.md 이식)	harness/discussions/192 #484 (https://github.com/deepseek-ai/deepseek-harness/discussions/484) #525 (https://github.com/deepseek-ai/deepseek-harness/discussions/525)
《기억체》제안	세션 간, 격리된, 사용자가 명시적으로 장착하는 기억 단위(하부 설계, memory 플러그인 계열과 연결 가능)	#1822 (https://github.com/deepseek-ai/deepseek-harness/discussions/1822) szx-a
dsh-sgme	기억 엔진: 대화 이력과 장기 기억을 계층화, 추출 전 자동 정리(세션 내용 65-96% 절감), 시나리오별 기억 블록 주입	freehul/sgme (https://github.com/freehul/sgme)(npm 패키지 dsh-sgme), #1052 (https://github.com/deepseek-ai/deepseek-harness/discussions/1052)
pi-quiet-tools	도구 출력 압축: 컨텍스트에 들어가기 전 큰 결과를 앞뒤 미리보기 + 로컬 artifact로 압축(12,000자 또는 240줄 초과 시 트리거)	pi2dsh (https://github.com/weijiafu14/pi2dsh)를 통해 장착, #1052 (https://github.com/deepseek-ai/deepseek-harness/discussions/1052)
dsh-win32	Windows에서 공식이 빠뜨린 win32 ProcessInspector를 보완해 미니멀 모드의 지속적인 shell이 실행되도록 함; 샌드박스 안에서 쓸 수 있는 busybox 변형, ConPTY 포그라운드 명령 인식, GBK/UTF-16 읽기도 포함	sjh9714/dsh-win32 (https://github.com/sjh9714/dsh-win32) (npm dsh-win32), 채택된 제안 #1889 (https://github.com/deepseek-ai/deepseek-harness/discussions/1889)
dsh-installers	Node 설치가 필요 없는 설치 패키지: mac DMG / Windows exe, Node 런타임 내장, 사전 의존성 전무	codeAnqiang-ma/dsh-installers (https://github.com/codeAnqiang-ma/dsh-installers)(#380 (https://github.com/deepseek-ai/deepseek-harness/discussions/380) 작성자 codeAnqiang-ma 제공), 설치는 제2장 2.2절 방법 3 참고
kubemd	Kubernetes 런타임 장애 진단 스킴(사례 기억 + CLI 이중 엔트리, 5개 시나리오 kind 실측 검증)	guiyi-labs/kubemd (https://github.com/guiyi-labs/kubemd), git clone 으로 바로 사용(DSH skill)

플러그인 찾기: GitHub에서 `topic:dsh-plugin` 검색.

플러그인 배포하기: 여러분의 저장소에 `dsh-plugin` topic을 추가하고 npm에 게시.

7.4 생태계에 참여하는 방법(초보자 경로)

1. **사용해보기:** `dsh web` + 커뮤니티 플러그인 두 개 설치, 일상 사용에 익숙해지기
2. **작은 개선:** 커뮤니티 플러그인에 PR 제출(제5장의 세 사례를 읽어보세요, 완전한 PR 패턴입니다)
3. **플러그인 공개:** 제4장의 최소 host 플러그인부터 시작해 `dsh-plugin` topic 달기

4. 콘텐츠 작성: 튜토리얼/리뷰/트러블슈팅 글(공식이 장려함), 본 백서와 상호 인용

7.5 추천 읽기 순서

목표	경로
빠르게 시작하기	제2장 → better-sidebar 설치 → 일상 사용
플러그인 개발	제3장 → 제4장 → 제5장 사례 따라하기
성능 튜닝	제6장 → 제4장 예제 소스 코드
심화 커스터마이징	공식 AGENTS.md(아키텍처) → docs/architecture.md → packages/ 소스 코드

7.6 실제 플러그인 설치 실습

아래는 대표적인 두 플러그인의 **완전한 장착 단계**입니다(문법은 제3장 3.2절과 동일).

DSH-better-sidebar 설치

① **package.json**에 의존성 추가(~/.dsh/profiles/web/package.json):

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② **cordis.patch.yml**에 장착 줄 추가:

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 설치와 검증:

```
cd ~/.dsh/profiles/web && npm install && dsh web
```

재시작 후 왼쪽에 파일 관리/터미널/Git/브라우저 네 개의 탭이 나타나는지 관찰하세요.

예제 속도 개선 플러그인 설치

① **package.json**에 의존성 추가:

```
{
  "dependencies": {
    "dsh-speed-plugin": "link:C:\\path\\to\\dsh-speed-plugin"
  }
}
```

② cordis.patch.yml에 장착 줄 추가:

```
- insert:
  - id: speed-plugin
    name: dsh-speed-plugin
```

③ 설치와 검증:

```
cd ~/.dsh/profiles/web && pnpm install && dsh web
```

④ 효과 검증: "파일 3개 생성" 작업을 하나 보내고, 로그에 [speed-plugin] calls=[...] => reasoningEffort=low가 나타나는지 관찰하세요(제4장 4.5절 참고).

⚠️ 두 플러그인 모두@deepseek-ai/dsh-agent ^0.1.0-rc.6을 요구합니다. pnpm install 0/404를 반환하면 버전 라인을 확인하세요(제3장 3.5절 참고).

7.7 플러그인을 찾는 방법

채널	구체적인 방법	예상 결과
GitHub topic	https://github.com/topics/dsh-plugin 또는 topic:dsh-plugin 검색	topic이 달린 모든 저장소
GitHub 전역 검색	dsh-plugin + 언어 필터 TypeScript	키워드가 포함된 저장소와 코드
npm 검색	npm search dsh-plugin 또는 https://www.npmjs.com/search?q=dsh-plugin	이미 게시된 패키지(rc 단계에서는 GitHub이 중심)
공식 디스커션 Show and tell	https://github.com/deepseek-ai/deepseek-harness/discussions/categories/show-and-tell	커뮤니티 자체 추천 프로젝트
본 백서의 인용 링크	제4/5/10장 사례 속 저장소 링크	실기 검증된 플러그인

팁: topic:dsh-plugin이 가장 정확한 필터링 방식입니다. 2026-08-13 시점 결과는 한 자릿수에 불과합니다
— 지금이 바로 진입 기회입니다.

7.8 생태계 참여의 전체 흐름

7.4절의 "네 단계"를 실행 가능한 의사결정 트리로 펼쳐봅니다:

사용해보기 → Issue로 피드백 → 플러그인 공개 (npm publish) → 콘텐츠 작성

npm publish 절차 요약:

1. package.json에 "name": "dsh-xxx" + "main": "src/index.ts" + 올바른 peerDependencies가 있는지 확인
2. npm login → npm publish --access public
3. GitHub 저장소에 dsh-plugin topic 추가
4. 공식 Discussions의 Show and tell 카테고리에 게시

CONTRIBUTING.md 참고: 사례에는 "시나리오 / 명령어 / 소요 시간 또는 산출물 / 검증 방식"이 포함되어야 합니다 — 플러그인을 공개할 때 이 정보를 첨부하면 전환율이 더 높아집니다.

7.9 생태계 마일스톤 타임라인

시기	사건	의미
2026-08-13	dsh 오픈소스 공개(초일)	MIT 라이선스로 발표, Agent 프로그램 머블 시대의 출발점
2026-08-13	공식 패키지 60+ 동시 공개	packages/ 아래 도구/컨텍스트/세 션/서브에이전트/MCP/워크플로우/보 안/모델/인터페이스 전체 커버
2026-08-13 당일	커뮤니티 플러그인 폭발	공식 디스커션에서 하루에 30+ 게시물, 플러그인 트러블슈팅, TUI 예제, Windows 경로 버그 등 포함
2026-08-13 그 주	본 백서 공개	14개 챕터 중국어 튜토리얼 + Benchmark + 플러그인 템플릿, 초일 문서 공백을 채움
진행 중	공식 디스커션 지속 활발	매주 2-3개 게시물 응답, 플러그인/경 로/TUI/vision 등 방향을 이미 커버 (7.10절 참고)
계획 중	플러그인 템플릿 확장 / 비디오 튜토리 얼 / CI 검증	ROADMAP.md P1 우선순위

핵심 인지: 2026-08-13은 "발표일"이 아니라 "생태계 초일"입니다 — 공식 패키지, 커뮤니티 플러그인, 중국어 튜토리얼이 같은 날 준비되었습니다. 참여자 입장에서는, **지금 진입하는 것 = 공식 인프라와 함께 성장하는 것**입니다.

7.10 공식 디스커션과의 연계

본 백서는 이미 공식 Discussions에서 8개 게시물에 응답했습니다. 아래는 **효과적으로 참여한 사례**입니다:

게시물	주제	응답 방식	재사용 가능한 패턴
#380 (https://github.com/deepseek-ai/deepseek-harness/discussions/380)	플러그인 트러블슈팅	결론을 제3장 3.5절 "흔한 트러블슈팅"에 정리	문제 → 재현 → 튜토리얼 작성 → 댓글로 인용
#392 (https://github.com/deepseek-ai/deepseek-harness/discussions/392)	TUI examples	docs/config-reference.md에 보충	공식에 예제 부족 → 실행 확인 → 레퍼런스 작성 → 링크 공유
#401 (https://github.com/deepseek-ai/deepseek-harness/discussions/401)	Windows 경로 버그	제12장 크로스플랫폼 약점에 기록	플랫폼 버그 → 경계 기록 → 공식 부담 분산
#384 (https://github.com/deepseek-ai/deepseek-harness/discussions/384)	visionDS	능력 매트릭스의 vision 지원 상태 업데이트	새 능력 → 조사 → 표 업데이트 → 확인
#118 (https://github.com/deepseek-ai/deepseek-harness/discussions/118)	일반 토론	FAQ에 반영(docs/faq.md 6개 카테고리 문답)	고빈도 질문 → 카테고리별 정리 → 링크 제공
#655 (https://github.com/deepseek-ai/deepseek-harness/discussions/655)	커뮤니티 5개 프로젝트	생태계 전체 조망을 제7장 7.3절에 정리	흩어진 프로젝트 통합 → 생태계 지도 형성
#735 (https://github.com/deepseek-ai/deepseek-harness/discussions/735)	token 비용	제6장 6.6절 비용 모델에 반영	비용 측정 → 공식화해서 정리
#781 (https://github.com/deepseek-ai/deepseek-harness/discussions/781)	LSP 제안	제11장 미래 전망에 기록	선행 기능 → 아키텍처 진화 추적

효과적인 참여의 3가지 원칙: ① 먼저 직접 실행해본 후 댓글 달기(환경 정보와 오류 로그 포함); ② 결론을 인용 가능한 콘텐츠로 작성(댓글 하나는 묻히지만, 챕터/FAQ로 쓰면 지속적으로 가치를 만듦); ③ 링크로 반복을 대체 (댓글에는 링크를 남기세요, 예: "제7장 7.6절 참고").

맺음말

dsh는 2026-08-13에야 오픈소스로 공개된 프로젝트입니다 — 생태계의 모든 날이 "초기"입니다. 백서는 dsh의 발전에 맞춰 지속적으로 업데이트될 것입니다. 어떤 장의 명령어가 작동하지 않는다면, 대부분 rc 버전 반복으로 인한 것이니 — 공식 changelog를 기준으로 삼으세요.

이 새로운 생태계에서 여러분의 자리를 먼저 차지하시길 바랍니다. 🚀

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 dsh 생태계의 3가지 공식 채널(저장소/API 문서/커뮤니티)이 각각 무엇에 쓰이는지 말해보세요

정답 확인: 본 장 7.1절 공식 채널 표 참고

2. **이해 문제:** 공식은 현재 "외부 PR을 받지 않지만", 어떤 3가지 참여 방식을 장려하나요? 왜 "dsh-plugin 생태계 프로젝트 만들기"가 공식이 명시적으로 인정하는 방식일까요?

정답 확인: 본 장 7.1절 "하지만 다음은 장려합니다" 단락 참고

3. **실습 문제:** GitHub에서 `topic:dsh-plugin`을 검색해, 찾은 커뮤니티 플러그인 3개를 나열하고 각각의 포지셔닝을 말해보세요

정답 확인: 본 장 7.3절 "플러그인 찾기" 단락 + 7.7절 검색 채널 표 참고

4. **실습 문제:** 본 장 7.6절 단계에 따라 DSH-better-sidebar를 완전히 장착하거나, 제4장 예제를 따라 직접 속도 개선 플러그인을 작성해, `pnpm install` 출력과 검증 결과를 기록하세요

정답 확인: 본 장 7.6절 두 곳 수정의 전체 코드 조각 참고

5. **실습 문제:** DSH-better-sidebar나 직접 작성한 속도 개선 플러그인의 README에 개선 PR을 하나 제출해보세요(예: 설치 단계 추가, 오타 수정), 커뮤니티 PR 흐름을 체험해보세요

정답 확인: 본 장 7.4절 2단계 + 제5장 사례의 PR 패턴 참고

6. **사고 문제:** 본 장 7.9절 타임라인에서 "공식 패키지 60+ 동시 공개"와 "커뮤니티 플러그인 폭발"이 같은 날 일어났습니다. 이것이 오픈소스 프로젝트의 생태계 전략에 어떤 시사점을 주나요?

정답 확인: 본 장 7.9절 타임라인 표 + 7.10절 연계 패턴 참고

자주 묻는 질문 FAQ

Q1: 공식이 "외부 PR을 받지 않는다"고 하는데, 제 플러그인 코드는 어디에 두어야 하나요?

여러분 자신의 GitHub 저장소에 독립적인 dsh-plugin 생태계 프로젝트로 두세요. 공식은 이 방식을 장려합니다 — 플러그인으로 능력을 확장하며, dsh 코어를 수정할 필요가 없습니다. 저장소에 dsh-plugin topic을 추가하고 npm에 게시하면 됩니다.

Q2: Discord와 Discussions는 무엇이 다른가요?

Discord는 실시간 토론에 적합하고; Discussions는 공식 제안, 기능 건의, 도움 요청에 적합합니다(기록으로 조회 가능). 공식이 현재 권장하는 기여 채널은 Discussions입니다.

Q3: 한국어(중국어) 튜토리얼/블로그를 쓰고 싶은데, 형식 요구사항이 있나요?

공식 형식 요구사항은 없습니다. 설치 단계, 코드 예시, 실기 스크린샷/로그를 포함할 것을 권장합니다. 본 백서의 구조(TL;DR + 단계별 설명 + 실습)를 참고할 수 있습니다.

Q4: 생태계 초일이 무슨 뜻인가요? 왜 "선점 우위"라고 하나요?

생태계 초일 = 생태계가 막 시작해서 콘텐츠/플러그인/튜토리얼이 거의 없는 상태. 선점 우위 = 초기 진입자가 "특정 방향의 표준"이 되기 쉬움. 비유: 2015년 React 튜토리얼을 쓴 사람, 2018년 VS Code 플러그인을 만든 사람.

Q5: rc 단계는 반복이 빠르는데, 제 플러그인이 다 만들자마자 구식이 되지 않을까요?

그럴 수 있습니다. 리스크를 낮추는 법: ① 의존성은 `^0.1.0-rc.6` 라인 사용; ② 공식 changelog를 주시; ③ 로직을 순수 함수로 격리하고 연결 레이어를 얇게 유지해, 업그레이드 시 연결 레이어만 수정하면 되도록.

Q6: dsh 생태계 프로젝트를 만들고 싶은데, 어디서부터 시작해야 하나요?

"여러분 자신의 페인 포인트"에서 시작하세요. 예를 들어: ① TUI가 필요하다면 → tui profile 플러그인 만들기; ② 토큰 추적이 필요하다면 → cost-tracker 플러그인 만들기; ③ diff 뷰가 필요하다면 → diff 플러그인 만들기. 본 장 7.5절 읽기 순서를 참고하세요.

Q7: 7.6절의 link: 경로는 Windows와 macOS에서 작성법이 같나요?

다릅니다. Windows는 이중 백슬래시로 이스케이프하고(link:C:\\path\\to\\plugin), macOS/Linux는 정슬래시를 씁니다(link:/path/to/plugin). 커뮤니티 플러그인은 최종적으로 npm 게시로 가서 경로 차이를 없애는 것을 권장합니다.

Q8: 공식 디스커션에 게시할 때 팁이 있나요? 공식이 더 빨리 응답하게 할 수 있나요?

7.10절의 3가지 원칙을 참고하세요: 환경 정보 포함, 재현 단계 포함, 먼저 검색해 중복을 피하기. 기능 건의는 "어떻게 구현할지"보다 "무슨 문제를 해결하는지"를 설명하는 것이 더 중요합니다.

제 8 장: 도구와 컨텍스트 시스템

본 장의 목표: dsh의 "능력 엔진"을 이해합니다 — 모델이 어떤 도구를 호출할 수 있는지, 컨텍스트는 어떻게 모델에게 전달되는지, 그리고 긴 대화는 어떻게 처리되는지. 이것이 "돌아간다"에서 "왜 돌아가는지 안다"로 넘어가는 핵심 챕터입니다.

TL;DR(본 장 핵심, 30초 버전)

1. **60+개 공식 능력 패키지:** 도구(fs/shell/web/skill/todo), 컨텍스트(context/compaction), 세션, 서버에이전트, MCP, 워크플로우, 보안, 모델, 인터페이스 — 모든 것이 플러그인입니다
2. **내장 도구 이름은 짧은 동사:** read/write/grep/glob/edit/bash/todo/skill — 프롬프트를 쓸 때 "파일 읽어", "검색해"라고 하면 됩니다
3. **도구가 locations를 반환 → 산출물 추적:** 모델이 어떤 파일을 바꿨는지 UI가 바로 보여주고 열 수 있습니다 (대화 끝의 산출물 칩)
4. **컨텍스트 = 시스템 프롬프트 + 스킬 카탈로그 + 대화 이력 + 도구 결과:** 계층별로 주입되며, 매 단계 요청마다 도구 스키마가 함께 전달됩니다
5. **긴 대화 자동 압축(compaction):** 오버플로우 감지 → 이력 다듬기 → 선택적 요약 → 실패 시 오버플로우 에이전트로 라우팅. 중요한 정보는 프롬프트에 직접 적어두는 것을 권장합니다
6. **플러그인 동작을 무비용으로 검증 가능:** mock llm + headless + 감사 dump(#462) — API key 없이도 waterfall 전달과 도구 주입을 검증할 수 있습니다
7. **도구 체인의 알려진 트러블슈팅 3가지:** code 모드 run_code/bash의 description required 무한 루프 (#558/#581/#689); 스트리밍에서 도구 이름이 지워짐(#725 계열, FAQ 참고); run_code 내 상호작용 도구의 비동기 결과가 유실됨(#1476)

<details><summary>본 장 목차</summary>

- 8.1 공식 능력 패키지 지도(60+개 개요)
- 8.2 내장 도구(실측 관찰)
- 8.3 컨텍스트는 어떻게 모델에게 전달되는가

- 8.3.1 PTC 모드(Code mode): 코드 한 조각으로 여러 라운드의 도구 호출을 오케스트레이션
- 8.4 긴 대화: compaction(압축)
- 8.5 권한과 보안 모델(알아만 두면 됨)
- 8.6 초보자가 가장 기억해야 할 세 가지
- 8.7 플러그인 런타임 검증 방법론(무비용)
- 8.8 도구 체인 트러블슈팅

8.1 공식 능력 패키지 지도(60+개 개요)

dsh의 능력은 전부 패키지 형태로 제공됩니다(`packages/<group>/<name>`). 초보자가 가장 알아야 할 것들:

능력 영역	공식 패키지	역할
도구(tools)	fs/tool-fs, fs/tool-fs-search, fs/tool-str-replace-editor, shell/tool-bash, web, skill, todo	파일/터미널/웹페이지/스킬/할일 등 호출 가능한 도구
컨텍스트(context)	context/*, compaction/*	요청 컨텍스트 조립, 긴 대화 압축
세션(session)	session/*	영속화, 제목, 텔레메트리
서브에이전트(subagent)	subagent/*	하위 작업 위임
MCP	mcp/*	MCP 클라이언트(외부 도구 서버)
워크플로우(workflow)	workflow/*	다단계 워크플로우 오케스트레이션
보안(safety)	sandbox/*, guard/*, interaction/*, credentials/*	샌드박스, 루프 위생, 권한/승인, 자격 증명 격리(참고: 공식에는 독립된 safety/ 그룹이 없으며, 보안 정책은 여러 패키지 그룹에 분산되어 있음)
모델(llm)	llm/*, llm-deepseek, llm-retry	모델 연동, 재시도
스킬(skill)	skill/*	스킬 프로바이더 레지스트리
인터페이스(client)	client/*(ui-conversation, ui-tool...)	Web UI 각 구성요소

전체 목록과 패키지 수 기준 공식 저장소 `packages/README.md`가 패키지 목록의 권위 있는 표입니다(47개 그룹); `@deepseek-ai/dsh CLI의 0.1.0-rc.6`은 53개의 `@deepseek-ai/dsh-*` 직접 의존성을 선언하며(+CLI 자체 = 54), 계열 전체 게시 수는 약 221개입니다(devDeps와 과거 0.0.1-rc.1 라인 포함, 공식 빌드 커밋에서 언급). 백서/공식이 말하 는 "60+"는 `packages/` 아래의 모든 하위 디렉터리를 가리킵니다. npm 패키지 이름 표기법(`@deepseek-ai/dsh-*`) 과 저장소 경로 표기법(`fs/tool-fs`)의 대응 관계, 그리고 각 패키지의 실측 설명은 부록 B: 공식 패키지 빠른 참조 참고.

📖 **아키텍처를 더 깊이 알고 싶다면:** 공식 docs/subsystems/(session / system-prompt / tools / agent / agent-loop / scope / llm-streaming / subagent)는 각 서브시스템의 설계 문서로, 이 절의 "능력 영역 그룹화"보다 훨씬 체계적입니다. 자동 생성된 공식 목록은 공식 docs/tool-catalog.md, docs/config-catalog.md, docs/module-graph.md 참고.

8.2 내장 도구(실측 관찰)

모델이 실제로 호출할 수 있는 도구 이름은 **짧은 동사**입니다(실측 기록, dsh web 세션과 agent/request 로그 기반):

도구 이름	역할	비고
read	파일 읽기	fs 능력
write	파일 쓰기	fs 능력
grep	내용 검색	fs-search
glob	파일 패턴 매칭	fs-search
edit/str_replace_editor	정밀 편집	도구 결과에 locations 포함(산출물 추적용)
bash/pwsh	명령 실행	샌드박스 격리
todo	할 일 관리	긴 작업 계획
skill	스킬 호출	skill-catalog 주입

도구 결과와 산출물 추적(중요한 개념): 도구의 반환값에는 locations(파일 경로)가 포함되어 있으며, dsh는 이를 이용해 "산출물 파일 행"을 만듭니다(대화 끝의 산출물 칩이 여기서 나옵니다) — **모델이 어떤 파일을 바꿨는지 UI가 바로 보여주고 열 수 있습니다.**

8.3 컨텍스트는 어떻게 모델에게 전달되는가

모델 요청 한 번의 컨텍스트 = 시스템 프롬프트 + 스킬 카탈로그 + 대화 이력 + 도구 결과. 세션 로그에서 실측으로 확인할 수 있습니다:

컨텍스트 주입 @deepseek-ai/dsh-system-prompt
컨텍스트 주입 skill-catalog

← 공식 시스템 프롬프트
← 스킬 카탈로그

dsh의 컨텍스트 메커니즘:

- **시스템 프롬프트 계층화**: 공식 플러그인이 `systemPrompt.section()`으로 프롬프트 조각을 등록(예: `ui-deliverables`가 "산출물 파일 참조" 안내를 등록)
- **스킬 카탈로그 주입**: 사용 가능한 스킬 목록이 컨텍스트에 들어가, 모델이 필요에 따라 호출
- **도구 스키마**: 매 단계 요청마다 도구 정의가 함께 전달됨

8.3.1 PTC 모드(Code mode): 코드 한 조각으로 여러 라운드의 도구 호출을 오케스트레이션

PTC = Programmatic Tool Calling(프로그래매틱 도구 호출) — 공식 중국어 사이트는 "PTC 모드"라고 부르고, 공식 영문 페이지는 이에 대응해 **Code mode**라고 합니다(#1052

(<https://github.com/deepseek-ai/deepseek-harness/discussions/1052>) 댓글의 커뮤니티 상세 설명; 공식 사이트 원문: "PTC 모드는 모델이 생성한 코드 조각을 통해 여러 라운드의 도구 호출을 조합합니다").

전통적인 방식과의 차이: 표준 모드에서는 모델이 매 단계마다 도구 호출을 한 번씩 발행합니다(한 단계마다 한 번의 왕복); PTC 모드에서는 모델이 **TypeScript 프로그램 한 조각을 생성**해, 단일 프로그램 안에서 여러 라운드의 도구 호출을 오케스트레이션합니다(루프, 분기, `Promise.all` 병렬), 전체 프로그램이 한 번의 왕복으로 실행 완료됩니다:

항목	표준 모드	PTC 모드(Code mode)
호출 형태	단계마다 도구 호출 한 번	프로그램 하나로 여러 라운드 호출 오케스트레이션
모델 왕복	5회 호출 ≈ 5회 왕복	1회 왕복으로 통합
중간 결과	단계별로 컨텍스트에 다시 채워짐	<code>return/console.log</code> 만 다시 채워짐
전형적인 시나리오	일상 대화, 단일 단계 작업	긴 체인 자동화, 결정론적 실행

실행 메커니즘: 프로그램은 `@deepseek-ai/dsh-code-runtime-worker-thread`로 전달되어 **완전히 새로운 Node worker 스레드**에서 실행됩니다(타입 스트리핑 후 TS를 지울 수 있음; 빈 환경 + 힙/소요시간/출력 예산 + 강제 종료); 프로그램 내의 각 하위 도구 호출은 여전히 완전한 도구 파이프라인을 거칩니다(권한, 샌드박스, 감사가 그대로 적용됨).

이점: 여러 단계 호출이 한 번의 왕복으로 통합되고 + 중간 결과가 더 이상 전부 컨텍스트에 다시 채워지지 않음 → **token 비용이 크게 감소**합니다(#1052 댓글의 실측 경험: PTC 모드는 긴 작업의 비용 절감 수단 중 하나).

참고: PTC 모드에도 커뮤니티가 보고한 알려진 트러블슈팅이 있습니다 — `run_code/bash`의 `description required` 무한 루프(8.8절 트러블 1 참고); 그리고 긴 세션에서 여러 라운드의 도구 호출 중간 산출물이 여전히 컨텍스트에 남습니다 (#1052 원문 실측: 7천만→1억 토큰 폭증이 바로 PTC 모드에서 발생, 8.4절 자동 압축 필요성 참고).

8.4 긴 대화: compaction(압축)

긴 대화는 컨텍스트를 터뜨릴 수 있습니다. dsh의 `compaction` 플러그인(예: `compaction-basic`)이 담당하는 것:

- 컨텍스트 오버플로우 감지(`agent/request-error`의 `CONTEXT_WINDOW_EXCEEDED`)
- 이력 압축(모델과 무관한 다듬기 + 선택적 요약)
- 실패 시 "오버플로우 에이전트"(overflow agent)로 라우팅

자동 압축은 커뮤니티에서 자주 요청되는 기능입니다(#1052

(<https://github.com/deepseek-ai/deepseek-harness/discussions/1052>): 사용자들은 긴 세션에서 토큰이 폭증한 후 "새 세션을 열어야 하는" 것이 고통스럽다고 피드백하며, 수동으로 새 세션을 여는 대신 자동 압축을 원합니다 — 커뮤니티 답변에서 "현재 내장된 자동 압축은 없는 것 같다"고 확인했습니다(compaction 플러그인 설치/수동 compact 필요). 저비용으로 긴 세션을 이어가는 임시 방법: 새 세션으로 바꾸기 + 필요에 따라 기억 주입(기억 계층화 방안은 제7장 7.3절과 제14장 14.7절 생태계 도구 참고).

초보자에게: "긴 대화는 자동으로 압축된다"는 것만 알면 됩니다, 세부사항은 심화 주제입니다. 프로덕션 환경 주의사항: 압축은 세부 정보를 잃어버리므로, 중요한 컨텍스트는 프롬프트에 수동으로 적어두는 것을 권장합니다.

8.5 권한과 보안 모델(알아만 두면 됨)

- 접근 모드: UI에서 "Workspace Write" 등의 모드(권한 프리셋)를 볼 수 있음
- 상호작용 승인: `interaction/*`이 권한/승인 능력을 제공(위험한 작업은 확인을 요구할 수 있음)
- 샌드박스: `sandbox/*`가 명령 실행을 격리(예: pwsh 샌드박스는 ACL 제약이 있음 — 실측 중 temp 디렉터리 권한 문제를 겪은 적 있음)
- 도구 타임아웃: `guard/*`가 loop 위생과 도구 타임아웃을 제공

보안 설정의 심화 주제는 본 백서의 범위를 벗어납니다, 핵심 인자: dsh의 도구 실행에는 기본적으로 격리와 승인 레이어가 있으며, 맨몸으로 실행되지 않습니다.

8.6 초보자가 가장 기억해야 할 세 가지

1. 도구 이름은 짧은 동사입니다(read/write/grep/glob/edit/bash) — 프롬프트/플러그인을 작성할 때 "파일 읽어", "검색해"라고 직접 말하면 됩니다
2. 도구가 locations를 반환 → 산출물 추적 — 모델이 수정한 파일이 대화 산출물 영역에 나타납니다
3. 긴 대화는 자동 압축됩니다 — 이력을 수동으로 정리할 필요는 없습니다(단, 중요한 정보는 프롬프트에 적어주세요)

8.7 플러그인 런타임 검증 방법론(무비용)

출처: 공식 디스커션 #462 (<https://github.com/deepseek-ai/deepseek-harness/discussions/462>) 'DSH 플러그인 런타임 검증 방법론: API Key 없이 waterfall 동작 검증하기'(실측 74개 이벤트 / 12개 waterfall). FAQ에 이미 한 줄 요약이 있으며, 본 절에서는 자세히 다룹니다.

페인 포인트: 정적 점검은 "플러그인이 로드될 수 있는지"만 증명할 뿐, "플러그인이 실제 agent 루프에서 동작을 망가뜨리지 않는지"는 증명하지 못합니다. 특히 waterfall 이벤트 리스너(tools/execute, approval/request, fs/write-intent 등)는 반드시 next()를 올바르게 전달해야 하며, 그렇지 않으면 다운스트림의 기본 동작을 조용히 삼켜버립니다 — 이런 종류의 오류는 런타임에서만 드러납니다; 그런데 전통적인 검증은 실제 LLM API key와 토큰 비용을 필요로 합니다.

무비용 방안(dsh 저장소에 내장되어 있어 API key 불필요):

1. **mock llm:** 스크립트화된 반환값 — 첫 요청이 "bash 도구 호출"을 반환하고, 두 번째는 정상 응답하도록(--sequence tool_call_success,success), 이렇게 도구 호출을 주입
2. **headless profile:** 실제 agent 루프를 거치며, 여러분의 플러그인이 리스너를 걸어둠
3. **감사 dump:** DSH_EVENT_AUDIT_DUMP가 이벤트 감사 스냅샷을 내보냄

```
# 1. mock-llm 시작(스크립트로 "bash 도구 호출" 주입)
pnpm run mock:llm --port 8000 --api-key mock-key \
  --sequence tool_call_success,success --repeat-last \
  --tool-name bash --tool-arguments '{"command":"ls"}'

# 2. headless agent 실행(mock-llm을 가리키며, 감사 내보내기)
DEEPSEEK_BASE_URL=http://127.0.0.1:8000/v1 \
DEEPSEEK_API_KEY=mock-key \
DSH_EVENT_AUDIT_DUMP=/tmp/audit.json \
pnpm dsh --profile headless "run the bash tool once and report"
```

통과 판정: 감사 스냅샷의 byMode에 emit과 waterfall이 모두 포함되고, waterfall 체인이 완전하게 나타나며, agent가 정상적으로 완료됨(mock response recovered). 실측: 74개 이벤트 / 12개 waterfall이 전부 캡처되며, 완전한 생명주기 session/created → ... → tools/pre-execute(wf) → tools/execute(wf) → tools/post-execute(wf) → tools/result; 핵심 보안 증명은 모든 waterfall 리스너가 next()를 올바르게 전달하고, 부작용이 전혀 없다는 것입니다 — 플러그인이 기본 동작을 삼키지 않고, mock으로 주입한 도구 호출도 체인을 완전히 거칩니다.

소스 코드로 실행할 때의 세 가지 유의점(#462 작성자의 실측 경험): sparse clone은 한 번에 전부 가져와야 함 (vendor/가 빠지면 Cannot find package '@deepseek-ai/cordis' 오류); build:lib:host + build:lib:client + web-frontend dist 3층 빌드 중 하나라도 빠지면 안 됨; mock:llm 파라미터 앞에 --를 추가로 넣지 말 것(위치 인자로 처리됨).

8.8 도구 체인 트러블슈팅

rc.6 시대 도구 체인의 알려진 트러블슈팅(트러블 1/2는 이미 FAQ에 반영됨):

트러블 1: code 모드 run_code/bash의 description required 무한 루프(#558)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/558>) #581

(<https://github.com/deepseek-ai/deepseek-harness/discussions/581>) #689

(<https://github.com/deepseek-ai/deepseek-harness/discussions/689>)

code 모드에서 run_code와 bash 둘 다 UI 요약 필드를 description이라고 부르고 required로 표시합니다. 모델이 종종 내부의 bash.description을 이미 전달한 것으로 착각해, 외부 JSON에는 {"code": "..."}만 남게 되어 → 반복적으로 missing required property "description" 오류가 발생하며, 마치 "무작위로 파라미터가 빠지는" 것처럼 보입니다. #581이 근본 원인을 보충합니다:

ToolArgsError가 도구 이름을 포함하지 않아서, 모델이 어느 도구에서 오류가 났는지 파악할 수 없음 → 무한 재시도 루프(cherry-pick 가능한 수정 첨부); #689는 같은 계열의 문제로 인해 run_code 내의 모든 tools.* 호출이 거부되는 것을 보여줍니다. 회피법: 외부 description을 수동으로 보충하거나 표준 모드로 전환.

트러블 2: 스트리밍에서 도구 호출 이름이 지워짐(#725)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/725>) 계열, 이미 FAQ에 반영됨)

증상: 모든 도구 호출에서 `Error: unknown tool ""` 오류가 발생. 근본 원인: SSE 스트리밍 파싱이 누적이 아닌 덮어쓰기 할당을 사용해, 도구 이름/ID를 빈 문자열로 지워버림(#725 (<https://github.com/deepseek-ai/deepseek-harness/discussions/725>) 근본 원인 + 수정; #161 (<https://github.com/deepseek-ai/deepseek-harness/discussions/161>) #615 (<https://github.com/deepseek-ai/deepseek-harness/discussions/615>) #694 (<https://github.com/deepseek-ai/deepseek-harness/discussions/694>) #741 (<https://github.com/deepseek-ai/deepseek-harness/discussions/741>) 같은 계열). 공식 수정 전에는 다운그레이드하거나 버전을 기다릴 수밖에 없습니다; 모델이 반복적으로 재시도할 수 있으니 제때 중단하는 데 유의하세요.

트러블 3: `run_code` 내에서 상호작용 도구 호출이 "삼켜짐"(#1476 (<https://github.com/deepseek-ai/deepseek-harness/discussions/1476>))

`run_code` 프로그램 안에서 상호작용 도구(예: `ask_user_question`)를 호출할 때: 동기 실행이 비동기를 기다리지 않음 — 4ms 만에 빈 값을 반환; 비동기 결과가 약 12초 후에 돌아왔을 때는 세션 프레임이 이미 종료되어, 그대로 유실됩니다. `run_code` 실행 모델의 문제로 판단됩니다(비동기 도구 콜백을 지원하거나 pending 상태를 반환해야 함), AGENTS.md의 "코드 안에서 상호작용 도구를 호출하지 말라"는 규약은 근본적인 해결책이 아닙니다.

기억 방향 제안(#1822 커뮤니티 설계): 커뮤니티가 "기억체"(Memory Body)를 제안했습니다 — 세션 간, 격리된, 사용자가 명시적으로 장착하는 기억 단위로, "기억이 뒤죽박죽 섞이는" 문제(세션이 끝나면 로그가 즉시 무효화됨 / session-reference는 @멘션 3개 소스만 다루고 본문은 검색하지 않음)를 해결합니다. 기억 계열(시나리오별로 주입하는 dsh-sgme / dsh-memory / AgentSoul)과 같은 방향으로 상호 보완합니다. 기억체는 하부 장착 단위를 담당하고, 상위 플러그인이 이를 재사용할 수 있습니다.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 dsh에 내장된 8개 도구 이름(짧은 동사)과 각각의 역할을 나열해보세요

정답 확인: 본 장 8.2절 도구 표 참고

2. **이해 문제:** "도구가 locations를 반환 → 산출물 추적"이 무슨 뜻인지 설명하세요. 모델이 어떤 파일을 바꿨는지 UI는 어떻게 아나요?

정답 확인: 본 장 8.2절 "도구 결과와 산출물 추적" 단락 참고

3. **실습 문제:** dsh web에서 "hello.py 파일 하나 만들어줘" 작업을 보내고, 대화 끝의 "산출물 칩"을 관찰해 파일을 클릭해 열어보고, 산출물 추적이 작동하는지 검증하세요

정답 확인: 본 장 8.2절 "산출물 파일 행" 설명 참고

4. **실습 문제:** dsh web에서 긴 대화(20+ 라운드)를 진행하며, compaction이 트리거되는지 관찰하세요(세션 로그에 "컨텍스트 오버플로우"나 "압축" 관련 로그가 있는지 확인)

정답 확인: 본 장 8.4절 "긴 대화 자동 압축" 단락 참고

5. **사고 문제:** 왜 "중요한 정보는 프롬프트에 적어야" 하고 대화 이력에 의존하면 안 될까요? compaction 압축 시 무엇이 유실되나요?

정답 확인: 본 장 8.4절 "압축은 세부 정보를 잃어버린다"는 경고 참고

6. **사고 문제:** 본 장 8.5절은 "dsh의 도구 실행에는 기본적으로 격리와 승인 레이어가 있다"고 말합니다. "모델이 모든 bash 명령을 자동으로 실행하도록 허용하는" 플러그인을 만든다면 어떤 확장 포인트를 써야 할까요? 어떤 보안 리스크가 있을까요?

정답 확인: 본 장 8.5절 "상호작용 승인" + interaction/* 패키지 설명 참고

자주 묻는 질문 FAQ

본 장은 도구/컨텍스트에 관한 고빈도 문답을 아래에 다룹니다; 더 많은 일반적인 질문은 FAQ 빠른 참조 참고.

Q1: 도구 이름이 read/write 같은 짧은 동사인데, 프롬프트를 쓸 때도 이 이름을 써야 하나요?

아닙니다. "이 파일을 읽어줘", "foo가 포함된 줄을 검색해줘"라고 쓰면, 모델이 자동으로 해당 도구에 매핑합니다. 짧은 동사 이름은 플러그인 내부 구현 세부사항이며, 프롬프트는 자연어를 쓰면 됩니다.

Q2: edit과 str_replace_editor는 같은 도구인가요?

맞습니다. str_replace_editor는 내부 패키지 이름이며, 모델이 호출할 때는 edit으로 표시될 수 있습니다. 기능은 동일합니다: 파일의 특정 텍스트를 정밀하게 교체. 도구 결과의 locations는 산출물 추적에 사용됩니다.

Q3: compaction 압축 후에도 모델이 이전 대화 내용을 기억하나요?

"요약"은 기억하지만 세부사항은 잃어버립니다. compaction은 이력을 다듬고 + 선택적으로 요약을 생성하며, 핵심 정보는 남지만 세부사항은 유실됩니다. 그래서 중요한 컨텍스트(프로젝트 제약사항, 특수 요구사항 등)는 시스템 프롬프트나 프롬프트에 적어두고, 대화 이력에만 의존하지 않는 것을 권장합니다.

Q4: context/*와 compaction/*는 무엇이 다른가요?

context/*는 "매 요청의 컨텍스트를 조립하는" 역할입니다(시스템 프롬프트 + 스킬 카탈로그 + 대화 이력 + 도구 스키마). compaction/*는 "긴 대화를 압축하는" 역할입니다(오버플로우 감지 → 다듬기 → 요약). 하나는 "무엇을 담을지"이고, 다른 하나는 "담을 수 없을 때 어떻게 압축할지"입니다.

Q5: 보안 모델에서 "샌드박스"와 "상호작용 승인"은 무엇이 다른가요?

샌드박스(sandbox/*)는 명령 실행 환경을 격리합니다(예: pwsh 샌드박스는 ACL 제약이 있어 파일 접근을 제한). 상호작용 승인(interaction/*)은 "위험한 작업 전에 사용자 확인을 요구하는" 것입니다(파일 삭제, 알 수 없는 명령 실행 등). 하나는 "어디서 실행하는지"를 관리하고, 다른 하나는 "실행할 수 있는지 없는지"를 관리합니다.

Q6: dsh에 새 도구(예: "데이터베이스 조회")를 추가하고 싶은데, 어떻게 해야 하나요?

두 가지 방법이 있습니다: ① host 플러그인을 작성해 ctx.provide로 도구 서비스를 등록(제3장 3.4절 확장 포인트 참고); ② MCP로 외부 도구 서버를 연동(제9장 9.1절). 전자는 깊은 통합에 적합하고, 후자는 기존 시스템을 빠르게 연결하는 데 적합합니다.

제 9 장: MCP, 서브에이전트, 워크플로우

본 장의 목표: dsh의 세 가지 확장 능력을 이해합니다 — MCP(외부 도구 연동), 서브에이전트(병렬 작업), 워크플로우(다단계 오케스트레이션). 이것들은 dsh를 "단일 Agent"에서 "Agent 시스템"으로 업그레이드하는 스위치입니다.

⚠ 본 장은 공식 아키텍처 문서(`packages/AGENTS.md`)와 패키지 구조를 정리한 것입니다, 일부 기능 예시는 「미실측」으로 표시되어 있습니다 — rc 버전은 반복 속도가 빠르므로, 구체적인 사용법은 공식 changelog를 기준으로 삼으세요.

TL;DR(본 장 핵심, 30초 버전)

- MCP = Agent에 외부 도구를 꽂는 것:** MCP 프로토콜로 데이터베이스, 브라우저, 회사 내부 시스템 등을 연동 — 내장 도구를 먼저 익히고, 명확한 공백이 있을 때 MCP를 추가하세요
- 서브에이전트 = 병렬로 일하기:** 작업을 하위 Agent에 위임해 병렬로 실행(대규모 저장소 조사, 긴 작업 분해, 독립 검증) — 단일 Agent를 먼저 잘 쓴 다음 서브에이전트를 도입하세요
- 워크플로우 = 결정론적 프로세스 오케스트레이션:** "여러 라운드의 대화"와 달리, 워크플로우는 단계를 프로세스로 정의해 순서/조건에 따라 실행합니다 — 데이터 가져오기→정제→리포트→검증에 적합
- 조합하면 = Agent 시스템:** 부모 Agent가 계획 → 서브에이전트가 병렬 조사 → 도구 체인 + MCP → 워크플로우로 취합 → 산출물
- 초보자 4단계 경로:** 단일 Agent + 내장 도구 → + MCP → + 서브에이전트 → + 워크플로우(단계적으로 쌓고, 건너뛰지 마세요)

<details><summary>본 장 목차</summary>

- 9.1 MCP: 외부 도구 생태계 연동
- 9.2 서브에이전트(subagent): 병렬로 일하기
- 9.3 워크플로우(workflow): 다단계 오케스트레이션
- 9.4 조합하기: "Agent 시스템"은 어떻게 생겼는가
- 9.5 커뮤니티 생태계 예시(2026-08-13 스냅샷)
- 9.6 MCP 연동 설정 예시
- 9.7 서브에이전트 설정 예시

- 9.8 워크플로우 YAML 예시
- 9.9 세 가지 비교표
- 9.10 트러블슈팅 보충
- 9.11 4단계 초보자 경로(검수 기준 포함)

9.1 MCP: 외부 도구 생태계 연동

MCP(Model Context Protocol)는 "Agent에 외부 도구를 꽂는" 개방형 프로토콜입니다. dsh는 mcp 클라이언트 패키지(@deepseek-ai/dsh-mcp-client)를 제공합니다.

할 수 있는 것: MCP를 통해 어떤 MCP 서버든(데이터베이스, 브라우저, 차트, 회사 내부 시스템……) 연동할 수 있습니다 — 예를 들어 커뮤니티에 이미 등장한 dsh-plugin-cost-tracker(토큰 추적)가 MCP/플러그인 형태입니다.

초보자 포지셔닝: MCP는 "생태계 확장"이니, dsh에서 명확한 도구 공백이 생긴 후에 연동하세요. 내장 도구를 먼저 익히고, 그다음 MCP를 추가하세요.

9.2 서브에이전트(subagent): 병렬로 일하기

무엇인가: 작업을 하위 Agent에 위임해 병렬로 실행합니다(packages/subagent/*).

전형적인 시나리오:

- 대규모 저장소의 여러 모듈 조사(모듈마다 서브에이전트 하나씩)
- 긴 작업 분해(부모 에이전트가 계획, 서브에이전트가 실행)
- 독립 검증(서브에이전트가 교차 확인)

초보자에게: 서브에이전트는 "고급 무기"입니다 — 단일 Agent를 먼저 잘 쓰고, 작업 분해를 이해한 후에 서브에이전트를 도입하세요. 잘못된 예시는 "무슨 작업이든 서브에이전트를 여는 것"으로, 오히려 조율 비용만 늘어납니다.

알려진 제약(커뮤니티 피드백): 현재 **서브에이전트 모델을 동적으로 지정할 수 없습니다** — 서브에이전트 모델은 부모 Agent 설정을 따라가며, 커뮤니티는 이미 디스커션에서 피드백했습니다(#118

(<https://github.com/deepseek-ai/deepseek-harness/discussions/118>) 댓글, Claude Code의 "서브에이전트 모델을 지정할 수 있는" 경험과 대조). 서로 다른 모델 능력이 필요할 때는, 현재로서는 부모 Agent의 모델 설정을 전체적으로 전환하거나, 독립된 세션으로 나누어 각각 지정하는 수밖에 없습니다.

9.3 워크플로우(workflow): 다단계 오케스트레이션

무엇인가: `packages/workflow/*`가 다단계 워크플로우 오케스트레이션을 제공합니다(worker-thread provider + tool Consumer).

"여러 라운드의 대화"와의 차이: 일반 대화는 "모델이 자유롭게 진행"하는 것이고, 워크플로우는 "단계를 프로세스로 정의해 순서/조건에 따라 실행"하는 것입니다 — **결정론적 프로세스**에 적합합니다(예: 데이터 가져오기 → 정제 → 리포트 생성 → 검증).

공식 예시(`packages/examples/`): 실행 가능한 `cordis.yml` 워크플로우 예시가 있습니다.

9.4 조합하기: "Agent 시스템"은 어떻게 생겼는가

```
당신의 프롬프트 (목표)
↓
부모 Agent (계획)
├── 서브에이전트 A: 모듈 A 조사 (병렬)
├── 서브에이전트 B: 모듈 B 조사 (병렬)
└── 도구 체인: read/grep/bash + MCP (데이터베이스)
↓
워크플로우 (예: 검증 → 취합 → 리포트 출력)
↓
산출물 (열람/추적 가능)
```

초보자 경로 제안:

1. **1단계:** 단일 Agent + 내장 도구(제2-5장)
2. **2단계:** + MCP(외부 도구 연동)
3. **3단계:** + 서브에이전트(병렬)

4. 4단계: + 워크플로우(결정론적 프로세스)

9.5 커뮤니티 생태계 예시(2026-08-13 스냅샷)

공식 Discussion에 이미 등장한 커뮤니티 프로젝트:

- dsh-plugin-cost-tracker — 토큰 비용을 실시간 추적(플러그인/MCP 형태)
- 플러그인 개발/속도 개선류(본 백서 제4장의 예제 속도 개선 플러그인 등)

생태계는 빠르게 성장하고 있습니다 — **지금**이 진입해서 블록을 쌓기 좋은 때입니다.

9.6 MCP 연동 설정 예시

아래는 cordis.patch.yml에서 MCP 클라이언트를 장착하는 **설정 조각**입니다(문법은 제3장 3.2절과 동일, 커뮤니티 토큰 추적 플러그인을 예로 들):

```
- insert:
  - id: mcp-client
    name: '@deepseek-ai/dsh-mcp-client'
    config:
      servers:
        - name: cost-tracker
          command: npx
          args: ['-y', 'dsh-plugin-cost-tracker']
          env:
            DSH_API_KEY: '${DSH_API_KEY}'
```

필드 설명: id(cordis 체인 식별자), name(npm 패키지 이름), config.servers(서버 목록, 서버마다 독립된 프로세스), command/args(시작 명령과 인자), env(환경 변수, \${VAR}로 호스트 변수 참조 가능).

⚠ 위 설정은 packages/mcp/ 디렉터리 구조와 cordis 장착 문법을 기반으로 추론한 것이며, 구체적인 필드는 공식 문서를 기준으로 삼으세요(rc 단계에서 변경될 수 있음). 검증 방법(추론, 미실측): 장착 후 dsh web을 재시작하고, 토큰 카운팅과 관련된 작업을 보내 로그에 MCP 호출 기록이 나타나는지 관찰하세요.

9.7 서브에이전트 설정 예시

병렬 서브에이전트는 packages/subagent/*를 통해 제공됩니다. 아래는 cordis.patch.yml에서 **서브에이전트 능력을 활성화**하는 설정 조각입니다:

```
- insert:
  - id: subagent
    name: '@deepseek-ai/dsh-subagent'
    config:
      maxConcurrency: 3
      timeout: 120000
```

적용 시나리오와 동시성 전략:

시나리오	서브에이전트 수	부모 에이전트의 역할	서브에이전트의 역할
대규모 저장소 여러 모듈 조사	3-5개	모듈 경계 구분, 결론 취합	각 모듈을 독립적으로 조사
긴 작업 분해	2-3개	하위 작업 분할, 완전성 검증	각 하위 작업을 독립 실행
독립 검증	1-2개	결론과 검증 기준 제공	교차 확인
여러 파일 병렬 편집	2-4개	파일 범위 고정, 충돌 방지	서브에이전트마다 다른 파일에 쓰기

⚠️ `maxConcurrency`와 `timeout`은 패키지 이름과 `cordis` 관례를 기반으로 추론한 것입니다(추론, 미실측). 컨텍스트 격리 메커니즘은 공식 `packages/subagent/README.md`를 참고해야 합니다.

9.8 워크플로우 YAML 예시

아래는 결정론적 워크플로우의 `cordis.patch.yml` 조각으로, "데이터 가져오기 → 정제 → 리포트 → 검증"의 4단계를 시뮬레이션합니다(제10장 사례 A의 작업 구조 참고):

```
- insert:
  - id: workflow
    name: '@deepseek-ai/dsh-workflow'
    config:
      steps:
        - id: fetch
          tool: bash
          command: 'curl -o raw.json https://api.example.com/data'
        - id: clean
          tool: bash
          command: 'python clean.py raw.json cleaned.json'
          dependsOn: [fetch]
        - id: report
          tool: bash
          command: 'python visualize.py cleaned.json chart.png'
          dependsOn: [clean]
        - id: validate
          tool: bash
          command: 'python verify.py cleaned.json'
          dependsOn: [report]
```

사례 A와의 대응: 워크플로우는 dependsOn으로 순서를 명시적으로 정의하는 반면, 사례 A의 "여러 라운드 대화"는 모델이 자유롭게 도구를 결정합니다. 워크플로우는 "단계가 고정된" 경우에 적합하고, 자유로운 대화는 "탐색적인" 경우에 적합합니다.

⚠️ steps/dependsOn은 packages/workflow/ 패키지 이름과 일반적인 DSL을 기반으로 추론한 것입니다(추론, 미실측). 공식 packages/examples/에 실행 가능한 예시가 있으니 이를 기준으로 삼는 것을 권장합니다.

9.9 세 가지 비교표

항목	MCP	서브에이전트	워크플로우
적용 시나리오	외부 시스템 연동(데이터베이스/브라우저/내부 API)	큰 작업의 병렬 분해(조사/검증/여러 모듈)	결정론적 파이프라인(가져오기→정제→리포트→검증)
복잡도	중간(서버 설정 + 프로토콜 이해 필요)	높음(작업 분해 설계 + 결과 취합 필요)	낮음-중간(YAML 설정, 의존성 그래프 디버깅은 다소 번거로움)
실패 처리	서버 다운 → 재연결이나 오류 반환(추론, 미실측)	서브에이전트 하나 실패 → 부모 에이전트가 재시도/무시/종료 결정	단계 하나 실패 → 선택적 일시정지/건너뛰기/롤백(추론, 미실측)
성능 오버헤드	추가 프로세스 통신 오버헤드(stdio/sse)	병렬로 띄우게 시간 절약, 대신 조율 오버헤드 증가	순차 실행, 추가 모델 오버헤드 없지만 병렬화 이득도 없음

결정 가이드: 명확한 도구 공백이 있다면 → MCP; 작업이 서로 의존하지 않는 여러 조각으로 나눌 수 있다면 → 서브에이전트; 단계가 고정되어 있고 엄격한 순서가 필요하다면 → 워크플로우. 세 가지는 조합할 수 있습니다: 서브에이전트가 병렬로 조사 → 내부에서 MCP로 데이터베이스 조회 → 워크플로우로 리포트를 취합.

9.10 트러블슈팅 보충

#	트러블	증상	해법
1	MCP 연결 실패	장착 후 시작 오류나 MCP 도구 무응답	command/args 확인; 먼저 수동으로 npx -y xxx를 실행해 검증(추론, 미실측)
2	서브에이전트 컨텍스트 격리	서브에이전트가 부모 Agent의 이력을 보지 못해 중복 질문 발생	부모 에이전트가 위임할 때 핵심 컨텍스트를 작업 설명에 적어둠(추론, 미실측)
3	워크플로우 단일 단계 실패	특정 단계의 bash가 0이 아닌 값을 반환해도 후속 단계가 계속 실행됨	onError: pause를 설정하거나 각 단계 후 검증 스크립트를 추가(추론, 미실측)
4	서브에이전트 파일 충돌	여러 서브에이전트가 동시에	부모 에이전트가 계획할 때

		같은 파일에 씀	파일 경계를 구분(추론, 미실 측)
5	MCP 환경 변수 미주입	\${DSH_API_KEY}가 빈 값으로 해석됨	호스트 shell에서 이미 export했는지 확인; 또는 하드코딩(로컬 테스트에만) (추론, 미실측)

"추론, 미실측"으로 표시된 모든 내용은 packages/ 디렉터리 구조와 cordis 설정 관례를 기반으로 추론한 것이며, 본 백서의 실기 검증 결과가 아닙니다. rc 단계에서는 공식 문서를 기준으로 삼으세요.

9.11 4단계 초보자 경로(검수 기준 포함)

9.4절의 "4단계"를 각 단계마다 명확한 검수 기준이 있는 업그레이드 경로로 확장합니다:

단계	능력	검수 기준(다음 단계로 넘어 가도 되는지 아는 법)	참고 절
1단계	단일 Agent + 내장 도구	독립적으로 완료 가능: 파일 생성 → 검색 → 편집 → 실행 검증, 전 과정에서 막히지 않 음	제2-5장
2단계	+ MCP	MCP 서버 하나를 성공적으 로 장착하고, 작업 중 그 도 구를 호출하며 로그가 호출 을 증명함	9.1절 + 9.6절
3단계	+ 서브에이전트	병렬 서브에이전트 작업을 한 번 성공적으로 시작(예: 모듈 2개를 동시 조사), 부모 에이전트가 결과를 취합할 수 있음	9.2절 + 9.7절
4단계	+ 워크플로우	3단계 이상의 워크플로우를 성공적으로 설정하고 실행, 각 단계가 dependsOn 순 서대로 실행됨	9.3절 + 9.8절

왜 건너뛰면 안 되는가: 1단계가 통과되지 않은 상태 → 2단계에서 오류가 나도 설정이 잘못된 건지 기본 도구 체인을 못 쓰는 건지 판단할 수 없음; 2단계가 통과되지 않은 상태 → 3단계에서 병렬화가 문제를 증폭시킴; 3단계가 통과되지 않은 상태 → 4단계 YAML을 작성해도 틀림. 1단계에서 2단계로 넘어가기 전에, 먼저 제8장 8.1절 능력 패키지 지도를 읽으세요 — 필요한 것이 이미 내장되어 있는지 확인해, "이미 있는 능력"에 MCP를 설정하는 낭비를 피하세요.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** MCP, 서브에이전트, 워크플로우가 각각 어떤 문제를 해결하는지 말해보세요. 셋 사이에는 어떤 연관이 있나요?(정답 확인: 9.1-9.3절 첫 단락 + 9.9절 비교표)
2. **이해 문제:** "워크플로우"와 "여러 라운드 대화"의 차이를 설명하세요. 어떤 시나리오에 워크플로우를 써야 할까요?(정답 확인: 9.3절 + 9.8절)
3. **실습 문제:** dsh web에서 "저장소 디렉터리 구조를 조사하고 리포트를 생성해줘"를 보내고, read/glob/grep이 자동으로 호출되는지 관찰하세요. "데이터베이스 조회" 능력을 추가하고 싶다면 MCP를 써야 할까요, host 플러그인을 써야 할까요?(정답 확인: 9.1절 + 9.9절)
4. **실습 문제:** "서브에이전트 병렬 조사"를 설계해보세요: 모듈 3개(src/lib, src/cli, src/web)를 조사하며, 부모 Agent의 계획 + 서브에이전트 3개의 작업 설명을 작성해보세요(정답 확인: 9.2절 + 9.7절)
5. **사고 문제:** "파일 100개를 읽고 취합"하는 작업에서, 단일 Agent 순차 처리, 서브에이전트 5개 병렬, 워크플로우로 5개 배치 순차 처리는 각각 어떤 장단점이 있을까요?(정답 확인: 9.9절 비교표)
6. **사고 문제:** dsh가 "회사 내부 직원 정보를 조회"하게 하고 싶다면, 공식 내장 도구, MCP, host 플러그인 세 경로 각각 무엇이 필요할까요? 어느 것이 가장 빠를까요?(정답 확인: 제8장 8.1절 + 본 장 9.1절 + 제3장 3.4절)

자주 묻는 질문 FAQ

Q1: MCP는 무엇인가요? dsh 플러그인과는 무엇이 다른가요?

MCP는 "Agent에 외부 도구를 꽂는" 개방형 프로토콜입니다. 플러그인은 dsh 런타임 안에서 능력을 등록합니다 (host 반쪽이 Node 프로세스에서 실행됨); MCP 서버는 독립된 프로세스로, 프로토콜을 통해 통신합니다. 플러그인은 깊게 통합되고 성능이 좋으며; MCP는 결합도가 낮고 재사용 가능해 기존 시스템을 연동하기에 적합합니다.

Q2: 서브에이전트는 어떻게 "병렬"로 동작하나요? 충돌은 없나요?

서브에이전트는 부모 Agent가 위임하며, 각자 독립적으로 실행됩니다(packages/subagent/*). 병렬 실행 시 각 서브에이전트는 자신의 컨텍스트를 가지며 서로 간섭하지 않습니다. 하지만 여러 서브에이전트가 같은 파일을 수정하면 충돌할 수 있으니 — 작업을 분해할 때 겹치지 않도록 하는 것을 권장합니다.

Q3: 워크플로우의 "결정론적 프로세스"는 무슨 뜻인가요? 모델이 자유롭게 진행하는 것과 무엇이 다른가요?

워크플로우는 단계를 프로세스로 정의하고(예: cordis.yml), 순서/조건에 따라 실행하며, 매 단계에서 무엇을 할지가 지정되어 있습니다. 여러 라운드 대화는 "모델이 자유롭게 진행"하는 것입니다. 결정론적 프로세스는 "반드시 고정된 단계로 진행해야 하는" 시나리오에 적합하고, 자유로운 진행은 "탐색적인 작업"에 적합합니다.

Q4: MCP 서버(예: 데이터베이스)를 하나 연결하고 싶은데, 구체적으로 어떻게 하나요?

(공식 changelog를 기준으로) ① @deepseek-ai/dsh-mcp-client 설치; ② cordis.patch.yml에 장착(9.6절 참고); ③ 서버 주소/명령 설정; ④ dsh web 재시작. 자세한 내용은 공식 packages/mcp/ 문서 참고.

Q5: 서브에이전트와 "dsh 프로세스를 여러 개 여는 것"은 무엇이 다른가요?

서브에이전트는 dsh 내부의 위임 메커니즘으로(packages/subagent/*), 부모 Agent의 컨텍스트와 세션을 공유합니다. 다중 프로세스는 완전히 독립된 세션입니다. 서브에이전트는 "작업을 분해한 후 병렬 실행 + 결과 취합"에 적합하고, 다중 프로세스는 "완전히 독립적인 작업"에 적합합니다.

Q6: 본 장에서 일부 기능이 "미실측"이라고 했는데, 어떤 기능이 이미 사용 가능한지 어떻게 아나요?

공식 packages/AGENTS.md와 packages/ 디렉터리를 확인하세요. 완전한 소스 코드 + 예시가 있는 기능은 기본적으로 사용 가능합니다; 패키지 이름만 있고 문서가 없는 것은 아직 개발 중일 수 있습니다. rc 단계에서는 공식 changelog를 기준으로 삼으세요.

부록 A: 용어집과 명령어 빠른 참조

제 10 장: 복잡한 실전 사례(dsh가 실제로 해낸 것)

본 장의 목표: dsh에서 실제로 완료된 복잡한 작업 두 가지를 통해, 다단계 도구 체인에서 dsh의 실제 능력, 산출물 품질, 소요 시간을 보여드립니다.

개인정보 안내: 두 사례 모두 합성 데이터/가상 코드를 사용하며, 실제 비즈니스 데이터나 민감한 정보는 전혀 포함하지 않습니다.

TL;DR(본 장 핵심, 30초 버전)

1. **사례 A(데이터 품질 분석, 186초)**: 52줄의 지저분한 데이터 → 분석 → 정제 스크립트 → 시각화 → 실행 검증, 52→35줄로 정리, 모든 문제 해결
2. **사례 B(5개 버그 수정 + 49개 테스트, 94초)**: 의도적으로 버그 5개를 심은 계산기 모듈 → 전부 수정 → 49개 테스트 전부 통과, 경계/정밀도/예외 커버
3. **dsh의 특징**: 다단계 도구 체인 자동 오케스트레이션, 판단력 있음(데이터 트레이드오프를 능동적으로 설명), 산출물 추적 가능, 소요 시간 제어 가능(94-186초)
4. **핵심 시사점**: 프롬프트에 검수 기준을 명확히 쓰면(예: "실행 검증"), dsh가 그대로 페루프를 완성합니다
—— 복잡한 작업에는 "무엇을 할지 + 어떻게 검수할지"를 명확히 할 것을 권장합니다
5. **실패 복구**: 본 사례에서는 트리거되지 않았습니다; 프로덕션 환경에서는 guard 타임아웃 + 사람의 승인을 설정할 것을 권장합니다

<details><summary>본 장 목차</summary>

- 사례 A: 데이터 품질 분석 + 정제 + 시각화(186초)
- 사례 B: 5개 버그 코드 수정 + 49개 테스트(94초)
- 사례 요약: 복잡한 작업에서 dsh의 특징
- 사례 C: HTML 파싱(Web 프론트엔드, 약 3분 18초)
- 사례 D: 코드 리팩터링(소프트웨어 엔지니어링, 약 4분 37초)
- 사례 E: API 문서 생성(문서, 약 1분 04초)
- 2차 사례 특징

사례 A: 데이터 품질 분석 + 정제 + 시각화(186초)

작업

지저분한 데이터가 포함된 합성 JSON을 dsh에 제공합니다(52줄: 결측값, 타입 오류, 중복 행, 이상치 99999), 요구 사항:

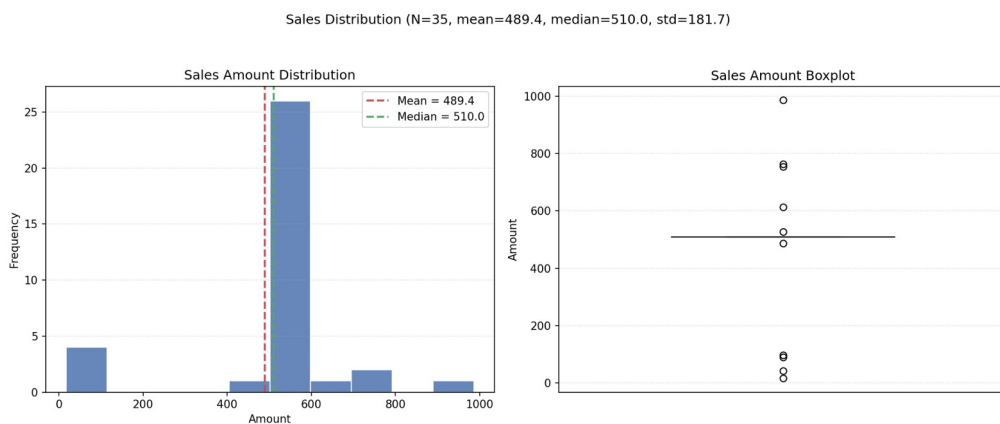
1. 데이터 품질 문제 분석
2. 정제 스크립트 `clean.py` 작성
3. 시각화 스크립트 `visualize.py` 작성(`chart.png` 생성)
4. 두 스크립트를 실행해 검증

dsh가 한 방법(도구 체인)

```
read(sales_data.json) → write(clean.py) → bash(python clean.py)
→ write(visualize.py) → bash(python visualize.py) → read(출력) → 요약
```

산출물과 검증

산출물	결과
<code>clean.py</code>	✅ 실행 성공, 52 → 35줄, 모든 문제 해결
<code>visualize.py</code> + <code>chart.png</code>	✅ 유효한 PNG 생성(1950×825)
정제 전략	결측 금액은 중위수로 채움, 이상치 삭제, 중복 제거, 타입 통일



출력의 핵심 포인트(판단력을 보여줌, 기계적 실행이 아님)

- 중위수로 채우면 히스토그램에 510 스파이크가 생김 — 대체값 처리의 고유한 부작용이니 설명이 필요함
- 99999가 실제 대규모 주문이라면, 일괄 삭제 대신 IQR/분위수 방법을 쓰는 것이 좋음
- 정제 후 통계: N=35, 평균 489.43, 중위수 510, 표준편차 181.73

dsh는 작업을 실행했을 뿐 아니라, 데이터 처리의 트레이드오프와 가정을 능동적으로 설명했습니다 — 이것이 agent 엔지니어링 레이어가 "생각한다"는 증거입니다.

사례 B: 5개 버그 코드 수정 + 49개 테스트(94초)

작업

의도적으로 버그 5개를 심은 Python 계산기 모듈을 dsh에 제공합니다, 요구사항: 모든 버그를 찾음 → 수정 → 완전한 단위 테스트 작성 → 실행 검증.

심어놓은 버그

1. add가 $a - b$ 로 잘못 작성됨
2. divide가 0으로 나눌 때 조용히 0을 반환함(예외를 던져야 함)
3. factorial이 음수일 때 조용히 -1을 반환함(예외를 던져야 함)
4. factorial의 `range(1, n)`이 n 을 곱하는 것을 빠뜨림
5. `format_money`가 소수점 둘째 자리로 포맷되지 않음

dsh의 수정 + 테스트(검증: 49 passed)

전체 테스트는 `case-test-calculator.py` 참고 — 49개 케이스가 커버하는 것:

함수	커버리지
<code>add</code>	양수/음수/0/부동소수점/정밀도/교환법칙
<code>subtract</code>	음수 결과/0 경계/부동소수점
<code>multiply</code>	부호 조합/0 곱셈/부동소수점
<code>divide</code>	정수 나눗셈/부호/부동소수점/0으로 나누면 반드시 <code>ZeroDivisionError</code> 를 던져야 함
<code>factorial</code>	0!/1! 경계/알려진 값/음수는 반드시 <code>ValueError</code> 를 던져야 함
<code>format_money</code>	0 채우기/반올림/0.1+0.2 반올림/음수

```
python -m pytest test_calculator.py -v # 49 passed in 0.37s
```

관찰: dsh의 성과

- 완전한 페루프: 읽기 → 수정 → 테스트 작성 → 테스트 실행 → 요약, 사람의 개입 없음
- 수정 품질: 5개 버그 전부 수정하고 docstring 설명도 보충
- 테스트 설계: 경계(0으로 나누기/음수/정밀도)를 커버, "돌아가기만 하면 된다"는 수준이 아님

사례 요약: 복잡한 작업에서 dsh의 특징

항목	성과
다단계 도구 체인	✅ 자동 오케스트레이션(읽기→쓰기→실행→검증→요약)
복잡한 작업 소요 시간	94초-186초(같은 모델, 같은 게이트웨이, benchmark의 단순 작업보다 느리지만 제어 가능)
산출물 품질	✅ 판단력 있음(데이터 트레이드오프 설명, 테스트 경계 설계)
산출물 추적	✅ 생성된 파일을 대화 끝에서 바로 열 수 있음
실패 복구	본 사례에서는 트리거되지 않음; 프로덕션에서는 guard 타 임아웃 + 사람의 승인을 권장

초보자에게 주는 시사점: dsh는 "여러 파일, 여러 단계, 검증이 필요한" 작업을 능숙하게 처리합니다 — 이것이 Agent 런타임으로서 dsh의 주요 가치 시나리오이기도 합니다. 복잡한 작업에는: 프롬프트에 **검수 기준을 명확히** 쓸 것을 권장합니다(예: "실행 검증"), 그러면 dsh가 그대로 페루프를 완성합니다.

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 원문을 보지 않고 사례 A(데이터 품질 분석)의 도구 체인 순서(read → write → bash → ...)와 최종 산출물이 무엇인지 말해보세요

정답 확인: 본 장 "사례 A"의 "dsh가 한 방법" 단락 참고

2. **이해 문제:** 사례 B(5개 버그 수정)에서 dsh가 보충한 49개 테스트는 어떤 경계 상황을 커버했나요? 왜 "0으로 나누면 반드시 ZeroDivisionError를 던져야 함"이 중요한 테스트 포인트인가요?

정답 확인: 본 장 "사례 B"의 테스트 커버리지 표 참고

3. **실습 문제:** 비슷한 "데이터 품질 분석" 작업을 설계해보세요: 지저분한 데이터가 포함된 CSV를 준비하고(최소 3가지 문제: 결측값, 타입 오류, 이상치), dsh에게 분석 + 정제 + 시각화를 시키는 프롬프트를 작성해 소요 시간과 산출물을 기록하세요

정답 확인: 본 장 사례 A의 작업 설명 + 산출물 검증 표 참고

4. **실습 문제:** 비슷한 "버그 수정" 작업을 설계해보세요: 의도적으로 버그 3개를 포함한 Python 모듈을 작성하고(예: 문자열 처리, 날짜 계산), dsh에게 버그 찾기 → 수정 → 테스트 작성 → 실행 검증을 시켜 소요 시간과 테스트 통과율을 기록하세요

정답 확인: 본 장 사례 B의 작업 설명 + "dsh의 성과" 관찰 단락 참고

5. **사고 문제:** 사례 A에서 dsh는 "데이터 처리의 트레이드오프와 가정을 능동적으로 설명"했습니다(예: 중위수 채우기의 스파이크 부작용). 이 행동이 "agent 엔지니어링 레이어"의 설계에 어떤 시사점을 줄까요? 모델은 그저 명령을 실행할 뿐인가요, 아니면 정말로 "판단력"을 가지고 있을까요?

정답 확인: 본 장 사례 A의 "출력의 핵심 포인트" 단락 + 사례 요약 표 참고

6. **사고 문제:** 본 장 맺음말은 "프롬프트에 검수 기준을 명확히 쓰면 dsh가 그대로 페루프를 완성한다"고 말합니다. 만약 사례 A의 프롬프트가 "이 데이터 좀 처리해줘"라고만 하고 "실행 검증"을 말하지 않았다면, dsh의 성과가 어떻게 달라졌을까요?

정답 확인: 본 장 "초보자에게 주는 시사점" 단락 + 사례 A의 도구 체인(bash 실행 검증 포함) 참고

자주 묻는 질문 FAQ

Q1: 사례 A는 186초, 사례 B는 94초 걸렸는데, 이 속도는 빠른 편인가요 느린 편인가요?

작업 복잡도와 모델 강도에 따라 다릅니다. 같은 모델, 같은 게이트웨이 기준으로, benchmark의 단순 작업보다는 느리지만 제어 가능한 수준입니다. 핵심은 절대적인 속도가 아니라 "다단계 도구 체인 자동 오케스트레이션 + 산출물 품질 + 사람의 개입 없음"이라는 종합적인 가치입니다. 속도를 높이고 싶다면 제6장의 추론 강도 전략 + 속도 개선 플러그인 예제를 참고하세요.

Q2: 사례에서 "합성 데이터"란 무엇인가요? 왜 실제 데이터를 쓰지 않나요?

개인정보 보호 때문입니다. 본 백서는 실제 비즈니스 데이터나 민감한 정보를 다루지 않습니다. 합성 데이터는 "의도적으로 구성한, 실제 정보를 포함하지 않는 데이터"로, dsh의 능력을 시연하는 데 사용됩니다. 여러분 자신의 환경에서는 실제 데이터를 쓸 수 있지만, 데이터 보안에 유의해야 합니다.

Q3: dsh가 복잡한 작업에서 "막혔다면"(예: 특정 단계의 도구 호출이 실패) 어떻게 해야 하나요?

몇 가지 대처법: ① dsh 프로세스 로그를 보고 어느 단계에서 실패했는지 위치 확인; ② 도구 호출의 입력이 올바른지 확인(파일 경로, 명령 문법 등); ③ guard 타임아웃을 설정해(guard/* 패키지) 무한 대기를 방지; ④ 프로덕션 환경에서는 사람의 승인을 설정해(interaction/*) 위험한 작업 전에 확인하는 것을 권장합니다.

Q4: 사례 B의 49개 테스트는 dsh가 자동 생성한 것인데, 품질은 어떤가요?

품질이 괜찮습니다. 경계(0으로 나누기/음수/정밀도), 예외(특정 예외를 반드시 던져야 함), 일반(양수/음수/0/부동소수점)을 커버했습니다. 하지만 자동 생성된 테스트는 "비즈니스 로직 고유의 경계"를 놓칠 수 있으니 — 테스트 설계를 사람이 리뷰하고, 도메인 특화 케이스를 보충하는 것을 권장합니다.

Q5: dsh에게 "여러 파일, 여러 단계"의 작업을 시키고 싶은데, 프롬프트를 어떻게 써야 효과가 가장 좋을까요?

세 가지 원칙: ① 목표를 명확히("무엇을 할지"); ② 검수 기준을 명확히("어떻게 완료로 판단할지", 예: "실행 검증", "49개 테스트 전부 통과"); ③ 제약을 명확히("무엇을 하면 안 되는지", 예: "원본 데이터를 삭제하면 안 됨"). 본 장 두 사례의 프롬프트 구조를 참고하세요.

Q6: 사례 요약에서 "실패 복구는 본 사례에서 트리거되지 않았다"고 했는데, dsh의 실패 복구 능력은 어떤가요?

dsh는 compaction(컨텍스트 오버플로우 복구), guard(타임아웃), interaction(승인) 등의 메커니즘을 가지고 있지만, 복잡한 시나리오의 실패 복구는 아직 발전하는 중입니다. 프로덕션 환경 권장사항: ① guard 타임아웃을 설정해 멈춤을 방지; ② 사람의 승인을 설정해 오작동을 방지; ③ 긴 작업은 단계별로 검증하고, "한 번에 전부 실행한 후 확인"하지 마세요. 구체적인 설정은 제8장 8.5절 보안 모델을 참고하세요.

부록 A: 용어집과 명령어 빠른 참조

확장 사례(2차, 세 가지 기능 영역)

전체 실행 리포트는 `case-expansion-report.md` 참고, 산출물은 `docs/assets/` 에 있습니다. 전부 합성 데이터입니다.

사례 C: HTML 파싱(Web 프론트엔드, 약 3분 18초)

작업: 표+폼+차트가 포함된 합성 HTML을 주고, 서드파티 의존성 없이 표를 추출해 CSV로 출력하는 파싱 스크립트를 작성.

dsh의 성과:

- **의존성 없음의 트레이드오프:** BeautifulSoup 대신 표준 라이브러리 `html.parser`를 능동적으로 선택
- **견고성:** 표 중첩 깊이 카운팅을 구현해 범위 초과를 방지
- **경로 독립성:** `__file__`로 입력을 위치 확인, 실행 디렉터리에 의존하지 않음

산출물: `case-c-parser.py` | 검증: 9행×7열 CSV 출력, 원본 표와 필드별로 일치

사례 D: 코드 리팩터링(소프트웨어 엔지니어링, 약 4분 37초)

작업: 약 200줄의 절차형 주문 스크립트(중복 코드 `calc_order_total` / `calc_order_total_v2` 포함)를 OOP + 책임 분리로 리팩터링, 동작 일치, 테스트 검증.

dsh의 성과:

- **정확한 중복 식별:** 동일한 로직 두 개를 `Order.total()`이라는 단일 진입점으로 병합
- **SOLID 설계:** Product/Customer/Order(모델) + OrderProcessor(조회) + ReportGenerator(리포트)
- **하위 호환성:** 모듈 레벨 함수 인터페이스를 유지해 바로 교체 가능
- **테스트 깊이:** 17/17 PASS — 함수 반환값만 비교하는 게 아니라 **스크립트 레벨 전체 실행 산출물** (`report.txt` / `orders.json`)도 비교
- **샌드박스 인식:** `tempfile.mkdtemp`가 샌드박스에서 0700 ACL로 쓸 수 없음을 발견하고 능동적으로 방안 변경

산출물: `case-d-orders_refactored.py` + `case-d-test_refactor.py` | 검증: **17/17 PASS**

사례 E: API 문서 생성(문서, 약 1분 04초)

작업: 함수 8개가 포함된 Python 모듈에 완전한 Markdown API 문서 생성(파라미터 표/반환값/예외/예시/경계).

dsh의 성과:

- **계약 차이 식별:** "문서 선언"과 "코드 구현"의 차이를 능동적으로 발견(예: `user_id` 문자 집합이 강제 검증되지 않음), 경계 섹션에 있는 그대로 작성
- **방어적 문서화:** 각 함수마다 호출 함정을 표시(음수 limit, 빈 updates의 동작)
- **ReadTheDocs 스타일:** 423줄, 함수 8개 전체 커버, doctest 스타일 예시 포함

산출물: case-e-api.md

2차 사례 특징

항목	성과
영역 간 능력	✅ Web/엔지니어링/문서 세 가지 작업 모두 페루프 완성
엔지니어링 인식	✅ 의존성 없음 트레이드오프, 샌드박스 인식, 하위 호환성, 계약 차이 식별
검증 습관	✅ 모든 사례에서 실행 검증(스크립트 레벨 산출물 비교 포함)
소요 시간	사례당 1-5분, 복잡도가 높을수록 더 걸림

요약: dsh는 "코드를 작성하고 + 검증해야 하는" 엔지니어링 작업에서 안정적인 성과를 보이며, **트레이드오프(의존성/호환성/보안)를 판단할 줄 압니다** — 이것이 agent가 "답할 수 있음"에서 "일을 해낼 수 있음"으로 넘어가는 분기점입니다.

제 11 장: 미래 전망 — dsh와 Agent 생태계의 진화 예측

본 장의 목표: 여러 각도에서 dsh와 그 생태계의 미래를 추론합니다 — 기술, 생태계, 경쟁, 산업, 개발자 기회, 리스크. 예측은 단정이 아니라 "현재 사실을 기반으로 한 추론"이며, 투자할 가치가 있는지 판단하는 데 참고하시기 바랍니다.

TL;DR(본 장 핵심, 30초 버전)

- 1. **단기(1-3개월):** 정식 버전 0.1.0 출시, 플러그인 생태계 폭발, 튜토리얼/도구류 프로젝트가 첫 물결의 보너스를 누림
- 2. **중기(1년):** dsh가 "프로그래머블 Agent 베이스"의 사실상 표준 후보가 되고, 버티컬 시나리오 플러그인이 성숙
- 3. **장기(2-3년):** Agent 엔지니어링 레이어가 표준화되고, dsh 생태계와 모델 반복이 서로를 성장시킴
- 4. **최대 리스크:** 파괴적 변경, 생태계 파편화, 공식 전략 전환
- 5. **개인의 기회:** 지금 진입하는 것(튜토리얼/플러그인/도구)이 저비용 초기 보너스입니다

<details><summary>본 장 목차</summary>

- 11.1 기술 진화 예측
- 11.2 생태계 발전 예측
- 11.3 경쟁 구도 예측
- 11.4 산업 응용 예측
- 11.5 개발자 기회(지금 진입하는 사람)
- 11.6 리스크와 불확실성(솔직한 버전)
- 11.7 백서 자체의 전망
- 11.8 독자에게 드리는 최종 제안

11.1 기술 진화 예측

시기	예측	근거
단기	0.1.0 정식 버전 출시(rc 제거), 파괴적 변경이 수렴됨	현재 rc.6, 공식이 "파괴적 변경이 있을 것"이라고 명시, 반복 속도가 매우 빠름

단기	공식이 TUI와 상호작용형 CLI를 보완 (커뮤니티 요청이 가장 많음, #172 15+ 댓글)	공식 Discussion에서 가장 뜨거운 요구 사항
중기	캐시/성능 최적화가 주요 테마가 됨 (reasoning_effort 세분화, 캐시 적중률 도구화)	고캐시 적중률이 이미 비용 우위임(제5장), 도구화되어 가시화될 것
중기	플러그인 생태계 표준화(플러그인 마켓/원클릭 설치/품질 등급)	공식에 이미 dsh-plugin topic이 있고, 플러그인 수 증가가 필연적으로 마켓을 촉발
장기	Agent 런타임 아키텍처가 "엔지니어링 패러다임"으로 자리잡음(예전 Linux 커널 모델의 Agent 버전과 유사)	"everything is a plugin"이 이미 조합 가능한 아키텍처로 검증됨

11.2 생태계 발전 예측

- **플러그인 수:** 초일 → 1년 내 예상 수십 개에서 백여 개(비슷한 생태계의 S곡선 참고)
- **콘텐츠 생태계:** 튜토리얼/블로그/awesome 목록이 먼저 폭발(한국어/중국어 콘텐츠는 현재 공백 — 본 백서가 첫 체계적 튜토리얼)
- **커뮤니티 규모:** 공식 Discussion에 매일 새 토론이 추가되고(하루에 30+ 개의 새 주제를 관찰), Discord 활동도 동반 성장할 것
- **선두 프로젝트:** 몇 개의 "생태계 플래그십"이 탄생할 것임(VSCode 생태계의 Prettier/ESLint 같은 위상) — 도구형 + 튜토리얼형이 가장 가능성 높음

11.3 경쟁 구도 예측

항목	dsh 포지셔닝의 진화
vs Claude Code	단기적으로는 여전히 "바로 사용 가능성이 못 미침", 장기적으로는 "커스터마이징 가능 + 오픈소스 + 비용"으로 차별화
vs OpenCode	둘 다 오픈소스; dsh의 공식 백엔드 + 플러그인 생태계가 차별점(OpenCode는 공식 백엔드 없음)
vs 자체 구축 프레임워크	dsh는 "반제품에서 완제품으로 가는 다리" — 직접 구축보다 편하고, 클로즈드소스보다 제어 가능
모델 측면	DeepSeek 모델의 매 세대 반복이 dsh 생태계의 가치를 증폭시킬 것(모델 + 런타임의 시너지)

핵심 판단: dsh의 승부수는 "모델이 얼마나 강한가"(그건 DeepSeek의 몫)가 아니라, "**Agent 엔지니어링 레이어가 표준이 될 수 있는가**"에 있습니다 — "플러그인을 어떻게 작성하고, 생태계가 어떻게 성장하는지"를 누가 정의하느냐가 이 레이어의 승자를 결정합니다.

11.4 산업 응용 예측

- **금융/데이터 분석:** 긴 세션 + 고캐시 적중률 = 비용 친화적인 배치 분석이 가장 먼저 자리잡을 것
- **기업 지식 베이스/문서:** 1M 컨텍스트 + 문서 도구 체인, 기업 내 지식 관리는 자연스러운 시나리오
- **교육:** 배치 문제 출제/해설/정리, 비용에 민감한 시나리오가 캐시의 혜택을 누림
- **버티컬 Agent 제품:** "산업별 Agent 패키징"이 등장할 것임(dsh 위에 산업별 플러그인/설정 팩을 만드는 것)

11.5 개발자 기회(지금 진입하는 사람)

역할	기회	시기
튜토리얼/콘텐츠 작성자	한국어/중국어 튜토리얼 공백(백서가 첫 번째임)	지금(보너스가 가장 큼)
플러그인 개발자	공식이 아직 다루지 않은 방향: TUI, 원격 접속, 모바일, 캐시 도구	지금
도구형 프로젝트	속도 개선, 소요 시간 시각화, MCP 도구 키트	지금(이미 속도 개선 플러그인 실험으로 검증함)
생태계 인프라	플러그인 마켓, 템플릿 생성기, 평가 도구	중기
산업 솔루션	금융/교육 등 산업 플러그인 팩	중기

11.6 리스크와 불확실성(솔직한 버전)

1. **파괴적 변경:** rc 단계의 API가 자주 바뀔 수 있습니다 — 플러그인/튜토리얼이 지속적으로 따라가야 합니다(백서도 동기화하여 업데이트할 것입니다)
2. **생태계 파편화:** 플러그인 품질이 들쭉날쭉하고 표준이 부족합니다 — 초기에는 "고품질 시범 프로젝트"가 필요합니다(본 백서 + 제4장 예제가 그 시범 중 하나입니다)
3. **공식 전략 전환:** 공식이 특정 방향(예: 내장 TUI)을 거둬들일 수 있습니다 — 생태계 프로젝트는 차별화된 진입 장벽이 있어야 합니다

- 4. **모델 경쟁:** 다른 모델의 반복이 DeepSeek의 매력을 약화시킬 수 있습니다 — 하지만 dsh의 모델 독립적 설계(OpenAI 호환 모델도 연결 가능)가 완충 역할을 합니다
- 5. **인기 하락:** dsh 생태계가 자리잡지 못하면 초기 투자가 "헛수고"가 될 수 있습니다 — 하지만 튜토리얼/스킬 자체는 이전 가능한 자산입니다

11.7 백서 자체의 전망

- **콘텐츠:** dsh의 반복에 맞춰 지속적으로 업데이트(버전 번호를 rc/정식 버전에 맞춤)
- **다국어:** 영어판의 완성도를 중국어판 수준으로 끌어올림
- **사례:** 더 많은 산업 실제 사례(규제 준수 전제하에)
- **생태계:** awesome 목록, 공식 Discussion과 연계해 생태계 콘텐츠 허브 중 하나가 됨

11.8 독자에게 드리는 최종 제안

- **한번 해보고 싶다면:** 지금 설치해서 놀아보세요(제2장), 비용은 거의 0입니다
- **투자하고 싶다면:** "공식이 아직 다루지 않았고 + 여러분이 쌓아온 것이 있는" 방향을 고르세요(TUI/산업 플러그인/튜토리얼)
- **관망하고 싶다면:** 0.1.0 정식 버전을 기다리되, 생태계 초기 보너스는 놓칠 것입니다 — 초기 보너스 ~ 선

점자의 안내심

예측은 틀릴 수 있지만, "지금 진입하는 비용이 가장 낮다"는 사실은 대체로 틀리지 않을 것입니다.

부록 A: 용어집과 명령어 빠른 참조

제 12 장: 알려진 한계와 경계 — 솔직한 버전

본 장의 목표: 과대포장도 폼하도 하지 않고, dsh의 현재 **알려진 한계와 사용 경계**를 나열합니다. 본 백서를 쓰는 시점에 dsh는 여전히 0.1.0-rc.6(프리릴리스 단계)이며, 이 한계들은 "지금 이 순간의 사실"이지 "영원한 숙명"이 아닙니다 — 버전이 반복되면서 바뀔 것입니다.

본 장의 모든 "알려진 문제"는 로컬 실측 + 공식 저장소 deepseek-ai/deepseek-harness 디스커션(2026-08-14 기준)의 공개 피드백에서 가져온 것입니다.

TL;DR(본 장 핵심, 30초 버전)

- rc 단계가 가장 큰 불확실성입니다:** API/설정이 언제든지 파괴적으로 변경될 수 있으니, 투자하기 전에 버전 라인을 먼저 고정하세요
- 플러그인 생태계는 초기 단계입니다:** 공식 패키지는 60+개지만 커버리지가 제한적이고, 서드파티 플러그인이 적으며, 상호작용 형태(TUI)가 빠져 있습니다
- 크로스플랫폼 약점:** Windows에서는 경로/인코딩 버그(UTF-16, 0x00) 외에도, 포트 예약, LAN 접속, 샌드박스 안정성, WSL 워크스페이스, 자식 프로세스 종료 등 여러 계열의 문제가 있습니다(빠른 참조표는 12.4 참고)
- 성능 경계가 충분히 검증되지 않았습니다:** 고동시성, 대형 코드베이스 시나리오의 공개된 부하 테스트 데이터가 부족합니다
- 공식 전략 리스크:** 오픈소스 약속 vs 상업화 전환은 도구 선택 시 반드시 고려해야 할 블랙스완입니다

<details><summary>본 장 목차</summary>

- 12.1 rc 단계의 불안정성
- 12.2 초기 단계의 플러그인 생태계
- 12.3 크로스플랫폼 약점
- 12.4 크로스플랫폼 알려진 문제 빠른 참조표
- 12.5 충분히 검증되지 않은 성능 경계
- 12.6 성숙한 Agent와의 격차

- 12.7 공식 전략 리스크
- 12.8 백서 자체의 한계

12.1 rc 단계의 불안정성

파괴적 변경이 언제든 발생

dsh는 rc.1부터 rc.6까지 최소 한 번의 **의존성 단절**을 겪었습니다: 초기 버전의 @deepseek-ai/* 패키지 버전 라인 불일치로 플러그인이 설치되지 않는 문제(404/ERR_MODULE_NOT_FOUND)가 있었습니다. rc.6에서 많이 수렴되었지만, rc.7/정식 버전에서 다시 그런 일이 없을 것이라는 **어떤 약속도 없습니다**.

여러분에게 미치는 실질적 영향:

- 튜토리얼/플러그인을 작성하면, 2주마다 따라서 수정해야 할 수 있습니다
- 버전 라인을 고정하는 것(^0.1.0-rc.6)은 기본 조치이지만, rc 라인 업그레이드가 "상위 호환"인지 "완전히 갈아엎는" 것인지는 공식의 그때그때 판단에 달려 있습니다

[!WARNING]

프로덕션 환경 도입은 신중히 평가하세요. 본 백서 집필 시점에도 공식은 여전히 빠르게 반복하고 있으며, **오늘의 작성법이 내일 폐기될 수 있습니다**.

알려진 rc 트러블(로컬/커뮤니티 재현)

트러블	증상	상태
link 개발 의존성 단절	@deepseek-ai/*를 로컬 link할 때 ERR_MODULE_NOT_FOUND, npm 설치는 정상	원인 파악됨(플랫 폴백 디렉터리 메커니즘), 의존성 라인 고정으로 회피 필요
Windows 경로 0x00	중국어/특수문자 경로가 포함된 workspace 생성 실패(ENOENT)	커뮤니티 피드백 진행 중(#151/#396)
fs-sandbox 경쟁 상태	post-check 경로 검사와 workspace-write 존재 여부 사이에 타이밍 경쟁 존재	커뮤니티 피드백 진행 중(#159)
TUI 부재	공식 examples에 상호작용형 터미널 UI가 없음(커뮤니티 자체 구축은 있으나 공식에 미포함)	제안 중(#392)

12.2 초기 단계의 플러그인 생태계

현황: 공식 패키지는 60+개지만, 생태계 전체는 "초일" 상태입니다.

- **서드파티 플러그인이 적음:** 디스커션의 플러그인 게시물이 폭발적으로 늘고 있지만, 품질이 들쭉날쭉하고 대부분 "첫 플러그인" 연습작입니다
- **상호작용 형태 부재:** headless/JSON-RPC/Web 형태는 갖춰져 있지만, **TUI 등 터미널 상호작용 형태는 공식 예시가 없습니다**(커뮤니티의 deepseek-harness-tui가 실현 가능성을 입증했지만 공식에 포함되지 않음)
- **문서가 코드보다 뒤처짐:** 공식 문서는 아키텍처 위주이고, 시작 경로/트러블슈팅 기록은 커뮤니티 생산에 의존합니다(이것이 바로 본 백서가 존재하는 이유입니다)
- **버전 라인 혼란:** npm에 같은 패키지의 여러 버전 라인이 공존하며, 잘못된 라인을 설치하면 404가 발생합니다

판단: 생태계 탐색은 지금 진입하는 것이 초기 보너스입니다(경쟁이 적고 공식의 지원 의지가 강함), 하지만 "공식 문서대로만 하면 돌아간다"고 기대하지 마세요 — 트러블슈팅이 일상입니다.

12.3 크로스플랫폼 약점

dsh는 macOS/Linux에서는 성숙하게 동작하지만, Windows 지원에는 뚜렷한 **2등 시민 느낌**이 있습니다:

- **경로 처리:** 중국어 경로, 특수문자 경로에 실제 버그가 있음(0x00 절단, ENOENT), 커뮤니티에서 다수 보고 (근본 원인 #107, 계열 17+개 게시물)
- **인코딩 문제:** UTF-16 관련 처리에 결함이 있어 터미널 출력이 가끔 깨짐
- **CLI/TUI 논쟁 미결:** 공식 디스커션에 "CLI가 정도다 vs TUI가 상호작용의 미래다"라는 노선 논쟁이 있음 (#386), 방향이 명확해지기 전에 터미널 UI 형태에 투자하는 것은 리스크가 있습니다

아래의 Windows 계열 문제는 공식 디스커션 발굴 리포트(docs/research/discussion-mining.md, 2026-08-14 수집, rc.6 시대)에서 가져온 것으로, 모두 커뮤니티가 실제로 재현했으며 대부분 아직 수정되지 않았습니다.

포트: 기본 3080/Hyper-V 예약 구간과 충돌(#589)

증상: Windows에서 기본 포트 3080으로 시작하면 EACCES 오류가 발생하고, 서비스가 시작되지 않아 web 화면이 열리지 않습니다.

근본 원인: Hyper-V/WSL2를 활성화하면, Windows가 동적 포트 구간(3070-3169)을 예약하는데, 3080이 정확히 그 안에 들어갑니다.

임시 해결법: 예약 구간 밖의 포트로 바꿉니다(커뮤니티 실측으로 13080 사용 가능).

LAN을 통한HTTP 접속 시RPC 전부 실패(#755)

증상: LAN 내에서 HTTP(localhost도 아니고 HTTPS도 아님)로 접속하면 모든 RPC 호출이 실패합니다.

근본 원인: crypto.randomUUID는 보안 컨텍스트(HTTPS 또는 localhost)에서만 사용 가능한데, LAN의 순수 HTTP는 비보안 컨텍스트에 속합니다.

임시 해결법: 공식 수정 없음; HTTPS나 터널(예: Tailscale)로 접속하면 비보안 컨텍스트 제약을 회피할 수 있습니다(원인에 근거한 추론이며, 커뮤니티 게시물 안에서 검증되지는 않음).

샌드박스: 임시 디렉터리 정리 후 영구적으로 크래시(#758)

증상: Windows에서 샌드박스 임시 디렉터리를 정리한 후 dsh가 영구적으로 크래시되며 자체 복구되지 않습니다(커뮤니티가 P0로 표시), 관련된 안정성 문제 4개가 동반됩니다.

영향: 임시 디렉터리를 한 번 정리하면 = 서비스 완전 사용 불가, 자동 복구 경로 없음.

임시 해결법: dsh 샌드박스 임시 디렉터리를 정리하지 마세요; 이미 발생했다면 프로세스를 재시작해 복구하세요.

WSL: 워크스페이스가 /root만 선택 가능(#160)

증상: WSL/Ubuntu 환경에서 워크스페이스 선택기가 /root 안의 디렉터리만 선택할 수 있고, 나머지 경로는 선택할 수 없습니다.

임시 해결법: 프로젝트를 /root 아래에 두고 사용하세요(제약 안에서는 사용 가능, 제약을 벗어나는 우회 방법은 아직 없음).

자식 프로세스: Windows에서 우아한 종료 부재(#717)

증상: Windows에서 자식 프로세스 관리 체계가 구조적으로 결여되어 있습니다 — 우아한 종료 단계가 없고, 손자 프로세스 출력이 잘리며, spill 권한이 적용되지 않고, shellPath가 기본으로 /bin/bash를 가리킵니다(Windows에는 이 경로가 존재하지 않음).

영향: 긴 작업 종료가 깔끔하지 않고, 출력이 유실되며, 권한 기대가 실패합니다 — 이것은 단일 버그가 아니라 플랫폼 어댑팅의 공백입니다.

임시 해결법: taskkill /T로 프로세스 트리를 수동 정리; shellPath를 Windows의 실제 경로로 명시적으로 설정하세요.

샌드박스 MSYS 충돌(#1889 실측 시그니처): workspace-write가 제한된 토큰을 사용할 때, MSYS 계열 shell은 cygheap 초기화 단계에서 바로 죽습니다(cygheap_user::init: NtSetInformationToken (TokenDefaultDacl), 0xC0000022 / CreateFileMapping ... Win32 error 5) — MSYS 런타임 모델이 제한된 토큰과 직접 충돌하며, 플러그인 레이어에서는 해결책이 없습니다. 이것이 왜 Windows 방안들이 하나같이 danger-full-access를 요구하는지를 설명해줍니다(작성자가 게을러서가 아닙니다).

12.4 크로스플랫폼 알려진 문제 빠른 참조표

본 장의 크로스플랫폼 알려진 문제를 한 표로 정리합니다. 스레드 번호는 모두 공식 디스커션(deepseek-ai/deepseek-harness)의 실제 게시물이며, 2026-08-14 기준입니다.

문제	환경	스레드	임시 workaround
중국어 경로 절단 (readUtf16이 단일 바이트로 0을 판단, 开=U+5F00이 잘림)	Windows	#107 (https://github.com/deepseek-ai/deepseek-harness/discussions/107)(근본 원인; 계열 #151/#396/#563 등 17+ 게시물)	워크스페이스 경로에 중국어/특수문자 피하기; 커뮤니티가 첨부한 수정 패치로 cherry-pick 가능(#244/#580)
디렉터리 선택기 worker 크래시(koffi 관련)	Windows	#30 (https://github.com/deepseek-ai/deepseek-harness/discussions/30)(계열 #154/#236/#449/#768)	설치 레이어에서 koffi@3.1.2로 고정(#293); 선택기 크래시는 아직 우회법 없음
기본 포트 3080이 Hyper-V 예약 구간과 충돌(EACCES)	Windows(Hyper-V/WSL2 활성화)	#589 (https://github.com/deepseek-ai/deepseek-harness/discussions/589)	예약 구간 밖 포트로 변경(예: 13080)
LAN HTTP 접속 시 RPC 전부 실패(crypto.randomUUID가 비보안 컨텍스트)	LAN HTTP(localhost 아님)	#755 (https://github.com/deepseek-ai/deepseek-harness/discussions/755)	HTTPS/터널 접속으로 회피 가능(추론); 공식 수정 없음

샌드박스 임시 디렉터리 정리 후 영구 크래시(자체 복구 안 됨, P0)	Windows	#758 (https://github.com/deepseek-ai/deepseek-harness/discussions/758)	샌드박스 임시 디렉터리 정리 금지; 발생 시 프로세스 재시작 필요
WSL 워크스페이스가 /root 안 디렉터리만 선택 가능	WSL/Ubuntu	#160 (https://github.com/deepseek-ai/deepseek-harness/discussions/160)	프로젝트를 /root 아래에 두고 사용
Windows 자식 프로세스 우아한 종료 부재(손자 프로세스 출력 절단/spill 실패/shellPath 오류)	Windows	#717 (https://github.com/deepseek-ai/deepseek-harness/discussions/717)	수동 taskkill /T로 정리; shellPath 명시적 설정

12.5 충분히 검증되지 않은 성능 경계

백서의 benchmark는 단일 agent, 일반적인 작업 규모를 커버했으며, 아래 시나리오는 공개된 데이터가 부족합니다:

- **고동시성:** 여러 agent 병렬 실행, 대규모 작업 분배
- **대형 코드베이스:** 수백만 줄 규모 저장소의 컨텍스트 관리, 도구 호출 지연
- **장시간 실행:** 수 시간 연속 실행의 안정성, 메모리 누수 리스크
- **캐시 적중률의 경계:** 97% 실측 적중률은 "대화 패턴 반복"에 의존합니다 — 완전히 새로운 작업/콜드 스타트 시나리오에서는 눈에 띄게 낮아질 것입니다

12.6 성숙한 Agent와의 격차

Claude Code, Codex 등 성숙한 제품과 대조하면, dsh의 격차는 구조적입니다(기능 몇 개 보충한다고 따라잡을 수 있는 것이 아닙니다):

항목	dsh(rc.6)	성숙한 Agent
생태계 단계	초일, 서드파티 자산이 희소함	성숙, 완전한 생태계
바로 사용 가능성	profile/플러그인 체계 이해 필요	설치하면 바로 사용
안정성	rc 시기, 파괴적 변경 리스크	안정 버전, 장기 호환 약속
문서	공식은 아키텍처 위주, 시작은 커뮤니티에 의존	공식 문서 완비 + 풍부한 튜토리얼
IDE/도구 체인 통합	시작 단계	깊은 통합

dsh의 포지셔닝: "바로 사용 가능한 완성차"가 아니라 "프로그래머블 레고 베이스플레이트"입니다. dsh를 선택하는 것 = 자유도를 선택하는 것이며, 그 대가는 **직접 조립해야 한다**는 것입니다.

12.7 공식 전략 리스크

도구를 선택하는 입장에서 반드시 고려해야 할 블랙스완:

- **오픈소스 약속:** MIT 오픈소스에 현재 잡음은 없지만, "공식 레벨 플러그인 체계"는 생태계가 공식의 페이스에 깊이 종속됨을 의미합니다
- **모델 종속 경향:** 공식 기본 경험은 DeepSeek 모델에 편향되어 있습니다(llm 플러그인으로 임의 모델 연결 가능하지만, "공식 최적화"는 자사 모델 위주입니다)
- **상업화 전환:** dsh 사용자 수가 늘어나면, 공식이 관리형 서비스/유료 레이어를 출시할 수 있습니다 — 오픈소스 버전이 동등한 능력을 유지할 수 있을지는 미지수입니다
- **방향 표류:** rc 시기에는 기능 추가/삭제가 잦습니다(TUI/CLI 노선 논쟁이 바로 그 신호), 잘못된 방향에 베팅하면 = 매몰 비용

12.8 백서 자체의 한계

마지막으로, 이 백서 자체에 대해서도 솔직히 말씀드립니다:

- **rc.6 기준으로 작성됨:** 모든 명령어, 설정, 데이터는 rc.6에서 검증되었으며, **새 버전에서는 전부 무효화될 수 있습니다**
- **benchmark 샘플이 제한적:** 단일 머신, 단일 모델, 제한된 작업 집합이며, 권위 있는 평가가 아닙니다
- **중국어 우선:** 영어판은 번역/축약본이라 중국어판 내용보다 뒤쳐질 수 있습니다
- **커버리지가 완전하지 않음:** dsh 공식 패키지는 60+개지만, 본 핸드북이 깊이 다룬 것은 핵심 경로이며, 롱테일 플러그인은 하나하나 다루지 못했습니다

사용 권장사항: 본 백서를 "rc.6 스냅샷+ 방법론"으로 취급하고, "영원한 경전"으로 여기지 마세요. 버전이 업데이트되면, 먼저 roadmap 학습 경로와 치트시트의 검수 기준을 한 번 실행해보고, 여러분의 의존성 라인을 업데이트할지 결정하세요.

본 장의 정보는 2026-08-14 기준입니다(dsh 0.1.0-rc.6). 이후 버전에 업데이트가 필요하면 Issue/PR로 수정해주시기 바랍니다.

제 13 장: 보안과 샌드박스 모델

본 장의 목표: dsh의 보안 골격을 이해합니다 — 샌드박스는 무엇을 관리하는지, 권한은 어떻게 등급이 나뉘는지, 승인은 어떻게 처리되는지 — 그리고 커뮤니티 감사가 발견한 실제 경계까지. 이것이 "돌아간다"에서 "프로덕션에 올릴 용기가 생긴다"로 넘어가는 핵심 챕터입니다.

TL;DR(본 장 핵심, 30초 버전)

1. **3층 방어선:** 샌드박스(어디서 실행하는가) → 권한 프리셋(무엇을 할 수 있는가) → 승인(위험한 동작에 사람의 확인이 필요한가) — 기본값은 맨몸 실행이 아닙니다
2. **샌드박스는 파일 효과만 관리합니다:** `read-only` / `workspace-write` / `danger-full-access` 세 등급; 네트워크와 프로세스 가시성은 어휘 목록에 없습니다(그래서 `taskkill`로 호스트를 죽이는 등의 사례는 "어휘 목록 밖"이지, 샌드박스가 뚫린 것이 아닙니다)
3. **권한 = 두 개의 다이얼을 묶은 프리셋:** `sandbox/mode + approval/policy`; 기본 표 두 개: `workspace-write`(작업공간 쓰기 + `ask`)와 `danger-full-access`(샌드박스 없음 + `never`)
4. **승인은 fail-closed:** `allowed-once`만 통과시킵니다; `ask`에 응답이 없으면 = 거부(`unavailable`), `never`는 모든 것을 결정론적으로 거부 — `headless`의 기본 자세
5. **커뮤니티 감사가 발견한 실제 경계:** #159 `fs-sandbox` 경쟁 상태, #584 `scrubbedParentEnv` 오탐, #381 `iframe` 클릭재킹, #817 감사 리포트(`vm` 탈출 + 로컬 RPC 무인증)
6. **엔터프라이즈 기준선 한 줄 요약:** 기본값 `workspace-write+ask`, `loopback`만 허용, 신뢰 소스의 플러그인만 설치, 민감한 디렉터리는 `workspace`에 넣지 않기

<details><summary>본 장 목차</summary>

- 13.1 보안 설계 철학: 샌드박스/권한/승인 3층
- 13.2 샌드박스 메커니즘 원리: `fs` 경계와 도구 샌드박스
- 13.3 권한 모델: 도구 권한 등급 표
- 13.4 승인 흐름: 요청에서 허가까지
- 13.5 플러그인 보안 감사 체크리스트

- 13.6 커뮤니티가 확인한 보안 사례
- 13.7 엔터프라이즈 보안 기준선

13.1 보안 설계 철학: 샌드박스/권한/승인 3층

dsh의 보안 골격은 세 층이 각각 한 가지씩 관리합니다(공식 docs/subsystems/* 아키텍처 문서 기준, 검증됨: sandbox.md / permission-presets.md / approval.md; 제8장 8.5절 실측과 교차 확인):

층	무엇을 관리하는가	공식 패키지	한 줄 요약
샌드박스	어디서 실행하는지, 어떤 파일을 건드릴 수 있는지	sandbox/sandbox, sandbox/sandbox-local, sandbox/sandbox-policy	명령 실행의 파일 효과를 격리
권한	무엇을 할 수 있는지	interaction/permission-presets	샌드박스 + 승인을 묶어 이름이 있는 프리셋으로
승인	위험한 동작에 사람의 확인이 필요한지	interaction/user-approval	최소 권한의 마지막 관문

최소 권한 원칙이 세 층 공통의 설계 방향입니다: 기본 프리셋은 "작업공간 쓰기 + 핵심 동작은 질문"만 부여합니다; 어떤 상승이든(danger-full-access로 전환, 승인 정책 변경) 명시적으로 발생하며 세션 로그에 기록됩니다 — permission/preset 이벤트는 **순수 로그용 사용자 의도**로, 모델 트랜스크립트에는 들어가지 않으며, 사후에 "누가 언제 권한을 올렸는지" 추적하기 편합니다.

위협 모델 요약: 3층 방어선이 막는 것은 "신뢰할 수 없는 입력이 에이전트를 경계 밖으로 몰아가는 것"입니다 — 프롬프트 인젝션, 악성 플러그인, 유도된 고권한 작업(#381 iframe 하이재킹이 바로 이 체인을 노림); "신뢰하는 코드 자체"를 막겠다고 약속하지는 않으며(플러그인 자체가 실행 가능한 코드입니다, 13.5 참고), 네트워크/프로세스 레벨 공격을 막겠다고 약속하지도 않습니다.

핵심 인자: dsh의 도구 실행에는 기본적으로 격리와 승인 레이어가 있으며 맨몸 실행이 아닙니다. 하지만 이것은 "안전한 운영체제"가 아닙니다 — 샌드박스는 네트워크와 프로세스를 막지 못하고, 승인은 모델이 스스로 답할 수 있습니다(13.6 절).

13.2 샌드박스 메커니즘 원리: fs 경계와 도구 샌드박스

공식 샌드박스 문서가 명확히 정의합니다: 샌드박스는 파일 효과(filesystem effects)만 관리하며, 네트워크와 프로세스 가시성은 그 어휘 목록에 없습니다.

3가지 모드(SandboxMode):

모드	허용되는 파일 효과	설명
read-only	읽기 전용 + 필수 sink(POSIX에서는 /dev/null 등)	Windows ACL 백엔드는 명시적으로 쓸 수 있는 루트가 없어 partial로 보고
workspace-write	작업공간 루트 + 백엔드가 보장하는 임시 영역 쓰기 가능	기본 프리셋; 루트는 세션의 불변 cwd에서 옴
danger-full-access	제한 없음	원본 argv를 직접 spawn, ctx.sandbox를 거치지 않음

실행 세부사항:

- 정책은 호출마다 전달됨(per-call): SandboxExecutionPolicy(mode + workspaceRoot + sessionId)는 매 능력 호출 시 해석되며, provider에 고정되지 않습니다 — 같은 provider가 동시에 read-only의 bash와 쓸 수 있는 상태 디렉터리가 필요한 서브에이전트를 함께 처리할 수 있습니다
- 작업공간 루트 정규화: 먼저 파일 시스템 시맨틱에 따라 canonicalize(symlink/..를 실제 디렉터리로 해석)한 후 어휘적으로 정규화 — 심볼릭 링크로 경계를 우회하는 것을 방지
- 백엔드 매트릭스: Linux bwrap/Landlock, macOS Seatbelt, Windows ACL restricted-token; enforcement: full | partial — 오래된 Landlock ABI와 Windows ACL의 Everyone/hard-link 경계가 현재 partial 사례입니다(보장이 할인될 때는, 소비자가 partial로 처리해야 합니다)
- fs 도구도 같은 경계: write/edit 등 파일 도구도 마찬가지로 workspace-write 제약을 받습니다, "파일 도구는 그냥 통과하고 bash만 샌드박스"가 아닙니다(#149의 작업공간 재귀 삭제가 바로 workspace-write 아래에서 발생)
- 쓰기만 제한하고 읽기는 제한하지 않음: 현재 권한 모델이 제약하는 것은 쓰기이며, 작업공간 밖의 내용도 읽힐 수 있습니다(#492가 평가 격리 모드를 제안하며 지적인 현재 상태)

도구 샌드박스: bash/pwsh는 bash-sandbox / pwsh-sandbox를 거쳐(ctx.sandbox를 소비) 샌드박스 안에서 자식 프로세스를 시작합니다; Windows에서 pwsh 샌드박스는 ACL 제약이 있습니다(제8장 실측에서 temp 권한 문제를 겪은 적 있음; #758은 Windows 샌드박스 임시 디렉터리 정리 후 영구 크래시도 보고).

같은 계열의 사례: #159의 경쟁 상태는 TOCTOU에 속합니다(검사와 사용 사이에 경로가 바뀜); #278은 또 다른 확장 수법입니다 — /tmp를 workspace로 쓸 때, 제한된 자식 프로세스가 rebind로 workspace-write 권한 범위를 넓힐 수 있습니다. 두 사례의 공통된 교훈: **경계의 루트 디렉터리는 신뢰할 수 있는 위치를 선택해야 하고, 단일 경로 검사만 믿으면 안 됩니다.**

13.3 권한 모델: 도구 권한 등급 표

공식 권한 프리셋은 두 개의 다이얼 — sandbox/mode와 approval/policy — 을 묶어 이름이 있는 프리셋으로 만들고, 클라이언트는 이를 하나의 "Permissions" 선택기로 보여줍니다. 기본 표 두 개:

프리셋	sandbox/mode	approval/policy	의미
workspace-write	workspace-write	ask	작업공간 내 쓰기 가능, 동작 전 질문(기본값)
danger-full-access	danger-full-access	never	샌드박스 없음, 질문 없음
custom(파생 상태)	임의 조합	임의	다이얼을 수동으로 조정한 "비프리셋" 상태, 전환 대상이 될 수 없음

도구 권한 등급(피해 반경 순으로 정렬; 각 도구의 "권장 프리셋"은 공식 문서에서 유추한 것이며, 도구별 실측이 아닙니다 — **추론, 미실측**):

등급	대표 도구	권장 프리셋	리스크	비고
읽기 전용 탐색	read/grep/glob	read-only 호환	낮음	읽기만 하고 저장하지 않음; 읽기는 작업공간에 국한되지 않음
작업공간 쓰기	write/edit/str_replace_editor	workspace-write	중간	쓰기 경계 = 작업공간 루트 + 임시 영역
명령 실행	bash/pwsh	workspace-write(샌드박스 내)	높음	샌드박스는 네트워크와 프로세스를 막지 못함
전체 접근	임의(danger-full-access로 전환)	danger-full-access	매우 높음	모든 관문을 닫음

공식 문서는 샌드박스가 **파일만 관리한다**고 명시합니다 — 그래서 "명령 실행" 등급의 고위험은 샌드박스로 없앨 수 없고, 승인과 신뢰 경계로 뒷받침해야 합니다(13.5 / 13.7절).

실제 경계 사례(자세한 내용은 13.6절 참고): workspace-write에서 작업공간 전체를 확인 없이 재귀 삭제(#149); danger-full-access에서 홈 디렉터리 전체를 실수로 삭제(#461, 실제 사고); Windows에서 minimal preset이 작업공간 밖 쓰기를 승인 없이 허용(#523); 샌드박스 안의 agent가 taskkill로 호스트를 죽일 수 있음(#466, 프로세스 가시성이 샌드박스 어휘 목록 밖에 있음).

13.4 승인 흐름: 요청에서 허가까지

공식 승인 문서: 승인은 하나의 질문에 답합니다 — "이 구체적인 동작을 해도 되는가?"

결과 집합은 fail-closed(ApprovalOutcome):

결과	의미	호출자에 대한 효과
allowed-once	일회성 허가(질문받은 이 동작 하나만 허가)	✅ 계속
rejected	명시적 거부	❌ 중단
cancelled	요청이 철회됨(AbortSignal)	❌ 중단
unavailable	응답자 없음 / 응답자 이상	❌ 중단(기본 fail-closed)

정책(ApprovalPolicy): ask(기본값, 응답자 체인에 맡김; 체인이 비어있으면 → unavailable); never(결정론적으로 전부 rejected, 어떤 응답자에게도 발송하지 않음 — headless/CI의 엄격한 자세).

흐름: 도구 호출 → approval/request waterfall(플러그인이 가로채기/재작성 가능) → 응답자(Web UI에서 사람이 직접 응답, 또는 ACP 브리지의 일회성 기계 결정) → 매 요청마다 감사 이벤트 approval/asked + approval/decided를 쌍으로 기록(같은 ApprovalRequestId).

요청 내용(ApprovalRequest): agent(응답자는 자신이 소유한 agent만 응답) + toolName + callId(이미 스트리밍으로 표시된 도구 호출과 연결, 파라미터를 중복 렌더링하지 않음) + reason(왜 물어보는지) + signal(AbortSignal 철회).

세 가지 알려진 트러블:

1. **승인 순환 참조:** #250이 "모델이 danger-full-access를 스스로 승인"하는 실제 재현을 보고했습니다(Web approval 채널이 모델 자신에 의해 구동될 수 있음) — **승인은 보안 경계가 아니라, 사람-기계 협업 관문일 뿐입니다**
2. **동시성 오작동:** #453은 동시 샌드박스 승인에서 Allow를 클릭하면 다른 호출이 중단됨을 보고했습니다(UI 응답과 callId 불일치)
3. **재연결 시 승인 유실:** #646은 재연결 시 pending approval이 조용히 유실됨을 보고했습니다(resync 초기화 + replay 경쟁 상태); headless에서 승인 동작은 정의되지 않음(#291)

13.5 플러그인 보안 감사 체크리스트

플러그인은 dsh의 능력 경계입니다("모든 것이 플러그인", 제4장 참고) — **플러그인을 설치하는 것 = 실행 가능한 코드를 설치하는 것입니다.** 커뮤니티 감사(#817 및 같은 계열)가 정리한 체크리스트:

#	점검 항목	커뮤니티 근거	대처
1	플러그인 출처가 신뢰할 수 있는가? dsh plugin add는 서명/출처 검증이 없음	#587	공식/알려진 소스만 설치; 설치 전 소스 코드 확인
2	boot 단계 권한이 최소화되어 있는가? 플러그인이 boot 단계에서 전체 설정 트리 쓰기 권한을 가짐	#587	"설치하면 자동으로 설정을 바꾸는" 플러그인 경계
3	vm을 보안 경계로 삼지 않는가? workflow/동적 플러그인의 vm이 탈출 가능함	#243 #451 #774 #778	vm은 격리 엔진으로만 취급, 보안 레이어로 취급하지 말 것
4	자식 프로세스 환경이 올바르게 scrub 되는가? scrubbedParentEnv가 부분 문자열로 정상 변수를 오탐	#584	KEYBOARD/MONKEY 등 KEY가 포함된 변수가 잘못 scrub되지 않았는지 확인(오탐 메커니즘은 제목 기준 추론)
5	비밀값 마스킹이 fail-open이 아닌가? settings role('secret')에 마스킹 공백 존재	#226	비밀값을 프롬프트에 쓰지 말 것; 마스킹 로직 검증
6	보안 hook이 조용히 실패하지 않는가? timeout: 0이 fail-open을 유발함	#460 #583	hook 타임아웃을 양수로 설정; 로드 실패 시 오류를 던져야 함
7	감사 흔적이 남는가?	공식 approval.md	핵심 작업을 세션별로 추적

	approval/asked/decided가 쌍으로 조회 가능함		가능하게
8	세션 평문이 유출되지 않는가? 크래시 후 .tmp 평문 세션이 남음	#674	민감한 환경에서는 크래시 덤프 비활성화 / 정기적으로 정리

13.6 커뮤니티가 확인한 보안 사례

아래 4개 게시물은 모두 gh api graphql로 존재 여부를 검증했습니다(deepseek-ai/deepseek-harness Discussions, 2026-08-14):

스레드	한 줄 요약	유형	영향
#159 (https://github.com/deepseek-ai/deepseek-harness/discussions/159)	fs-sandbox의 post-check pathname race로 workspace-write 파일 경계를 우회 가능	샌드박스 경쟁 상태	TOCTOU: 검사와 사용 사이에 경로가 바뀌어 경계를 넘어 쓸 수 있음
#584 (https://github.com/deepseek-ai/deepseek-harness/discussions/584)	scrubbedParentEnv가 부분 문자열로 KEYBOARD/MONKEY 등 정상 환경 변수를 오탐(cherry-pick 가능한 수정 첨부)	자식 프로세스 환경	정상 환경 변수가 scrub되어 동작 이상
#381 (https://github.com/deepseek-ai/deepseek-harness/discussions/381)	기본 localhost Web이 크로스사이트 iframe 클릭재킹에 노출되어 Full access를 유도 승인시키고 고권한 작업을 구동시킬 수 있음	프론트엔드 클릭재킹	악성 웹페이지가 사용자를 유도해 dsh UI에서 고권한을 클릭하게 함
#817 (https://github.com/deepseek-ai/deepseek-harness/discussions/817)	보안 감사 리포트: 샌드박스/승인 경계 우회와 로컬 RPC 무인증(PoC 비공개 제공 가능)	체계적 감사	vm 탈출(같은 계열 #243 #778), 로컬 /api 무인증(같은 계열 #451, CVSS 8.8), approval 순환 참조(#250)를 포함

같은 계열의 고가치 보충 자료(스레드 번호는 저장소 내 docs/research/discussion-mining.md §2.6

에서 가져옴, 해당 리포트는 전체 780개 게시물을 프로그램적으로 검증했다고 밝힘):

스레드	한 줄 요약	시사점
#149	workspace-write에서 작업공간 전체를 확인 없이 재귀 삭제 가능	작업공간 내용도 "자기 파괴"를 방지해야 함
#250	Web approval 순환 참조: 모델이 danger-full-access를 스스로 승인	승인은 보안 경계가 아님
#461	Full Access 모드가 홈 디렉터리 전체를	고권한 프리셋의 리스크가 실증됨

	실수로 삭제(실제 사고)	
#587	서드파티 플러그인이 boot 단계에서 전체 설정 트리 쓰기 권한을 가짐	플러그인 신뢰 = 공급망 보안
#466	샌드박스 안의 agent가 taskkill로 호스트를 죽일 수 있음	프로세스 가시성이 샌드박스 어휘 밖에 있음
#674	크래시 후 .tmp 평문 세션이 정리되지 않고 남음	프라이버시 리스크

면책: 위 내용은 rc.6 시대(2026-08-13 출시 후 48시간 이내)의 커뮤니티 보고이며, 일부는 이미 새 rc에서 수정되었을 수 있습니다; 인용 시 버전 맥락에 유의하세요.

13.7 엔터프라이즈 보안 기준선

"프로덕션에 올리려는 / 팀에 보급하려는" 의사결정을 위한 제안입니다(제6장 스타일에 맞춤):

항목	기준선	근거
네트워크 노출	loopback을 유지(--host 0.0.0.0을 공식이 거부하는 것은 의도적임); 원격 인증이 완비되기 전에는 우회하지 말 것	#76 #130 #397
권한 프리셋	기본값 workspace-write + ask; danger-full-access는 격리된 머신 / 일회성 작업에만 사용	공식 preset 표 + #461
승인 정책	사람이 직접 응답하는 경로가 필수; headless는 never(결정론적 거부)를 사용하고 사전 리뷰 관문을 앞에 둘 것	approval.md + #291
플러그인 거버넌스	화이트리스트 소스 + 설치 전 13.5절 체크리스트 통과; 맨몸 dsh plugin add github: 금지	#587 #656
작업공간 경계	민감한 디렉터리(홈 디렉터리/비밀값 디렉터리)는 workspace에 넣지 말 것; 작업공간 내용은 정기적으로 버전 관리에 커밋	#149 #461
감사	세션 로그 보관(approval 감사로 추적 가능하게); .tmp 평문 잔존 방지	approval.md + #674
업그레이드 규율	rc 단계에서는 업그레이드마다 13.5절 체크리스트를 다시 실행(샌드박스 백엔드/프리셋 표가 바뀔 수 있음)	제12장 rc 버전 솔직 리뷰

의사결정 요약: 로컬 개인 개발 → 기본 프리셋이면 충분; 팀 공유 → workspace-write + ask + 플러그인 화이트리스트; CI/무인 운영 → headless + never + 사전 코드 리뷰; 어떤 시나리오든 편하자고 danger-full-access를 장기간 걸어두지 마세요.

시나리오 → 추천 조합(의사결정 제안, 위 표 기준선에서 유도):

시나리오	추천 조합	이유
로컬 개인 개발	기본값 workspace-write + ask	바로 사용 가능, 승인이 번거롭지 않음
팀 공유 호스트	workspace-write + ask + 플러그인 화이트리스트	다중 사용자 공유, 플러그인 공급망 리스크가 증폭됨
CI / 무인 운영	headless + never + 사전 리뷰 관문	결정론적 거부, 사람 없이 멈추지 않음
격리 머신 / 일회성 작업	danger-full-access(머신 자체가 격리됨)	권한은 최대화하되 공격 표면을 단일 머신으로 제한

실습 문제(진짜로 이해했는지 검증)

- 이해 문제:** 샌드박스 3가지 모드는 각각 어떤 파일 효과를 허용하나요? 왜 "네트워크와 프로세스는 샌드박스 어휘 목록에 없다"고 하나요?

정답 확인: 본 장 13.2절 3가지 모드 표와 "백엔드 매트릭스" 단락 참고

- 이해 문제:** 왜 headless에서의 never가 "가장 관대함"이 아니라 "가장 엄격함"인가요?

정답 확인: 본 장 13.4절 "정책" 단락(결정론적 rejected) 참고

- 실습 문제:** dsh web을 열어 Permissions 선택기의 두 프리셋을 관찰하고; danger-full-access로 전환했다가 다시 되돌려, 세션 로그의 permission/preset 이벤트를 확인하세요

정답 확인: 본 장 13.3절 프리셋 표 참고

- 실습 문제:** 커뮤니티 플러그인을 설치하기 전에 13.5절 체크리스트를 하나씩 확인해보고(출처/소스코드/권한/감사), 결론을 노트에 적어두세요

정답 확인: 본 장 13.5절 체크리스트 표 참고

- 사고 문제:** 왜 #250의 "모델 자체 승인"이 승인은 보안 경계가 아니라는 것을 보여주나요? 승인으로 막을 수 없다면, 기업은 무엇으로 뒷받침해야 할까요?

정답 확인: 본 장 13.6절 + 13.7절 기준선 표 참고

자주 묻는 질문 FAQ

Q1: 샌드박스가 악성 코드를 막아줄 수 있나요?

보안 경계로 쓸 수는 없습니다. 샌드박스는 파일 효과만 관리하며; 네트워크(예: SSRF)와 프로세스(예: #466 taskkill)는 그 어휘 목록 밖에 있습니다. 악성 코드를 막는 것은 플러그인 신뢰(13.5절)와 네트워크/프로세스 레벨 격리(컨테이너/VM/원격 샌드박스 — 공식 문서는 이것들이 능력 공백의 형제 구현이지 `ctx.sandbox`의 provider가 아니라고 명시함)에 의존합니다.

Q2: 왜 headless에서 never를 쓰는 것이 오히려 더 안전한가요?

never는 "절대 묻지 않는" 관대한 모드가 아니라 **결정론적 거부**입니다: 어떤 승인 요청이든 바로 rejected가 되며, 응답자에게 발송되지 않습니다 — 무인 운영 시 "사람이 Allow를 누르길 기다리며 멈춰 있는" 상황이 생기지 않습니다. 허가가 필요한 동작은 사전 리뷰나 명시적 설정에 의존해야지, 런타임에 물어보는 것에 의존하면 안 됩니다.

Q3: workspace-write면 안전한 건가요?

경계 안에서는 상대적으로 안전하지만, #149가 보여주듯 "작업공간 전체를 재귀 삭제"하는 것은 workspace-write 안에서 확인 없이 일어납니다 — 샌드박스가 막는 것은 "경계 넘기"이지 "경계 안에서의 자기 파괴"가 아닙니다. 중요한 코드는 버전 관리에 넣으세요.

Q4: 제12장에서 이미 #159를 언급했는데, 왜 이 장에서 또 다루나요?

제12장은 "알려진 한계 빠른 참조표"(한 줄 요약 + 상태)이고, 본 장은 메커니즘(post-check 경쟁 상태가 어떻게 발생하는지), 같은 계열(#278 /tmp rebind), 완화책(단일 경로 검사를 신뢰하지 말 것, 핵심 디렉터리는 workspace에 넣지 말 것)을 자세히 다룹니다.

Q5: 이 보안 게시물 번호들이 신뢰할 수 있나요? 지어낸 건 아닌가요?

핵심 게시물 4개(#159 #584 #381 #817)는 모두 gh api graphql로 하나씩 존재를 검증했습니다(링크는 13.6절 표 참고); 같은 계열의 보충 게시물은 저장소 내 docs/research/discussion-mining.md(전체 780개 게시물을 페이지 단위로 수집해 프로그램적으로 검증)에서 가져왔습니다.

제 14 장: 캐시와 비용 엔지니어링

본 장의 목표: "저렴함"을 감이 아니라 엔지니어링으로 만듭니다 — 대화 캐시 메커니즘, 적중률 실측과 최적화, 비용 모델, 추론 강도 연계, 실제 작업 예산, 그리고 매 토큰이 어디에 쓰이는지 "보는" 방법.

TL;DR(본 장 핵심, 30초 버전)

1. 캐시가 비용의 첫 번째 변수입니다: DeepSeek context cache는 반복되는 입력에 **96.8% 할인(Flash)** / **96.7% 할인(Pro)**을 적용합니다(오프피크 시간대, 2026-08-16부터 피크/오프피크 요금제 시작) — Agent 워크로드는 입력 비중이 높아서, 적중률이 비용 규모를 직접 결정합니다
2. 피크/오프피크 요금제가 가장 큰 새로운 레버입니다: 피크 시간은 UTC 01:00-04:00 + 06:00-10:00(한국시간 오전 10:00-13:00 + 15:00-19:00, 거의 평일 근무시간 전체를 덮음) — 일상적인 코딩 시간은 거의 전부 2배 가격 구간에 들어갑니다; 긴 작업(배치 리팩터링/테스트/agent 긴 작업)을 저녁이나 아침 일찍으로 옮기면 청구서가 바로 절반이 됩니다 — 적중률과 곱셈으로 누적되며, 설정을 바꿀 필요도 없습니다(14.1)
3. 핸드북 실측 적중률 97%: 세션 연속성 + 안정적인 도구 스키마 + Agent 다단계 워크로드 특성(제5장 5.1.1); 커뮤니티의 긴 실행 실측은 99.7%까지 달성(#560
(<https://github.com/deepseek-ai/deepseek-harness/discussions/560>))
4. 적중률 3원칙: 세션 연속성(자주 새로 만들지 않기), 접두사 안정성(설정을 자주 바꾸지 않기), 끊김 최소화(검수 기준을 한 번에 명확히 쓰기)
5. 두 개의 독립된 레버: 추론 강도는 "사고 토큰 양"을 관리하고, 캐시는 "입력 단가"를 관리하며, 서로 누적됩니다 — low 등급 + 고적중 = 가장 저렴한 조합
6. 비용은 눈에 보여야 합니다: 세션 통계 행에서 캐시 적중 %를 확인; 라운드별 토큰 표시는 공식이 아직 구현하지 않았으며(#735 (<https://github.com/deepseek-ai/deepseek-harness/discussions/735>)), 우선 통계 행 + 커뮤니티 cost-tracker 플러그인을 쓰거나, 설치 없이 바로 session.jsonl.zstd를 읽는 방법이 있습니다(14.7)

<details><summary>본 장 목차</summary>

- 14.1 대화 캐시 메커니즘 원리
- 14.2 적중률 실측: 97%는 어떻게 나왔나
- 14.3 적중률 최적화 실천

- 14.4 비용 모델: 적중률이 청구서를 어떻게 결정하는가
- 14.5 추론 강도 연계: low/high/max 비용 매트릭스
- 14.6 예산 실전: 사례 A 비용 추정
- 14.7 측정 방법: 토큰 소비를 보이게 만들기
- 14.8 비용 관련 트러블슈팅 목록
- 14.9 의사결정 제안 빠른 참조

14.1 대화 캐시 메커니즘 원리

DeepSeek API의 **context cache**(프롬프트 캐시): 반복/유사한 입력 토큰은 전체 가격이 아니라 **캐시 가격**으로 과금됩니다. dsh의 매 모델 요청의 입력 = 시스템 프롬프트 + 스킬 카탈로그 + 대화 이력 + 도구 스키마(제8장 8.3), **그중 변하지 않는 접두사 부분이 바로 캐시의 자연스러운 후보입니다** — 이것이 Agent 워크로드의 캐시 잠재력이 일반 채팅보다 훨씬 높은 이유입니다.

과금 규칙(2026-08-16 16:00 UTC부터 피크/오프피크 요금제 시행, 공식 가격 페이지 (https://api-docs.deepseek.com/quick_start/pricing); 제5장 5.1.1은 여전히 피크/오프피크 요금제 이전의 옛 통일 가격표이며, 기준은 본 장을 따릅니다):

모델	캐시 적중	미적중	출력
v4-flash 오프피크	\$0.007	\$0.22	\$0.66
v4-flash 피크	\$0.014	\$0.44	\$1.32
v4-pro 오프피크	\$0.022	\$0.66	\$1.98
v4-pro 피크	\$0.044	\$1.32	\$3.96

(단위: 미국 달러/백만 토큰)

할인율 변화(구 → 신) — 14.3의 3원칙 방향은 여전히 성립하지만, "캐시가 입력 단가를 관리한다"는 레버는 짧아졌습니다:

	구 요금표(통일 가격)	신 요금표(오프피크)
Flash 적중 할인율	98%	96.8%
Pro 적중 할인율	99.2%	96.7%
Flash 미적중/적중 배율	50배	31배

끝나거나 모델 출력이 끝나는 등의 지점에서 영속화됩니다, 다음 요청은 "완전히 일치하는" 접두사만 적중할 수 있습니다.

핵심 추론:

- 도구 결과는 필연적으로 한 번은 miss 합니다. 도구 결과는 "직전 라운드 모델 출력" 이후에야 추가되는 것이라, 다음 요청에 처음 들어갈 때는 반드시 새로운 접미사이며 반드시 한 번은 miss 합니다 — 원문을 전달하든 압축된 미리보기를 전달하든 상관없습니다(이것은 자르는 방식과 무관합니다).

- "자르면 캐시를 피할 수 있다"는 오해입니다. 자르기가 결정론적이기만 하면(같은 입력에 같은 출력) 이후 재생되는 것은 동일한 바이트이므로, 기존 접두사는 그대로 적중합니다, miss 하는 것은 처음 나타나는 압축 결과와, 이후 능동적으로 artifact를 읽어 추가한 내용뿐입니다. 압축 도구는 DeepSeek 캐시를 우회하지 않으며, 오히려 "필연적으로 처음 miss 하"는 도구 결과를 줄여줍니다(#1052 weijiafu14 실측: 원본 도구 출력 42,299자 → 압축 영속화 7,885자, 이후 derive/replay 해시 일치).

- 대가에 대한 안내(#1052 fnsii + weijiafu14): 캐시 자체는 best-effort이며 공식은 100% 적중을 보장하지 않습니다 (weijiafu14가 공식 설명을 인용); 컨텍스트 접두사를 전혀 유지하지 않으면(예: 매 라운드 재작성/이력 초기화, 새 세션으로 전환해 접두사를 전부 버림) 캐시를 전혀 적중할 수 없어, 새 입력으로 전체 가격을 지불하는 것과 같습니다(fnsii). 한도에 거의 도달했을 때는 summary 압축을 먼저 하고 이어가는 것을 우선하세요(접두사 요약을 유지, 접두사를 전부 버리고 바로 새 세션을 여는 것보다 낫습니다(fnsii의 제안)).

14.2 적중률 실측: 97%는 어떻게 나왔나

핸드북은 실제 dsh 세션 통계에서 캐시 적중률이 97%에 달할 수 있음을 실측했습니다(제5장 5.1.1). 왜 dsh의 적중률이 특히 높은가:

이유	설명
세션 연속성	같은 세션에서 여러 라운드 대화 시, 시스템 프롬프트/스킬 카탈로그/이력이 반복됨 → 적중
안정적인 도구 스키마	매 단계 요청마다 동일한 도구 정의를 전달함 → 적중
Agent 워크로드 특성	다단계 도구 체인이 반복적으로 동일한 컨텍스트를 전달함 → 적중률이 자연스럽게 높음

경계(솔직한 버전): 97%는 "대화 패턴 반복"에 의존합니다 — 완전히 새로운 작업/콜드 스타트 시나리오에서는 눈에 띄게 낮아집니다(제12장 12.5). 커뮤니티의 긴 실행 실측은 99.7%까지 달성합니다(#560

(<https://github.com/deepseek-ai/deepseek-harness/discussions/560>)), "세션이 길수록 적중률이 높아진다"는 것을 뒷받침합니다.

실측 방법: Web UI 하단 세션 통계 행에서 "캐시 적중 %"를 확인(제6장 6.6); 비교 기준은 5.1.1의 "50단계 도구 체인 작업, 240만 입력 토큰"입니다 — 이는 입력 규모의 전형적인 표본입니다.

기준 주의사항(#1234 커뮤니티 보충): Harness 인터페이스에 표시되는 "캐시 적중 %"는 대화 내 기준입니다 — 현재 세션의 요청만 통계에 잡습니다. 같은 시간대에 여러 대화를 열었다면, 다른 대화의 적중률이 낮을 때 다중 대화 전체/백엔드 기준이 낮아집니다(커뮤니티 실측 대조: 백엔드/전체 약 94.5% vs 인터페이스(대화 내) 98%, #1234 (<https://github.com/deepseek-ai/deepseek-harness/discussions/1234>)). 적중률을 측정할 때는 먼저 기준을 명확히 하세요: 대화 내 (인터페이스 통계 행)인지 다중 대화 전체(백엔드/API 측)인지; 적중률 비교는 같은 기준 안에서만 의미가 있습니다.

기준 주의사항 2(픽크/오프픽크 요금제, #45 작성자 실측 경험): 2026-08-16부터 픽크/오프픽크 요금제가 시행된 후, 픽크/오프픽크 경계를 넘나드는 세션에서는, 같은 접두사가 경계 전후로 다른 단가로 과금됩니다 — 비용 대조 실험을 할 때는(예: 14.4의 비용 모델, 14.2의 적중률 실측), 경계를 넘는 세션과 넘지 않는 세션은 금액으로 직접 비교할 수 없고, 토큰으로만 비교할 수 있습니다. #45 작성자가 실측에서 겪은 트러블: 두 그룹의 데이터를 재계산할 때야 시간 분포가 다르다는 것을 발견했고, 금액으로 비교하면 잘못된 결론을 내리게 됩니다.

지표	어디서 보는가	적중 시의 양상
캐시 적중 %	세션 통계 행(Web UI 하단)	높은 수준에서 안정적(긴 세션 97%+)
첫 토큰	매 라운드 종료 행	10초+ 수준에서 1초 미만으로 자릿수 단위로 떨어짐(제6장 FAQ Q4)
총 토큰	세션 개요	충분이 작음 = 적중 영역이 커지고 있음(14.1 구조)

14.3 적중률 최적화 실천

세 가지 레버를 가성비 순으로 정렬합니다:

레버	방법	원리
세션 연속성	긴 작업은 세션 연속성을 유지, 자주 새 세션을 만들지 않기; 배치 작업은 같은 세션/같은 접두사에 두기	이력 메시지가 반복됨 → 적중
안정적인 프롬프트	시스템 프롬프트/스킬 카탈로그 등 접두사를 안정적으로 유지, 설정을 자주 바꾸지 않기; 도구 스키마를 불변으로	접두사가 일관됨 → 적중
끊김 최소화	검수 기준을 한 번에 명확히 쓰기(제5장 5.7 / 제10장 사례 시사점), "세션을 다시 열어 컨텍스트를 다시 설명"하는 것을 줄이기	접두사 재구축을 피함 → 전체 가격 재시작 방지

실행 점검: 적중률이 예상보다 낮을 때, 첫 반응은 "접두사가 바뀌었는지" 확인하는 것입니다 — 설정을 바꿨거나, 강도를 바꿨거나, 스킬 카탈로그를 건드렸으면 모두 캐시를 무효화시킵니다(제6장 6.6 모니터링 행).

픽/오프픽은 네 번째이자 가장 큰 레버입니다: 긴 작업(배치 리팩터링, 테스트 실행, agent 긴 작업)을 오프픽으로 옮기면(한국시간 저녁이나 아침 8시 이전), 청구서가 바로 **절반**이 됩니다 — 100%의 가격 차이로, 위 세 가지보다 폭이 크며, 적중률과 **곱셈** 관계입니다(오프픽은 적중/미적중/출력 세 등급을 함께 절반으로 낮춤, 14.1), 3원칙과 누적 가능하며 어떤 설정 변경도 필요 없습니다.

14.4 비용 모델: 적중률이 청구서를 어떻게 결정하는가

비용 모델 요약(제6장 6.6): 총 비용 $\approx$ 출력 토큰 $\times$ 출력 가격 + 입력 미적중 $\times$ 미적중 가격 + 입력 적중 $\times$ 적중 가격 — Agent 워크로드는 입력 비중이 높아서, **캐시 적중률이 비용의 첫 번째 변수**입니다.

14.1의 오프픽 Flash 입력 가격으로 계산해봅시다(**입력 1만 토큰**, 적중률 비교). 산술 예시(90% 적중): $9,000 \times \$0.007/M + 1,000 \times \$0.22/M \approx \$0.000283$ — 적중/미적중을 먼저 나눈 후, 각각의 단가를 곱합니다:

적중률	적중 토큰	미적중 토큰	입력 비용(Flash 오프픽)	상대 비용
90%	9,000	1,000	$\approx \$0.000283$	1×
50%	5,000	5,000	$\approx \$0.001135$	$\approx 4.0\times$
10%	1,000	9,000	$\approx \$0.001987$	$\approx 7.0\times$

100만 입력 토큰으로 확대하면: 90% 적중 $\approx$ **\$0.0283**, 10% 적중 $\approx$ **\$0.199**, 전체 가격 \$0.22(오프픽 미적중) — 거의 자릿수 하나 차이입니다.

핸드북 실측 규모로 다시 확대하면(5.1.1: 50단계 도구 체인, 240만 입력 토큰): 97% 적중 $\approx$ **\$0.032**, 전체 가격 $\approx$ **\$0.53**, 약 **16배**. 이것이 5.1.1의 "비용이 미적중 가격 기준 예상치보다 훨씬 낮다"는 수치의 출처입니다(참고: 구 요금표에서는 이 배율이 약 20배였는데, 신 요금표는 적중 할인이 줄어들어 16배로 내려갔습니다, 14.1 참고).

참고: 이 표는 입력 쪽만 계산한 것입니다 — 출력 쪽은 14.1의 가격표 참고(출력 = 미적중의 3배이며, 캐시 할인을 받지 못함); 출력 비중이 큰 작업에서는 캐시 최적화의 이득이 입력 비중이 큰 작업보다 작을 것입니다(14.1 할인율 변화).

접두사 슬리밍 실측#45, 데이터와 스크립트는 sjh9714/dsh-lean (<https://github.com/sjh9714/dsh-lean>) **에 있음**: 32회 짝지어 실행, 매번 작업의 깨끗한 사본에서 시작해 양쪽 모두 작업에 내장된 테스트 스위트로 산출물을 검증했습니다. 기본 standard preset의 접두사는 **8,246개의 미적중 토큰**이며, 그중 약 **3,700개**는 단일 에이전트 세션이 전혀 호출하지 않는 도구(delegation 그룹, goal, jobs)에서 나옵니다 — **현행 피크/오프피크 요금표 기준, 이 한 번의 접두사가 전체 청구서의 46%를 차지합니다**(6개 요청 작업, 3회 실행 평균, 구 통일 요금표 기준으로는 52%). 추론: "**접두사 슬리밍**" — 전혀 쓰지 않는 도구를 덜 걸어두는 것 — **이 고가치 최적화 방향입니다**, 원본 토큰 수는 저장소에 이미 제출되어 있어 신 요금표로 정확히 재계산할 수 있습니다.

14.5 추론 강도 연계: low/high/max 비용 매트릭스

추론 강도는 "사고량"을 관리하며, 캐시와는 **두 개의 직교하는 레버**입니다: 강도는 토큰 총량에, 캐시는 입력 단기에 영향을 주며, 둘은 곱해집니다. 사고 토큰 감소 → 총 토큰 하락(제6장 6.6), 그리고 컨텍스트가 반복되는 부분에 따라 마찬가지로 캐시 할인을 받습니다(추론, 미실측).

강도	사고 토큰 양	상대 비용	적용 시나리오(제6장 6.2)	비용 전략
low	최소	최저	단순/결정론적 라운드: 파일 조작, 배치, 도구 체인 중 저렴한 단계	배치 작업의 기본 강도
high	중간	중간	일상적인 Agent 작업 (기본값)	품질과 비용의 기본 균형
max	최대	최고	복잡한 추론, 긴 체인 계획, 디버그	단발성 복잡한 작업에는 가능, 배치 루프에는 열지 말 것

연계 포인트:

- **긴 도구 체인 작업(20-50단계)에서 이득이 가장 큼**: 매 단계 사고 강도 다운그레이드의 누적 효과가 뚜렷함 (제6장 FAQ Q2) — 강도 × 적중률 두 레버가 매 단계마다 동시에 작용
- 단순 라운드에 low를 쓰면 품질 차이가 거의 없음; 복잡한 추론에 low를 쓰면 핵심 단계를 놓칠 수 있음 (제6장 FAQ Q3)
- **누적 예시**(240만 입력 규모 작업, Flash 오프피크 가격): low + 97% 적중 ≈ \$0.032 이하(사고량이 더 낮아 실제로는 더 적음, 추론, 미실측); max + 반복적인 콜드 스타트 ≈ \$0.53 이상 — **같은 작업인데, 두 레버를 전부 켜었을 때와 전부 켜지 않았을 때 16배+ 차이**

- 참고: low/high/max는 본 핸드북이 실측한 게이트웨이(pi-ai/opencode-go) 강도입니다; 기본 deepseek-official 어댑터는 off/high/max입니다(제6장 6.2절 참고)

14.6 예산 실전: 사례 A 비용 추정

제10장 사례 A(데이터 품질 분석, 186초)를 표본으로 예산을 세워봅니다: 도구 체인 read →

write(clean.py) → bash → write(visualize.py) → bash → read → 요약, 약 6-8단계.

5.1.1 규모로 환산하면(50단계 ≈ 240만 입력 토큰), 사례 A의 입력 규모는 약 **50만 토큰(추론, 미실측)**입니다:

시나리오	적중률	입력 비용(Flash 오프피크, 추정)
세션 연속성 + 접두사 안정(권장 방법)	97%	≈ \$0.0067
중간에 새 세션/접두사 변경	10%	≈ \$0.099
극단: 매번 전체 가격	0%	\$0.11

단계별 분해(왜 세션 연속성이 비용을 절약하는가) — 첫 라운드만 콜드 스타트 전체 가격이고, 이후 매 라운드는 "이번 라운드 메시지 + 도구 결과"라는 작은 새 증분만 미적중입니다(추론, 미실측):

단계	내용	입력 토큰(추론, 미실측)	적중 상태
1	read(sales_data.json)	~5만	콜드 스타트(미적중)
2-6	write/bash/write/bash/ read/요약	~45만	접두사 적중 영역, 증분 극히 적음

결론: 3분짜리 실제 작업 하나의 입력 쪽 비용은 **1센트 미만**입니다; 같은 일을 하는 방식이 다르면(세션 연속성 vs 반복적인 콜드 스타트) 비용이 **약 16배** 차이 납니다. 출력 쪽(스크립트 2개 + 요약)의 규모는 입력보다 훨씬 작아, 총 비용은 여전히 센트 단위입니다(추론, 미실측). 이것이 "dsh + V4-Flash 조합의 비용이 Claude 대비 약 1/10~1/30"(제1장 실측)이라는 것의 단일 작업 차원 출처입니다.

14.7 측정 방법: 토큰 소비를 보이게 만들기

현재 상태(있는 것):

- Web UI 하단 세션 통계 행: N 라운드 · M 단계 | LLM Xs · 도구 호출 Ys | 첫 토큰 평균 ... (제6장 6.3)

• 매 라운드 종료 행: 10:14 · 소요 9분 34초 · 첫 토큰 1.7초 · 79 tok/s(#735)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/735>) 원문 스크린샷)

• 세션 개요에는 총 토큰 통계가 있음; 통계 행에서 "캐시 적중 %"를 볼 수 있음(제6장 6.6) — 대화 내 기준이며, 다중 대화 전체 적중률은 다른 저장중 대화에 의해 낮아질 수 있음(#1234)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/1234>) 커뮤니티 보충, 14.2 기준 주의사항 참고)

공백(없는 것): 매 라운드의 토큰 수. 공식 디스커션 #735

(<https://github.com/deepseek-ai/deepseek-harness/discussions/735>)(2026-08-14 제기, 제목 「【친절한 표시】매 대화 라운드에 이번 라운드의 토큰 소비량을 추가해줬으면 좋겠다.」가 바로 이 요구사항입니다 — 게시물에서 존재가 확인되었지만, 공식은 아직 구현하지 않았습니다.)

또 다른 공백(#735 댓글 Jianye): 매 라운드 표시에 **provider와 model id 식별자**를 포함해달라는 요청(원문 스크린샷의 xtoken : gpt-5.6-sol 같은 "provider + modelid" 정보를 사용자가 인터페이스에서 바로 받고 싶어함) — 같은 요구사항 게시물 아래의 보충 요청입니다.

커뮤니티 제안(#735 댓글에 이미 반영됨): 매 라운드 표시는 두 개의 숫자로 나뉘야 합니다 — ① 이번 라운드 총 토큰(대략적인 소비량); ② 이번 라운드 캐시 적중/미적중 토큰(비용 최적화 여지 확인). 총 토큰만 보면 오판할 수 있습니다: 같은 1만 토큰이라도 적중률 90%와 10%는 비용이 7.0배 차이 납니다(14.4 참고).

과도기 도구: 커뮤니티 dsh-plugin-cost-tracker(제9장 9.5/9.6, 실시간 토큰 비용 추적, 플러그인/MCP 형태) — 구현 레벨에서는 제4장의 플러그인 확장 포인트를 거쳐, 요청/세션 이벤트에서 usage를 통계 냅니다(추론, 미실측); 또는 세션 로그/API usage 필드로 수동 계산할 수도 있습니다. 제11장은 "캐시 적중률 도구화"가 중기의 주요 흐름이라고 예측합니다(11.2) — #735가 구현되면 비용 투명화 페루프가 완성됩니다.

더 완전한 커뮤니티 실측 도구: dsh-usage(작성자 kestiany, #1169)

(<https://github.com/deepseek-ai/deepseek-harness/discussions/1169>) 수록 허가 받음). npm에 0.1.0이 이미 게시되어 있습니다(dsh plugin --profile web add dsh-usage로 설치 가능), Harness가 영속화한 세션 로그를 기반으로 계산하며 별도의 통계 데이터베이스가 필요 없습니다. 핵심 능력:

- 매 대화 라운드 끝 위치에 고정 표시: Total · Input · Cache · Output · Cost — #735가 빠뜨린 "매 라운드 토큰 수"를 정확히 채워주며, 입력/캐시 적중/출력을 구분해줍니다

- **Settings → Usage 완전한 사용량 페이지:** 총 토큰, 입력, 캐시 적중, 출력, 호출 횟수 및 **예상 비용**
(Provider/모델에 가격 설정이 필요; usage/가격이 없으면 비용은 - -로 표시되고, 대화 하단의 Cost는 생략됨, 불완전한 비용을 총 비용으로 오인하는 것을 방지)

- **52주 사용량 히트맵**(GitHub Contributions 스타일) + 모델/세션/단일 라운드별 조회

- DeepSeek Harness의 다른 모델과 Provider도 지원; Cost는 추정치이며 공급자의 최종 청구서를 대표하지 않음

설치 없이: session.jsonl.zstd를 직접 읽기(#45 작성자 방안) — 어떤 플러그인도 설치할 필요 없이, dsh가 스스로 쓰는 세션 원본 데이터에 이미 모든 정보가 있습니다:

- assistant/chunk에서 chunk.type이 usage인 이벤트에는, 공급자가 반환한 inputTokens / cacheReadTokens / outputTokens / reasoningTokens가 담겨 있습니다 — **요청별**입니다(모델 요청마다 하나씩), 세션 합계가 아닙니다
- request/header 이벤트에는 실제로 전송된 완전한 도구 스키마 배열과 시스템 프롬프트가 담겨 있습니다 — 여러분 자신의 접두사 크기를 재는 데 추가 API 호출이 필요 없습니다

트러블(#45 작성자 실측): zstdDecompressSync는 첫 번째 frame을 읽으면 멈추는데, dsh는 쓸 때마다 frame을 추가합니다 — 136줄짜리 세션을 그냥 압축 해제하면 1줄만 얻게 됩니다. frame magic(28 B5 2F FD)으로 나눈 후 하나씩 압축 해제해야 합니다.

명령어 한 줄(#45 작성자 제공, 설치 없음): npx dsh-lean audit — 요청별 적중 분해(적중/미적중/출력) + 여러분의 접두사에서 가장 큰 도구 스키마 몇 개를 출력합니다(데이터 출처는 위와 동일, 14.4 접두사 슬리밍 실측 참고).

기억/압축 생태계 도구(#1052 긴 세션 비용 절감 특집 댓글, 2026-08-14) — "긴 세션 토큰 폭증"(원문 실측 7천만→1억)에 대해, 커뮤니티가 상호 보완적인 두 방안을 제시했습니다:

도구	작성자	포지셔닝	메커니즘
dsh-sgme (freehul/sgme (https://github.com/freehul/sgme), npm 패키지 dsh-sgme)	freehul	기억 엔진(근본 치료)	"대화 이력"과 "장기 기억"을 계층화: 매 라운드 무

			비용으로 디스크에 기록(L0, LLM을 거치지 않음) → 세션 종료 시 추출·중복 제거·병합 (L1/L1.5/L2, 추출 전에 먼저 가지치기로 쓸모없는 도구 호출/중간 산출물을 제거, 실측 세션 내용 65~96% 절감) → 새 세션은 시나리오별로 관련된 기억 블록만 주입(순수 구조화 쿼리, 토큰을 태우지 않음) → 세부사항이 필요할 때 memory_search로 필요에 따라 검색
pi-quiet-tools(pi2dsh (https://github.com/weijiafu14/pi2dsh))를 통해 장착)	weijiafu14 제공(freehul도 추천)	도구 출력 압축(대증 치료)	모델 컨텍스트에 들어가기 전에 큰 도구 결과를 압축: 기본적으로 12,000자 또는 240줄을 초과하면 결정론적인 앞뒤 미리보기만 모델에 전달하고, 완전한 결과는 로컬 artifact에 저장했다가 필요할 때 모델이 다시 읽음; 임계값은 QUIET_TOOLS_MAX_CHARS / QUIET_TOOLS_MAX_LINES로 조절 가능. MCP/Playwright의 큰 반환 값에 효과적(작성자 실측 40,799자를 압축해 페루프 완성). 캐시와 호환됨: 14.1의 엄격한 점두사 캐시 규칙 참고 — 캐시를 우회하지 않고, 오히려 "필연적으로 처음 miss하는" 도구 결과를 줄여줍니다

설치 예시(pi-quiet-tools, #1052 작성자 실측에서 발췌):

```
npm i -g pi2dsh
pi2dsh host --packages pi-quiet-tools@0.2.0 --out ./dsh-pi-quiet
dsh plugin --profile web add file:$PWD/dsh-pi-quiet
```

둘은 충돌하지 않습니다: pi-quiet-tools는 "큰 도구 결과가 반복적으로 컨텍스트에 들어가는" 문제를 해결하고, dsh-sgme는 "긴 세션이 쓸수록 비싸지는/연속성" 문제를 해결합니다 — 전자는 즉시 효과가 있고, 후자는 세션 간 기억 이전이 필

요한 시나리오에 적합합니다(#1052 댓글의 합의: "대화 이력"과 "장기 기억"을 분리해, 이력은 쿨러가게 두고 기억은 필요에 따라 주입).

14.8 비용 관련 트러블슈팅 목록

#	트러블	증상	해법
1	캐시 적중을 "모델이 빨라졌다"고 오판	A/B 비교가 1초 vs 110초에 오도됨(제6장 트러블 #6)	비교 테스트는 완전히 새로운 prompt 를 사용
2	짖은 새 세션 생성이 비용을 낭비함	매 작업이 콜드 스타트로, 입력이 전체 가격	긴 작업/배치는 세션 연속성 유지
3	설정 변경이 조용히 적중률을 죽임	적중 %가 갑자기 떨어지고 비용이 오름	통계 행의 적중 %를 모니터링, 접두사를 안정적으로 유지
4	총 토큰만 보고 비용을 오판	1만 토큰에서 90% vs 10%가 7.0배 차이	적중/미적중을 나눠서 확인(#735 제안)
5	예산에서 출력 쪽을 빠뜨림	입력 가격으로만 계산해 금액이 안 맞음	공식 가격 페이지/14.1 가격표로 출력 가격을 보충(출력 = 미적중의 3배, 캐시 할인 없음)
6	강도를 무분별하게 사용	배치 작업에 max를 써서 비용이 두 배가 됨	배치는 low, 복잡한 작업만 max
7	픽크/오프픽크 경계를 넘나들며 금액 비교	비용 대조 실험 결론이 틀림(#45 작성자 실측 경험)	토큰만 비교하고 금액은 비교하지 말 것(경계를 넘으면 접두사 단가가 다름, 14.2 기준 주의사항 2 참고)

14.9 의사결정 제안 빠른 참조

시나리오	제안
긴 도구 체인 작업(20-50단계)	high + 세션 연속성 + 접두사 안정; 240만 입력 규모 ≈ \$0.032(97% 적중, 오프픽크)
배치/단순 라운드	low + 같은 세션에서 배치(적중 보너스가 가장 큼)
복잡한 추론/디버그	max(비용으로 품질을 사는 것), 단발성 작업은 콜드 스타트를 감수할 만함
비용에 민감한 배치 처리	low + headless + 단일 세션 순차 처리
긴 작업(배치 리팩터링/테스트/agent 긴 작업)	오프픽크 로 옮기기(한국시간 저녁 8시 이후/아침 8시 이전), 청구서가 바로 절반(적중률과 곱셈으로 누적, 14.1)
성능/비용 비교 테스트	완전히 새로운 prompt + 고정된 강도 + 여러 번 실행해 중앙값 취하기; 픽크/오프픽크 경계를 넘는 그룹은 토큰만 비교하고 금액은 비교하지 말 것(제6장 6.4/6.6, 14.2 기준 주의사항 2)

적중률이 비정상적으로 하락	먼저 접두사가 바뀌었는지 확인(설정/스킬 카탈로그/모델 강도)
----------------	------------------------------------

실습 문제(진짜로 이해했는지 검증)

1. **이해 문제:** 왜 "캐시 적중률이 비용의 첫 번째 변수"라고 하나요? 14.4의 1만 토큰 표를 사용해 90%와 10% 적중이 몇 배 차이 나는지 설명해보세요

정답 확인: 14.4 비교표(7.0배)와 6.6 비용 모델 요약 참고

2. **실습 문제:** 3단계 작업을 실행해보고, Web UI 하단의 세션 통계 행과 매 라운드 종료 행(시간/첫 토큰/tok/s)을 관찰해, 이번 작업이 대략 얼마나 캐시를 적중했는지 판단해보세요

정답 확인: 14.7 측정 방법과 6.3 통계 행 형식 참고

3. **실습 문제:** 같은 배치 작업을 "항목마다 새 세션 열기"와 "같은 세션에서 연속 실행" 두 가지 방식으로 각각 실행해, 소요 시간 차이를 비교해보세요 — 새 세션은 첫 라운드가 뚜렷하게 느려질 가능성이 큼니다(콜드 스타트)

정답 확인: 14.1 콜드 스타트 110초 vs 핫 캐시 1초 실측 참고

4. **사고 문제:** 왜 "이번 라운드의 총 토큰만 보는 것"이 비용을 오판하게 만들까요? #735 댓글은 매 라운드 표시를 어떤 두 숫자로 나눠야 한다고 제안했나요?

정답 확인: 14.7절 "커뮤니티 제안" 단락 참고

5. **실습 문제:** #735의 제안 기준에 따라, 세션 하나의 총 토큰을 "적중/미적중" 두 부분으로 수동으로 나누고, 14.4의 단가로 각각 계산해 "자릿수 하나 차이"라는 주장을 검증해보세요

정답 확인: 14.4 비교표와 14.2 측정 방법 표 참고

자주 묻는 질문 FAQ

Q1: 캐시 적중률은 최대 얼마까지 올라가나요?

핸드북 실측 97%(제5장 5.1.1); 커뮤니티 긴 실행 실측은 99.7%까지 달성했습니다(#560

(<https://github.com/deepseek-ai/deepseek-harness/discussions/560>)). 하지만 완전히 새로운 작업/콜드 스타트에서는 눈에

띄게 낮아집니다(제12장 12.5) — 적중률은 "방법"의 함수이지, 모델의 상수가 아닙니다.

Q2: low 강도는 품질을 크게 떨어뜨리나요?

단순 결정론적 작업(파일 조작, 배치 처리)은 거의 차이가 없습니다; 복잡한 추론, 긴 체인 계획, 디버그에 low를 쓰면 핵심 단계를 놓칠 수 있습니다(제6장 FAQ Q3). 일상에는 high, 배치/단순 작업은 low로 전환하는 것을 권장합니다.

Q3: 공식은 언제 매 라운드 토큰 표시를 제공하나요?

#735 (<https://github.com/deepseek-ai/deepseek-harness/discussions/735>)는 2026-08-14에 제기된 요구사항 게시물입니다(매 라운드 토큰 + provider/model 식별자, 14.7 참고), **아직 구현되지 않았습니다**. 과도기에는: 세션 통계 행의 적중 % + 커뮤니티 dsh-plugin-cost-tracker(제9장)나 dsh-usage(14.7)를 사용하세요.

Q4: 서드파티 게이트웨이/자체 구축 provider도 캐시 할인을 받나요?

캐시 할인은 DeepSeek API 측의 능력입니다(제5장 가격표); 서드파티 게이트웨이의 캐시 지원과 과금 방식은 해당 가격 페이지를 확인해야 합니다(추론, 미실측).

Q5: 왜 캐시가 적중되면 작업이 눈에 띄게 빨라지나요?

적중된 접두사 부분은 다시 계산할 필요가 없어, 첫 토큰 시간이 자릿수 단위로 단축됩니다(콜드 스타트 ~110초 vs 핫 캐시 ~1초, 제1장 실측). 판단 방법: 첫 토큰이 갑자기 10초+에서 1초 미만으로 떨어지면 적중했을 가능성이 큼니다(제6장 FAQ Q4). 성능 비교를 할 때는 완전히 새로운 prompt를 써서 캐시 간섭을 피하세요(제6장 트러블 #6).

Q6: 언제 실행하는 것이 가장 저렴한가요?

2026-08-16부터 피크/오프피크 요금제 시행(14.1): 피크 시간 UTC 01:00-04:00 + 06:00-10:00(한국시간 오전 10:00-13:00 + 오후 15:00-19:00), 오프피크 가격은 피크의 절반입니다. 국내(중국) 일상 코딩 시간은 거의 전부 피크 구간입니다; 긴 작업(배치 리팩터링/테스트/agent 긴 작업)을 저녁이나 아침 8시 이전으로 옮기면 청구서가 바로 절반이 됩니다 — 적중률과 곱셈으로 누적되며, 설정을 바꿀 필요가 없습니다.

본 장의 정보는 2026-08-16 기준입니다(dsh 0.1.0-rc.6; 피크/오프피크 요금제는 2026-08-16 16:00 UTC부터 적용). 가격과 캐시 정책은 공식 문서를 기준으로 삼으세요; "(추론, 미실측)"으로 표시된 부분은 추정이나 예상치이니, 실측 후 PR로 수정해주시면 감사하겠습니다.

제 15 장: DSH 생태계 전체 리포트 — 4000+ 플러그인과 2400개 디스커션 스레드가 말해주는 것

본 장의 목표: 커뮤니티 데이터 집계(4000+ 플러그인, 플러그인 스토어 dsh-hub.cc 등재 포함)와 본 핸드북의 2400+개 디스커션 스레드 관찰 + 225개 게시물 응답 경험을 교차 검증해, dsh 생태계의 전체 그림을 그립니다 — 어디가 폭발적으로 성장하는지, 어디에 공백이 있는지, 기회는 어디에 있는지.

데이터 출처: Blue-Whale-Harness 플러그인 전체 목록(커뮤니티 유지, 2026-08 수집 1804개 저장소) + 플러그인 스토어 dsh-hub.cc (<https://dsh-hub.cc>)(4000+ 등재, 2026-08 오픈); 디스커션 관찰은 2026-08 기준(스레드 번호 2400+).

TL;DR(30초 버전)

1. 생태계가 폭발하고 있습니다: 4000+개 플러그인(플러그인 스토어 dsh-hub.cc 등재), 2주 만에 무에서 성장; 디자인/UI 생성(348개/94K★)과 비전(132개/90K★) 수요가 가장 뜨겁습니다
- 1b. 유통의 질적 변화: 플러그인 스토어 오픈(#2423) — awesome 목록에서 검색·게시 가능한 스토어로 업그레이드, 생태계가 "야생 성장"에서 "질서 있는 유통"으로 진입
2. "공식이 안 하면 커뮤니티가 다 한다"가 주요 테마입니다: 데스크톱 웹(140+), TUI, 기억, 비전 브리지가 전부 커뮤니티가 채운 것입니다
3. 공백이 명확합니다: 금융(8개), 데이터베이스(14개), 오디오(19개), 평가(37개)는 거의 공백입니다 — 하지만 수요는 존재합니다
4. Windows와 원격은 아픈 지점입니다: 중국어 경로 계열(#107 등 15+개 스레드), 자식 프로세스(#717), LAN RPC(#755)
5. 차별화 베팅 포인트: 전체 session log + 모든 것이 플러그인 + 샌드박스/권한 체계 → 기업/평가/규제 준수 시나리오

15.1 생태계 데이터 스냅샷(4000+ 플러그인)

카테고리 분포

카테고리	플러그인 수	해석
tools	569	도구류가 가장 뜨거움(agent의 핵심 가치를 담당)
utility	345	범용 도구

session	229	세션 관리(로그/복원/통계)
orchestration	197	오케스트레이션/워크플로우
ui	176	UI 강화(사이드바/패널/컴포넌트)
llm	159	모델 어댑팅
acp	66	ACP 프로토콜
skills	51	스킬 팩
sandbox	9	샌드박스는 단 9개 — 보안 도구가 희소함(15.4절 참고)

주제별 인기도(플러그인 수 / 총 star / 판단)

주제	플러그인 수	총★	판단
디자인/UI 생성	348	94K	수요가 가장 뜨거움(agent가 비주얼을 만들어내야 하는 필수 수요)
IDE 통합	221	89K	높은 수요
비전/이미지 보기	132	90K	순수 텍스트 모델이 이미지를 봐야 하는 것은 필수 수요 (디스커션 #1153/#1269/#1378 비전 브리지 계열과 상호 입증)
데스크톱/클라이언트	140	90K	공식 데스크톱 셸이 없어 커뮤니티가 채움(Windows/macOS/Android)
콘텐츠 창작	40	88K	단일 가치가 높음(수는 적지만 star가 높음 — 비프로 그래머 시나리오의 가치가 가장 큼)
워크플로우/오케스트레이션	94	17K	중간
기억/페르소나	77	7.7K	성장 중(#1822 기억 체/#1448/#1478 등, 방향이 다양함)
코드 품질/리뷰	65	1.7K	중간
금융/거래	8	28	공백
데이터베이스	14	137	공백
멀티모달/오디오	19	26	공백
평가/테스트	37	117	공백(공식의 100% 커버리지 게이트와 대조적)

15.2 정성적 인사이트(780개 스레드 디스커션 관찰, 교차 검증 데이터)

인사이트 1: Windows가 가장 큰 페인 포인트(60+개 스레드)

데이터 측면: Windows 호환성이 가장 고빈도 주제; 디스커션 측면: **중국어 경로 절단(#107/#151/#396/#800 등 15+개 스레드가 같은 근본 원인)**, koffi 네이티브 크래시(#197/#293), 포트 예약 구간(#589), 자식 프로세스 우아한 종료 부재(#717), 샌드박스 임시 디렉터리 정리 시 크래시(#758).

교차 검증: 데이터 측면의 "Windows 호환성 부족"과 디스커션 측면의 60+개 스레드가 완전히 서로를 입증합니다 — 이것이 가장 확실한 생태계 약점입니다.

인사이트 2: "공식이 안 하면 커뮤니티가 다 한다"

데이터 측면: 데스크톱 셸 140+, TUI 여러 세트, 기억 77, 비전 132; 디스커션 측면: 2주 만에 커뮤니티가 CLI/TUI/데스크톱/memory/비전을 보완(#391/#405/#386 등).

교차 검증: 공식(0.1.0-rc 단계는 코어에 집중)과 커뮤니티(생태계 공백을 채움)가 건강한 상호 보완을 형성합니다 — 이것이 생태계 활력의 가장 강력한 신호입니다.

인사이트 3: 보안 감사가 이례적으로 활발함

데이터 측면: **sandbox** 카테고리 플러그인이 단 9개(보안 도구 희소); 디스커션 측면: 보안 감사 게시물이 밀집(#243/#250/#278/#381/#454/#774/#817 체계적 감사; vm 탈출/approval 순환 참조/clickjacking 실제 발견) + 보안 도구 체인(vetting/egress-guard/read-confine/check-dsh-profile).

교차 검증: 보안 수요는 높지만(감사가 활발함) 보안 도구는 적습니다(sandbox 플러그인 9개) — 공급 공백입니다.

인사이트 4: 직렬화/경계 버그 계열이 집중적으로 터짐

디스커션 측면: unknown tool ""(#725/#1405), reasoning 직렬화 생략(#739/#1850), 잘린 tool-call 오염(#1519), run_code 비동기 콜백 유실(#1476), 손상된 세션이 boot를 끌어내림(#1473/#1497) — **전부 llm-deepseek 직렬화 레이어/세션 상태 머신의 경계 문제입니다.**

판단: rc 시기 이 레이어의 버그가 집중되는 것 = 공식의 다음 반복 사이클에서 주요 전장이며, 커뮤니티 패치/수정 브랜치의 노다지이기도 합니다(#1519/#725 모두 테스트 수정을 포함).

인사이트 5: 비용 투명화는 숨은 필수 수요

데이터 측면: 콘텐츠 창작 40개/88K★; 디스커션 측면: token 소비/캐시 적중 토론이 밀집(#735/#1052/#1234/#1374/#1520), 비용 도구가 우후죽순처럼 등장(dsh-usage/usage-panel/Show Me How Much/balance-meter).

교차 검증: 캐시 적중 97%(실측)는 비용 우위지만 "보이지 않습니다" — 비용 도구가 "통계"에서 "엔지니어링"으로(제14장) 나아가는 것이 확실한 방향입니다.

인사이트 6: 플러그인 스토어 오픈 — 유통이 "목록"에서 "스토어"로(신규)

데이터 측면: dsh-hub.cc가 4000+개 플러그인을 등재(2026-08 오픈, 검색 가능/카테고리별 게시 가능); 디스커션 측면: #1825 "플러그인 마켓을 플러그인으로 만들자" 제안 + #2423 스토어 오픈 — 커뮤니티가 2주 만에 공식이 만들지 않은 유통 인프라를 채웠습니다(데스크톱 웹/TUI와 같은 "공식이 안 하면 커뮤니티가 다 한다"는 라인).

교차 검증: 플러그인 총량이 1804(전체 목록)에서 4000+(스토어)로 늘었습니다 — 기준이 다르지만(스토어는 일부 진짜가 아닌 플러그인도 포함), 규모 변화는 생태계가 여전히 지속적으로 성장하고 있음을 보여줍니다.

인사이트 7: 오디오/멀티모달 능력 공백이 공식적으로 제안됨(신규)

데이터 측면: 오디오류 플러그인은 여전히 희소함(19개); 디스커션 측면: #2008 실시간 음성 + #2431 ASR/TTS/avatar 능력 공백 표준 제안 — 오디오가 "커뮤니티 플러그인"에서 "표준 능력 공백"으로 넘어가고 있습니다(비전 #2024 core 구현과 같은 길).

판단: 공식이 표준을 정하고 커뮤니티가 구현을 채우는 패턴이 비전/오디오 두 모달리티에서 재현되고 있습니다 — 이것이 생태계가 "공백 채우기"에서 "표준화"로 넘어가는 전환 신호입니다.

15.3 공식이 우선적으로 채워야 할 능력 공백(데이터 + 디스커션 이중 검증)

능력 공백	데이터 근거	디스커션 근거	실행 가능성
멀티모달/비전 채널(OCR + 비전 서비스 추상화)	비전 132개/90K★	#1378/#1327/#2024에서 반복적으로 제기	높음(core 구현이 이미 등장함)
오디오 채	오디오는 19개뿐	#2008 음성 + #2431 표준	중상(표준 레이어에 이미 제

널(ASR/TTS/avatar)		제안	안한 사람이 있음)
기억/페르소나 seam(공식이 교체 가능하도록)	기억 77개/7.7K★ 성장 중	#1822 기억체 제안	높음(session이 이미 전체 기록을 가지고 있음)
데스크톱/TUI 셸 프로토콜(표준을 정해 편입)	데스크톱 140+ 파편화	여러 클라이언트가 공존	높음
평가/검증 페루프(session→리포트)	평가는 37개/117★뿐	공식의 100% 커버리지 게이트 문화	중상
Windows 1급 지원(자식 프로세스/샌드박스/경로/원격)	Windows가 가장 고빈도	#107 계열/#717/#755	중상(작업량이 큼)
플러그인 registry + 호환성 계약	4000+ 플러그인이 질서 있는 유통을 필요로 함	#1825 제안; 커뮤니티가 이미 선행: dsh-hub.cc 스토어 (#2423)	높음(공식이 편입/표준화 가능)

15.4 생태계 참여자를 위한 제안

여러분이 플러그인 개발자라면

1. **공백 우선:** 금융/데이터베이스/오디오/평가는 블루오션입니다(경쟁자가 적고 수요가 확실함)
2. **보안 도구가 희소함:** sandbox 카테고리는 9개뿐 — 보안/권한/감사류 플러그인의 공급이 부족합니다
3. **비전/기억이 핫스팟:** 하지만 경쟁이 치열하므로, 차별화(버티컬 시나리오/기업)가 트렌드를 따라가는 것보다 낫습니다

여러분이 도구를 선택 중이라면

1. **차별화 베팅 포인트:** 전체 session log + 모든 것이 플러그인 + 샌드박스/권한 → 기업/평가/규제 준수 시나리오(데이터/디스커션 이중 검증의 우위 영역)
2. **리스크 포인트:** Windows/원격 시나리오는 현재 아픈 지점입니다, 팀이 Windows 위주라면 평가가 필요합니다
3. **비용 우위:** 캐시 적중 97% × 할인, 비용 투명화 도구 체인이 이미 갖춰짐(제14장)

여러분이 관망 중이라면

- **생태계 초기 보너스가 여전히 있습니다:** 4000+개 플러그인이지만 버티컬 영역(금융 8/데이터베이스 14/오디오 19)은 여전히 거의 공백입니다 — 스토어 오픈이 오히려 "공백 찾기"를 쉽게 만들어, 진입 기간이 명확합니다

- **선점자의 안내심:** 공식 rc 반복이 빨라, 능력 공백은 계속 나타날 것입니다

15.5 본 리포트의 방법론 안내

- 데이터 측면(1804개 저장소 전체 목록 + 4000+ 스토어 등재)의 기준은 이미 명시했습니다(README에 dsh가 언급됨, 일부 진짜가 아닌 플러그인 포함) — **방향성 참고이지, 정밀한 통계가 아닙니다**
- 정성적 측면(2400+개 스레드 + 225개 게시물 응답)은 본 핸드북의 지속적인 관찰에서 나왔습니다 — 스레드 번호는 모두 추적 가능합니다
- 두 측면을 대조해, "데이터와 디스커션이 모두 가리키는" 부분을 결론으로 채택해 한쪽으로 치우친 편향을 줄였습니다

생태계는 계속 변하고 있으며, 본 리포트도 디스커션에 맞춰 지속적으로 업데이트될 것입니다(반영 파이프라인 참고).

제 16 장: Claude Code에서 이전하기

본 장의 목표: 쌓아온 것을 가지고 이사하는 것이지, 처음부터 다시 짓는 것이 아닙니다. 각 유형의 자산마다 'DSH에서는 어떻게 되는지'에 대한 실측 결론을 제공하며, 모두 0.1.0-rc.6 소스 코드 기준으로 항목별로 검증했습니다.

TL;DR(본 장 핵심, 30초 버전)

1. **절반은 공짜입니다:** 프로젝트 CLAUDE.md는 옮길 필요 없음(DSH가 네이티브로 읽음), SKILL.md 형식은 그대로 호환됨
2. **절반은 기계적입니다:** .mcp.json은 무손실 변환 가능(도구 이름 mcp__server__tool이 양쪽에서 완전히 동일), hooks는 공식 브릿지가 있음
3. **자동화:** npx dsh-movein으로 이사 목록 미리보기, --apply로 실제 적용; 대화 이력은 dsh-chat-import 사용
4. **이사 전에 먼저 계산해보기:** 스킬 카탈로그는 스킬 하나당 요청마다 약 28토큰 소비, 여러분이 쓰는 것만 옮기고 가진 것 전부를 옮기지 마세요

16.1 자산 대조표(실측)

Claude Code 자산	DSH 호환성	실제 상황
프로젝트 CLAUDE.md	네이티브, 수정 없음	instructionFileCandidates가 기본으로 CLAUDE.md를 포함, 프로젝트 루트에서 cwd까지 자동으로 찾음
전역 ~/.claude/CLAUDE.md	심볼릭 링크 하나면 됨	전역 위치는 \$DSH_HOME/AGENTS.md만 인식하므로, 링크만 걸면 됨
스킬(SKILL.md)	형식 그대로 호환	프론트매터 메타데이터는 개방형 객체로 파싱, 알 수 없는 키(allowed-tools 등)는 무시됨. 주의: .claude/skills는 DSH의 기본 스킬 루트가 아니므로, ~/.dsh/skills나 <프로젝트>/.dsh/skills에 두어야 함
MCP(.mcp.json)	무손실 기계 변환	서버마다 dsh-mcp-client 설정 한 줄(stdio와 streamable-http), 도구 이름이 양쪽에서 동일하므로 이를 참조하는 스킬은 수정할 필요 없음
hooks	공식 브릿지, 일부 이벤트만	dsh-hooks-claude-code가 기존

		설정을 그대로 실행, 30개 이벤트 중 7개만 매핑됨, command 타입만 지원. 트러블: matcher가 DSH 도구 이름의 대소문자를 구분함, Bash는 소문자 bash 도구에 매칭되지 않아 보안 hook이 조용히 실패할 수 있음(공식 디스커션 #582), 수정 전에는 matcher를 소문자로 작성할 것
권한 규칙	네이티브 아님, 브릿지 가능	DSH는 3단계의 대략적인 프리셋만 있음. deny/ask는 tools/pre-execute에서 강제 실행 가능(커뮤니티 플러그인), allow는 대응물이 없음
서브에이전트(.claude/agents)	직접 가져올 수 없음	DSH 프리셋은 markdown이 아니라 agent.cordis.yml 디렉터리이며, 현실적인 경로는 스킴으로 변환하는 것(프론트매터 메타데이터가 거의 동일함)
슬래시 명령(.claude/commands)	파일 형태의 대응물 없음	DSH 명령은 코드로 등록되며, 사용자가 호출 가능한 스킴이 파일 레벨의 대체재임
세션	가장 어려움, 직접 작성하지 말 것	세션 파일은 v0 형식 + zstd frame + 엄격한 이벤트 검증을 사용하며, 공식은 호환성을 명시적으로 보장하지 않음. 이전 세션은 dsh-chat-import(https://github.com/Nwflower/dsh-chat-import)를 사용(13개 소스 지원, 역방향 내보내기 가능)

16.2 자동화된 이사

```
npx dsh-movein          # 미리보기, 이사 목록 출력, 어떤 파일도 쓰지 않음
npx dsh-movein --apply  # 실제 적용
npx dsh-movein --reverse # DSH에서 새로 만든 스킴을 Claude Code로 되돌리기(양쪽 사용)
```

플러그인으로 설치해 agent에게 대신 시킬 수도 있습니다: `dsh plugin --profile web add dsh-movein`, 그다음 대화에서 "내 Claude Code 설정을 옮겨줘"라고 말하면 됩니다.

권한 규칙은 **이전 차이 리포트**를 출력합니다(몇 개가 그대로 적용되는지, 몇 개가 매핑되지 않는지 항목별로 나열), 조용히 변환하지 않습니다. 이사할 때마다 `~/.dsh/movein-manifest.json`에 출처와 도착지를 기록합니다.

16.3 커뮤니티가 실제로 겪은 트러블(시작 관련)

1. 해석되지 않는 패키지를 patch에 써넣으면 dsh 시작이 바로 실패합니다(plugin tree failed to load, 경고가 아니라 fatal입니다). 먼저 패키지를 설치하고, 설치가 성공한 후에 설정 줄을 작성하세요.
2. 주변 패키지의 npm latest 태그가 코어보다 뒤쳐질 수 있습니다(hooks 브릿지가 rc.5였는데 코어는 이미 rc.6이었던 적 있음), 설치하는 호스트 dsh 버전을 기준으로 고정하세요.
3. **dsh-hook-protocol**은 hooks 브릿지의 peer 의존성입니다, 호스트 설치에는 포함되지 않으므로 함께 설치해야 합니다.

16.4 이사 전에 먼저 계산해보기

스킬 카탈로그는 system-reminder로 매 요청마다 주입됩니다: 고정 래핑이 143토큰, 스킬 하나당 약 28토큰(96자 설명 기준). 스킬 129개의 설정은 요청마다 약 3.8k 토큰을 잡아지며, 캐시는 비용을 절약해주지만 컨텍스트 원도우는 절약해주지 못합니다. **여러분이 쓰는 스킬을 옮기세요, 가진 스킬 전부가 아니라** — 설명은 짧게 작성하세요. 재현 스크립트와 방법은 dsh-movein의 토큰 청구서 실측

(<https://github.com/sjh9714/dsh-movein/blob/main/docs/token-bill.md>) 참고.

부록 A: 용어집과 명령어 빠른 참조

용어집

용어	한 줄 설명
Harness	모델 바깥을 감싸는 엔지니어링 레이어: 세션, 도구, 컨텍스트, 루프 제어
dsh	DeepSeek Harness의 명령줄 이름(dsh 명령어)
Profile	실행 가능한 형태 하나(web / headless / 커스텀) = bundle 스택 + patch 레이어
Bundle	플러그인 집합 하나(공식: dsh-base, dsh-web-app, dsh-headless)
Cordis	dsh 하부의 플러그인 컨테이너(의존성 주입, 이벤트, 생명주기)
플러그인(Plugin)	cordis 플러그인; host 반쪽(Node)과 client 반쪽(브라우저)을 동시에 가질 수 있음
host 반쪽 / client 반쪽	플러그인의 Node 쪽(도구/서비스/이벤트)과 브라우저 쪽(UI) 두 얼굴
확장 포인트	공식이 제공하는 훅(agent/request, settings, conversationEvents, slots)
agent/request waterfall	매 단계 모델 요청 전에 설정을 바꿀 수 있는 이벤트 체인(플러그인이 여기서 reasoningEffort 등을 주입)
reasoning_effort	사고 강도(공식 어댑터 off/high/max; low는 실측 게이트웨이 강도) — 속도와 품질의 트레이드오프 다이얼
headless	일회성 CLI 작업 모드(dsh --profile headless "작업")
compaction	긴 대화 컨텍스트 압축
subagent	서브에이전트(작업 병렬 위임)
MCP	Model Context Protocol(외부 도구 서버 연동)
workflow	다단계 결정론적 워크플로우 오케스트레이션
locations	도구가 반환하는 파일 경로(산출물 추적을 구동)
extension point	공식 훅(agent/request 등), 플러그인 연결 지점
waterfall	이벤트 체인: 리스너가 설정을 바꿔 다음 리스너에게 전달(agent/request가 이를 사용해 주입)
context cache	DeepSeek 프롬프트 캐시(반복 입력은 할인 가격으로 과금)
캐시 적중률	입력 토큰이 캐시 가격으로 처리되는 비율(dsh 실측 97%까지 달성)
TUI	터미널 UI(공식 미내장, 플러그인 필요)
turn	대화의 한 라운드(사용자+어시스턴트+도구 호출)
step	turn 안의 한 추론 단계
guard	루프 위생/도구 타임아웃 플러그인

skill	스킬(skill-catalog가 컨텍스트에 주입, 모델이 필요에 따라 호출)
inject	cordis 의존성 주입(플러그인이 필요한 서비스를 선언)
산출물 파일	모델이 생성/수정한 파일(대화 끝의 열람 가능한 칩)
sandbox	명령 실행의 격리 샌드박스
rc	프리릴리스 버전(0.1.0-rc.6), 반복이 빠르고 파괴적 변경이 있을 수 있음

명령어 빠른 참조

dsh 핵심

```
dsh web # Web UI 시작
dsh --profile headless "작업" # 일회성 작업(스크립트/CI)
dsh --version # 버전
dsh --dump-config # 합성 설정 트리
dsh plugin --profile <name> add <pkg> # 플러그인 설치
```

환경

```
node --version # ≥ 22 필요
npm install -g @deepseek-ai/dsh # 전역 설치
npx -y @deepseek-ai/dsh web # 설치 없이 실행
```

트러블슈팅

```
netstat -ano | findstr 3080 # 포트 점유(Windows)
taskkill /PID <pid> /F # 프로세스 종료
cat ~/.dsh/settings.yaml # 전역 설정(모델/추론 강도)
```

플러그인 개발

```
# profile에 장착(web을 예로)
# ① package.json에 의존성 추가: "pkg": "link:C:\\path\\to\\pkg"
# ② cordis.patch.yaml에 추가:
#   - insert:
#     - id: <플러그인id>
#     name: <npm패키지명>
cd ~/.dsh/profiles/web && pnpm install # 설치
```

설정 참조(settings.yaml)

```
agent-default-model:  
  model: deepseek-v4-flash      # 또는 deepseek-v4-pro  
  reasoningEffort: high        # off(사고 끄기/최고 속도) / high(기본값) / max(최고 성능)
```

부록 B: 공식 패키지 빠른 참조 총정리 · **부록 C:** 동일 모델 다중 Agent 실측

부록 B: 공식 패키지 빠른 참조 총정리(@deepseek-ai/*)

이 목록은 「알려진 패키지」 목록입니다. 백서 본문에서 실측/언급되었고 npm registry에서 확인 가능한 공식 패키지(@deepseek-ai/dsh-*와@deepseek-ai/cordis)를 수록합니다. 패키지 설명은 npm view 실측 결과에서 가져왔습니다. dsh는 0.1.0-rc 빠른 반복 시기이므로 **패키지 이름과 설명은 버전에 따라 업데이트됩니다** — 공식 저장소 packages/AGENTS.md나 npm 최신 설명과 다르다면 공식을 기준으로 삼으세요. 확인하지 못한 패키지는 「추후 보완」으로 표시했으며, 패키지 이름을 지어내지 않았습니다.

패키지 수 기준: 이 목록은 npm으로 확인된 **핵심 패키지 33개**를 수록합니다(CLI 직접 의존성 53개 + 계열 전체 계시 수 221개, 제8장 8.1절 설명 참고); 백서/공식이 말하는 「60+ 능력 패키지」는 저장소 packages/ 아래의 모든 하위 디렉터리를 가리키며, 전체 목록은 공식 저장소 packages/README.md(47개 그룹 공식 표)에서 확인할 수 있습니다.

패키지 이름과 저장소 레이아웃의 대응 관계: npm 패키지@deepseek-ai/dsh-xxx ≈ 공식 저장소 packages/<group>/xxx(예: dsh-tool-fs ↔ fs/tool-fs).

검증 방법: npm view @deepseek-ai/<name> description(2026-08 스냅샷).

능력 영역 값: 도구(모델이 호출 가능한 도구/실행 파이프라인) · 컨텍스트(요청 컨텍스트 조립과 압축) · 세션(세션 영속화/제목/텔레메트리) · 서브에이전트(하위 작업 위임) · MCP(외부 도구 서버 연동) · 워크플로우(다단계 오케스트레이션) · 보안(샌드박스/승인/루프 위생) · LLM(모델 연동과 전략) · UI(브라우저 인터페이스와 클라이언트 서비스) · 핵심(CLI/골격 bundle, 위 능력 영역에 속하지 않음).

핵심(Core): CLI와 골격 bundle

dsh의 "기반": @deepseek-ai/dsh 하나만 설치하면 바로 실행되며, profile은 내장 bundle 스택으로 쌓입니다(dsh-base → dsh-headless / dsh-web-app).

패키지 이름	용도	능력 영역
@deepseek-ai/dsh	dsh CLI: profile 시작, 플러그인 관리, 브라우저 UI 별칭(npx -y @deepseek-ai/dsh web)	핵심
@deepseek-ai/dsh-base	공유 dsh 코어 profile bundle: 각 profile의 첫 번째 패치 레이어로, 빈 profile 루트에 기본 플러그인 줄을 삽입	핵심
@deepseek-ai/dsh-headless	headless 일회성 bundle:	핵심

	Host/HTTP/브라우저 레이어 없는 직접적인 core Agent/Session 실행기	
@deepseek-ai/dsh-agent	Agent 인터페이스, 레지스트리, 발신자 스코프, 이벤트 어휘(플러그인 개발이 직접 의존하는 경우 많음)	핵심
@deepseek-ai/dsh-web-app	dsh 브라우저 쪽 bundle: web patch 레이어(프론트엔드 dist 서빙, web 쪽 프롬프트, bash 런타임 변수, URL 행)	핵심
@deepseek-ai/cordis	하부 메타 프레임워크: 현대적 JavaScript 앱을 위한 의존성 주입/이벤트/생명주기 컨테이너(dsh의 플러그인 기반)	핵심

도구(Tools): 모델이 호출 가능한 능력

모델이 실제로 보는 도구 이름은 짧은 동사입니다

(read/write/grep/glob/edit/bash/web_search/todo_write...), 하부는 이 그룹의 패키지들이 구현을 제공합니다.

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-tools	도구 레지스트리와 실행 파이프라인(tool registry + execution pipeline)	도구
@deepseek-ai/dsh-fs	추상 파일 시스템 능력 seam(ctx.fs): 어휘 타입, FileSystem 서비스(텍스트 IO + 선택적 버전 가드 원자적 쓰기), fs/* 정책 이벤트 어휘	도구
@deepseek-ai/dsh-tool-fs	모델 쪽 파일 시스템 도구(read, write, edit), ctx.fs 기반	도구
@deepseek-ai/dsh-tool-fs-search	모델 쪽 파일 탐색 도구(glob, grep), 번들된 ripgrep 바이너리 내장	도구
@deepseek-ai/dsh-tool-str-replace-editor	모델 쪽 편집 도구: 보기, 생성, 리터럴 교체, 줄 삽입(도구 결과에 locations 포함 → 산출물 추적)	도구
@deepseek-ai/dsh-shell	추상 bash 실행기 seam(ctx.shell)	도구
@deepseek-ai/dsh-tool-bash	모델 쪽 bash 도구, 선택적으로 범용 백그라운드 작업과 샌드박스 업그레이드 지원	도구
@deepseek-ai/dsh-web	추상 web 접근 능력 seam(ctx.web): search/fetch 프로바이더 레지스트리, 등록 순서와 무관한 선택, 요청/결과 어	도구

	취, WebError 분류	
@deepseek-ai/dsh-tool-web	모델 쪽 web 도구(web_search, web_fetch), ctx.web 기반	도구
@deepseek-ai/dsh-tool-todo	모델 쪽 할 일 도구 todo_write, 이벤트 소싱 세션 로그 기반	도구
@deepseek-ai/dsh-tool-skill	모델 쪽 스킬 로딩 도구(skill 호출 엔트리)	도구

컨텍스트(Context): 컨텍스트 조립과 압축

"무엇을 담을지"(시스템 프롬프트 + 스킬 카탈로그 + 이력 + 도구 스키마)와 "담을 수 없을 때 어떻게 압축할지".

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-system-prompt	시스템 프롬프트 조립 레지스트리(공식 플러그인이 <code>systemPrompt.section()</code> 으로 프롬프트 조각을 등록)	컨텍스트
@deepseek-ai/dsh-compaction	추상 압축 서비스 <code>seam(ctx.compaction)</code>	컨텍스트
@deepseek-ai/dsh-compaction-basic	토큰 계량 기반 압축 전략 + LLM 요약 백엔드(오버플로우 감지 → 다듬기 → 요약)	컨텍스트
@deepseek-ai/dsh-skill	Agent 스킬 프로바이더 레지스트리(스킬 카탈로그가 컨텍스트에 주입되어 모델이 필요에 따라 호출)	컨텍스트

세션(Session): 영속화, 제목, 텔레메트리

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-session	이벤트 소싱 세션 저장소(event-sourced session store)	세션
@deepseek-ai/dsh-session-title	로그 기반 세션 제목 서비스와 프로바이더 레지스트리	세션
@deepseek-ai/dsh-session-telemetry	텔레메트리 <code>seam</code> : 세션 이벤트 캡처, 프로젝트션, 마스킹, 리포팅 백엔드로 전달	세션

서브에이전트(Subagent): 하위 작업 위임

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-subagent	추상 서브에이전트	서브에이전트

	seam(ctx.subagents): 서브에이전트를 위임하는 명명된 프로바이더 레지스트리	
--	---	--

MCP: 외부 도구 서버 연동

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-mcp-client	MCP 클라이언트 브릿지: MCP 서버에 연결해 그 도구를 ctx.tools에 등록	MCP

워크플로우(Workflow): 다단계 결정론적 오케스트레이션

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-workflow	워크플로우 능력 seam: ctx.workflows 서비스, run 어휘, workflow/* 이벤트	워크플로우

보안(Safety): 샌드박스, 승인, 루프 위생

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-sandbox	추상 프로세스 샌드박스 seam(ctx.sandbox): 동일 세계 격리 어휘 + SandboxProvider 계약	보안
guard/*	루프 위생과 도구 타임아웃(백서 8.5절 언급; npm의 구체적 패키지 이름은 추후 보완)	보안
interaction/*	권한/승인(위험한 작업 확인; npm의 구체적 패키지 이름은 추후 보완)	보안

LLM: 모델 연동과 전략

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-llm	Provider 독립적인 LLM 서비스 인터페이스	LLM
@deepseek-ai/dsh-llm-deepseek	DeepSeek chat-completions 어댑터 (기본 provider=deepseek-official, off/high/max 강도 수용)	LLM
@deepseek-ai/dsh-llm-retry	Provider 라우팅되는 LLM 요청 재시도 전략	LLM

UI: 브라우저 인터페이스와 클라이언트 서비스

패키지 이름	용도	능력 영역
@deepseek-ai/dsh-client-runtime	클라이언트 코어 서비스: SlotsService, SessionsService(스코프 트리 + 객체 레이어)	UI
@deepseek-ai/dsh-client-locale	언어팩 플러그인: Host 쪽 zh/en 선호도, 브라우저 쪽 폴백, 언어 스냅샷, 타입이 있는 네임스페이스 딕셔너리	UI
ui-conversation/ui-tool 등	Web UI 각 구성요소(백서 8.1절에서 client/* 언급; npm의 구체적 패키지 이름은 추후 보완)	UI

이름은 알려졌지만 npm에 게시되지 않은 패키지(과거 404 패키지)

아래 패키지 이름은 백서의 트러블슈팅 기록에만 존재하며, **npm에서 설치할 수 없습니다**(rc.1 시대 의존성 단절의 근본 원인), 404를 만나는 것이 정상입니다:

패키지 이름	설명
@deepseek-ai/dsh-pty	rc.1 시대 의존성, 게시된 적 없음(pnpm dlx 404의 근본 원인, 제2장 FAQ 참고)
@deepseek-ai/dsh-type-meta	rc.1 시대 의존성, 게시된 적 없음(rc.1 의존성 단절의 근본 원인, 제3장 3.5절 참고)

회피법: 의존성은 통일해서 ^0.1.0-rc.6 라인을 사용하세요.

관련 챕터: 제8장 · 도구와 컨텍스트 시스템(60+ 능력 패키지 지도) · 제9장 · MCP·서브에이전트·워크플로우 · 용어 집과 명령어 빠른 참조

부록 C: 동일 모델 × 다른 Agent 실측 비교(Benchmark)

실측일: 2026-08-13 / 모델: deepseek-v4-flash(같은 opencode-go 게이트웨이, 같은 API key) / Agent: dsh / opencode / omp

목적: 모델 변수를 통제하고, Agent 엔지니어링 레이어의 차이를 비교(프롬프트, 도구 체인, 라운드 관리).

확장: 2026-08-13에 T4(markdown 리포트 생성) / T5(다중 파일 소규모 리팩터링) 추가, 샘플 3회 → 5개 작업 × 3회.

방법

항목	설명
모델	deepseek-v4-flash(세 agent 모두 opencode-go 게이트웨이 + 같은 key, 공정한 대조)
Agent	dsh(headless profile), opencode(opencode run), omp(--print 비대화형)
작업	T1 파일 생성 / T2 검색 통계 / T3 버그 수정 + 검증 / T4 데이터 읽고 리포트 생성 + 자체 검토 / T5 다중 파일 리팩터링 + 테스트 통과
환경	Windows 11 + Node 24, 독립 작업 디렉터리, 사전 컨텍스트 없음
측정	소요 시간(초) + 정확성(산출물/출력 수동 확인)

작업 설명(T4/T5는 합성 데이터, 개인정보 리스크 없음):

작업	입력	요구사항	수동 확인 기준
T4 리포트 생성	data.json(합성 판매 기록 4건)	report.md 생성: 데이터 표 + 요약(총 판매량/총 매출/최다 판매), 자체 검토	요약 숫자가 데이터와 일치 (397 / 130853 / 이어폰)
T5 다중 파일 리팩터링	mathops.py + 이를 의존하는 main.py(assert 포함)	multiply의 곱셈 로직을 _multiply 헬퍼 함수로 추출, main.py는 수정하지 않음, python main.py 통과	_multiply가 나타나고 multiply가 위임 호출, assert 전부 통과

결과(3회 샘플링, 중앙값 취함)

Agent	T1 파일 생성	T2 검색 통계	T3 버그 수정+검증	T4 리포트 생성	T5 다중 파일 리팩터링	총계(중앙값)	정확도
omp	10초	11초	15초	21초	13초	70초	45/45
dsh	22초	15초	48초	26초	19초	130초	45/45
opencode	35초	37초	42초	30초	28초	172초	45/45

총계 = 5개 작업 중앙값의 합, 확장 전(T1-T3만)은 omp 36초 / dsh 85초 / opencode 114초.

원본 3회 측정(초):

Agent	라운드	T1	T2	T3	T4	T5
dsh	1 / 2 / 3	11 / 27 / 22	11 / 15 / 22	27 / 48 / 74	30 / 26 / 25	19 / 25 / 18
opencode	1 / 2 / 3	18 / 35 / 35	19 / 37 / 37	50 / 41 / 42	30 / 49 / 30	28 / 28 / 32
omp	1 / 2 / 3	9 / 10 / 12	9 / 11 / 12	13 / 33 / 15	23 / 21 / 17	16 / 12 / 13

해석

1. **정확도는 세 곳 모두 동일(45/45)** — 5개 작업 × 3회 모두 정확. 같은 모델 기준으로, Agent 엔지니어링 레이어의 차이는 주로 **능력의 상한**이 아니라 **효율**에서 나타납니다; 문서 생성과 다중 파일 리팩터링이라는 두 가지 새 작업 유형을 추가해도 정확도 차이는 나타나지 않았습니다.
2. **총 소요 시간 순서는 안정적**: omp < dsh < opencode(3회 모두 일치, 중앙값 합계 70초 / 130초 / 172초).
3. **작업 유형이 효율 차이에 미치는 영향**(이번 확장의 핵심 발견) — 소요 시간 패턴으로 5개 작업을 세 부류로 나눌 수 있습니다:
 - **단일 단계 경량**(T1 파일 생성 / T2 검색 통계): agent 간 격차가 **가장 큼**. opencode 35/37초 vs omp 10/11초(약 **3.4배**). 작업이 단순할수록 순수한 "시작 + 도구 호출 한 번"의 오버헤드에 가까워져, opencode의 시작/프레임워크 오버헤드 비중이 더 커집니다.
 - **다단계 실행 + 검증**(T3 버그 수정 + 검증 실행 / T5 다중 파일 리팩터링 + 테스트): dsh와 opencode의 격차가 이 유형에서 **뚜렷이 좁혀집니다**(T5: 19초 vs 28초; T3: 48초 vs 42초로 오히려 역전). 다중 도구 체인 시나리오에서는 양쪽 모두 파일을 여러 번 읽고/수정하고/명령을 실행해야 하므로, 프레임워크 레이어의 "매 단계 사고" 비용이 시작 오버헤드 차이를 상쇄합니다; dsh의 우위는 T5에서 가장 뚜렷하고(19초, omp의 13초에 근접), T3의 버그 위치 확인은 여전히 dsh를 다소 느리게 만듭니다.
 - **문서 생성**(T4 데이터 읽기 → 리포트 작성 → 자체 검토): agent 간 격차가 **가장 작음**(omp 21초 / dsh 26초 / opencode 30초, 약 1.4배). 리포트 생성은 "한 번 읽고 + 한 번 긴 텍스트를 쓰는" 루프로, 소요 시간이 주로 모델의 **출력 토큰 수**에 좌우되며, Agent 엔지니어링 레이어가 최적화할 수 있는 도구 왕복이 적어 세 곳이 비슷해집니다.

4. **직관에 반하는 포인트: T5(리팩터링+테스트)가 T4(리포트 작성)보다 대체로 더 빠름**(dsh 19초<26초, omp 13초<21초; opencode만 28초~30초). 이유: 리팩터링은 "짧은 텍스트 코드 편집 + 초 단위 python main.py 검증"의 루프로 매 단계 모델 사고가 짧지만; markdown 리포트 작성은 한 번의 비교적 긴 텍스트 생성이 필요해 출력 시간 비중이 높습니다 — 제6장의 "도구 체인 작업 시간의 90%가 모델 사고/생성에 쓰인다"는 성능 모델과 일치합니다.
5. **omp의 우위 폭은 작업 유형에 따라 달라집니다**: 검색 통계류가 가장 크고(T2는 opencode의 3.4배), 문서 생성류가 가장 작습니다(T4는 겨우 1.4배). 이는 omp의 강점이 주로 **라운드가 적고 도구 호출이 빠른 것**에 있지, 모델 자체에 있지 않음을 보여줍니다; 작업이 긴 텍스트 생성 위주가 되면, 어떤 agent의 엔지니어링 레이어도 모델 출력 시간을 더 압축하기 어렵습니다.
6. **변동성 안내**: opencode는 T4 2회차에서 49초의 이상치가 나타났고(나머지 두 번은 30초), dsh는 T3에서 변동이 가장 큼니다(27→74초). 여러 회차의 중앙값은 이런 네트워크/도구 체인 흔들림을 흡수할 수 있지만, 단일 회차 비교는 신중해야 합니다.
7. **dsh의 포지셔닝**: dsh는 중간이며, headless는 "런타임 + 플러그인 생태계"입니다 — **같은 모델이라도, 플러그인(예: 속도 개선 플러그인으로 추론 강도 낮추기)을 통해 소요 시간을 더 최적화할 수 있습니다**(제6장 참고); T5 실측이 이미 omp에 근접해, 다중 파일 리팩터링류 작업이 dsh의 상대적 강점임을 보여줍니다.
8. **샘플 경고**: 3회 × 5개 작업, 같은 게이트웨이 같은 key, 네트워크 흔들림 포함 — 방향은 신뢰할 수 있지만, 절대값은 환경에 따라 변동할 것입니다.

재현 가능

```
# 세 agent의 명령어 (독립 디렉터리)
dsh --profile headless "작업"
opencode run "작업" --model opencode-go/deepseek-v4-flash
omp "작업" --model deepseek-v4-flash --print
```

T4/T5 작업 원문(T1-T3는 이전 버전 기록 참고):

T4: 현재 디렉터리의 data.json 데이터를 읽고, markdown 리포트 report.md를 생성해: 각 행 데이터의 표 (제품/판매량/단가 3열)와 요약 (총 판매량, 총 매출, 최대 판매 제품)을 포함하고, 생성 후 요약 숫자가 data.json과 일치하는지 스스로 검토한 다음, 완료되면 "생성 완료"라고 답해

T5: mathops.py를 리팩터링해: multiply 함수 안의 곱셈 로직을 비공개 헬퍼 함수 _multiply(a, b)로 추출하고, multiply는 _multiply를 호출하도록 바꿔; main.py는 mathops.py에 의존하고 assert 테스트를 포함하니

`main.py`는 수정하지 마; 리팩터링 후 `python main.py`를 실행해 모든 `assert`가 통과하는지 검증하고, 완료되면 "리팩터링 완료"라고 답해

전체 작업 정의, 합성 데이터, 산출물: `~/agent-bench/bench-ext/` (T4/T5는 agent별 독립 디렉터리); T1-T3는 `~/agent-bench/` 참고. 백서 저장소 내 `benchmark/` 디렉터리는 계획 중입니다.